



UNIVERSIDADE ESTADUAL DO CEARÁ
CENTRO DE CIÊNCIAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
DOUTORADO EM CIÊNCIA DA COMPUTAÇÃO

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES
VEICULARES 5G

FORTALEZA – CEARÁ

2026

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES VEICULARES
5G

Proposta de qualificação apresentada ao Curso de Doutorado em Ciência da Computação do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências e Tecnologia da Universidade Estadual do Ceará, como requisito parcial para a defesa de tese.
Orientador: Dr. Joaquim Celestino Júnior

FORTALEZA – CEARÁ

2026

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES VEICULARES
5G

Proposta de qualificação apresentada ao Curso de Doutorado em Ciência da Computação do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências e Tecnologia da Universidade Estadual do Ceará, como requisito parcial para a defesa de tese.

Aprovada em:

BANCA EXAMINADORA

Dr. Joaquim Celestino Júnior (Orientador)
Universidade Estadual do Ceará – UECE

Membro da Banca Dois
Universidade do Membro da Banca Dois -
SIGLA

Membro da Banca Três
Universidade do Membro da Banca Três -
SIGLA

Membro da Banca Quatro
Universidade do Membro da Banca Quatro -
SIGLA

Membro da Banca Cinco
Universidade do Membro da Banca Cinco -
SIGLA

Membro da Banca Seis
Universidade do Membro da Banca Seis -
SIGLA

RESUMO

Este trabalho de qualificação propõe e avalia, uma solução para o problema de descarregamento de tarefas em redes de Computação de Borda Veicular (*Vehicular Edge Computing*) (VEC) combinando ranqueamento multicritério por *Technique for Order of Preference by Similarity to Ideal Solution* (TOPSIS) com Aprendizado por Reforço Profundo (*Deep Reinforcement Learning*) (DRL) baseado no algoritmo Ator-Crítico Suave (*Soft Actor-Critic*) (SAC). O desafio central é tomar decisões de descarregamento sob restrições de prazo (*deadline*) em ambientes com número variável de candidatos — Unidade de Borda de Estrada (*Road Side Unit*)s (RSUs) e veículos vizinhos —, o que impede, sem retreinamento, o uso direto de redes neurais com entrada de dimensão fixa. A principal contribuição técnica desta etapa é uma engenharia de estado compacta (vetor 6D) que encapsula as características normalizadas do melhor candidato identificado pelo TOPSIS, garantindo invariância à escala e estabilidade de aprendizado independentemente da densidade veicular. O TOPSIS avalia multicritérios como latência estimada, ocupação de fila, taxa de canal e consumo energético, reduzindo a complexidade do espaço de estados sem perda de informação crítica para a tomada de decisão. O uso do SAC é motivado pela sua regularização por entropia máxima, que previne colapso de política em ambientes não-estacionários, e pela eficiência amostral do aprendizado *off-policy*. Os experimentos, conduzidos em 658 cenários de simulação com comunicação Veículo-para-Tudo (*Vehicle-to-Everything*) (V2X), mostram que a combinação SAC-TOPSIS alcança taxa de cumprimento de prazo de 45,21%, *Success-Weighted Quality* (SWQ) de 0,1484 e função objetivo $J(\pi) = 0,2454$ — métricas superiores às de cinco outras famílias de algoritmos de DRL e nove heurísticas de referência avaliadas. Como extensão planejada, o TOPSIS será substituído por *Fuzzy-TOPSIS*, incorporando incerteza linguística nos critérios de avaliação dos candidatos, com o objetivo de aprimorar a robustez da representação de estado em cenários de alta mobilidade e informação imprecisa.

Palavras-chave: descarregamento de tarefas; computação de borda veicular; aprendizado por reforço profundo; SAC; TOPSIS; *Fuzzy-TOPSIS*; V2X; generalização *zero-shot*; engenharia de estado.

ABSTRACT

This qualification work proposes and evaluates a solution to the task offloading problem in Vehicular Edge Computing (VEC) networks by combining multicriteria ranking via TOPSIS (*Technique for Order of Preference by Similarity to Ideal Solution*) with Deep Reinforcement Learning (DRL) based on the Soft Actor-Critic (SAC) algorithm. The central challenge is making *offloading* decisions under deadline constraints in environments with a variable number of candidates — Road Side Units (RSUs) and neighboring vehicles — which prevents the direct use of fixed-input neural networks without retraining. The main technical contribution of this stage is a compact state engineering approach (6D vector) that encapsulates the normalized features of the best candidate identified by TOPSIS, guaranteeing scale-invariance and learning stability regardless of vehicle density. TOPSIS evaluates multicriteria such as estimated latency, queue occupancy, channel rate, and energy consumption, reducing the complexity of the state space without losing information critical for decision-making. The use of SAC is motivated by its maximum-entropy regularization, which prevents policy collapse in non-stationary environments, and by the sample efficiency of *off-policy* learning. Experiments conducted over 658 simulation scenarios with Vehicle-to-Everything (V2X) communication show that the SAC-TOPSIS combination achieves a deadline compliance rate of 45.21%, a *Success-Weighted Quality* (SWQ) of 0.1484, and an objective function value of $J(\pi) = 0.2454$ — metrics superior to those of five other families of DRL algorithms and nine reference heuristics evaluated. As a planned extension, TOPSIS will be replaced by *Fuzzy-TOPSIS*, incorporating linguistic uncertainty into the candidate evaluation criteria, with the aim of improving the robustness of state representation in high-mobility and imprecise-information scenarios.

Keywords: task offloading; vehicular edge computing; deep reinforcement learning; SAC; TOPSIS; *Fuzzy-TOPSIS*; V2X; zero-shot generalization; state engineering.

LISTA DE ILUSTRAÇÕES

Figura 1 – Formas de descarregamento de tarefas em VEC: total (<i>full</i>), parcial (<i>partial</i>) e adaptativo (<i>adaptive</i>), com seus trade-offs de confiabilidade e complexidade.	18
Figura 2 – Modelo de sistema básico <i>3rd Generation Partnership Project</i> (3GPP) evidenciando os dois grandes blocos operacionais do Quinta Geração de Redes Móveis (<i>Fifth Generation</i>) (5G).	26
Figura 3 – Integração do Computação de Borda de Múltiplos Acessos (<i>Multi-access Edge Computing</i>) (MEC) à rede 5G distribuindo a função Função de Plano de Usuário (<i>User Plane Function</i>) (UPF) localmente visando reduzir a latência de tráfego sensível.	27
Figura 4 – Arquitetura das comunicações 5G <i>Cellular Vehicle-to-Everything</i> (C-V2X): coexistência da interface celular Interface de Rádio Celular (Uu) (Veículo-para-Infraestrutura (<i>Vehicle-to-Infrastructure</i>) (V2I)/Veículo-para-Rede (<i>Vehicle-to-Network</i>) (V2N)) com a interface direta Interface de Comunicação Direta PC5 (PC5) <i>Sidelink</i> (Veículo-para-Veículo (<i>Vehicle-to-Vehicle</i>) (V2V)/Veículo-para-Pedestre (<i>Vehicle-to-Pedestrian</i>) (V2P)), integrando o núcleo Núcleo 5G (<i>5G Core</i>) (5GC), o servidor MEC de borda e os elementos veiculares e pedestres.	29
Figura 5 – Estrutura temporal básica da camada física em um <i>slot</i> do 5G Novo Rádio (<i>New Radio</i>) (NR) <i>Sidelink</i> , ilustrando a multiplexação dos canais físicos Canal Físico de Controle do Sidelink (<i>Physical Sidelink Control Channel</i>) (PSCCH), Canal Físico Compartilhado do Sidelink (<i>Physical Sidelink Shared Channel</i>) (PSSCH) e Canal Físico de Feedback do Sidelink (<i>Physical Sidelink Feedback Channel</i>) (PSFCH) (COX, 2025; GARCIA; AL., 2021).	29
Figura 6 – Abstração do Ecossistema Internet dos Veículos (<i>Internet of Vehicles</i>) (IoV): V2V, V2I, V2N e V2P em Cruzamento	31
Figura 7 – Arquitetura de Protocolos <i>Dedicated Short-Range Communications</i> (DSRC) / WAVE (IEEE 1609 / 802.11p)	32
Figura 8 – Arquitetura hierárquica em três camadas para computação em borda e nuvem.	34

Figura 9 – Fluxograma do processo decisório e ciclo de vida do descarregamento de tarefas.	35
Figura 10 – Ciclo básico de interação Agente-Ambiente no Aprendizado por Reforço (<i>Reinforcement Learning</i>) (RL). A cada passo, o agente observa o estado atual, escolhe uma ação, e o ambiente responde com uma recompensa e um novo estado.	37
Figura 11 – Sequência de interação em um Processo de Decisão de Markov (<i>Markov Decision Process</i>) (MDP).	38
Figura 12 – Dinâmica Ator-Crítico.	44
Figura 13 – Representação geométrica bidimensional do ranqueamento TOPSIS baseada em critérios de benefício.	46
Figura 14 – Distância de Alternativas para as Soluções Ideais no <i>Fuzzy</i>-TOPSIS . .	48
Figura 15 – Taxonomia adaptada para classificação dos trabalhos relacionados em descarregamento computacional veicular, com ênfase na modelagem de Aprendizado por Reforço Profundo (DRL) e ambiente de simulação. . .	49
Figura 16 – Representação do ambiente de simulação: um cruzamento veicular contemplando comunicação V2I, V2V e os principais componentes do cenário estocástico.	64
Figura 17 – Arquitetura da rede <i>Actor</i> (Perceptron Multicamadas (<i>Multilayer Perceptron</i>) (MLP) $6 \rightarrow 64 \rightarrow 64 \rightarrow 2$): o vetor de estado compacto S_t alimenta duas camadas ocultas com ativação ReLU; a camada Softmax final emite as probabilidades das ações a_0 (LOCAL) e a_1 (<i>OFFLOAD rank</i>₁).	70
Figura 18 – SWQ médio por política sobre 661 cenários de teste ($\alpha = 0,35$, $\beta = 0,35$, $\gamma = 0,30$). Barras em cinza representam heurísticas; barras escuras representam políticas DRL. A borda dourada indica a melhor política. Maior é melhor.	76

LISTA DE TABELAS

Tabela 2 – Resumo dos aspectos de Padrão de Comunicação, Servidor e Condições de Ambiente dos trabalhos relacionados	53
Tabela 3 – Classificação dos algoritmos clássicos de DRL quanto à adequação ao espaço de ação	57
Tabela 4 – Análise comparativa dos trabalhos relacionados por Função Objetivo, Estado, Recompensa e Ambiente Simulado	60
Tabela 5 – Distribuição dos 658 cenários de treinamento/teste por taxa de chegada.	65
Tabela 6 – Parâmetros físicos do ambiente simulado.	66
Tabela 7 – Hiperparâmetros do SAC compartilhados por todas as políticas DRL. .	66
Tabela 8 – Critérios e pesos do módulo TOPSIS.	68
Tabela 9 – Resultados das variantes do estudo de ablação avaliadas em 658 cenários de teste. A proposta completa é destacada em negrito.	74
Tabela 10 – <i>Ranking</i> completo das 25 políticas avaliadas em 658 cenários comuns (ordenado por SWQ decrescente). * = usa TOPSIS.	80
Tabela 11 – Ganho do TOPSIS por família de algoritmo (médias sobre 658 cenários comuns). ΔTaxa e ΔSWQ representam diferença absoluta; $\Delta J(\pi)$ é a diferença na função objetivo.	80
Tabela 12 – Atividades de pesquisa e cronograma de conclusão (2026–2027).	81
Tabela 13 – Exemplo de conversão de escala de tempo para uma tarefa do <i>trace</i> Alibaba.	92

LISTA DE ABREVIATURAS E SIGLAS

3GPP	<i>3rd Generation Partnership Project</i>
5G	Quinta Geração de Redes Móveis (<i>Fifth Generation</i>)
5GC	Núcleo 5G (<i>5G Core</i>)
6G	Sexta Geração de Redes Móveis (<i>Sixth Generation</i>)
A2C	Ator-Crítico com Vantagem (<i>Advantage Actor-Critic</i>)
AMF	Função de Gerenciamento de Acesso e Mobilidade (<i>Access and Mobility Management Function</i>)
ANATEL	Agência Nacional de Telecomunicações
B5G	Além do 5G (<i>Beyond 5G</i>)
BER	Taxa de Erro de Bit (<i>Bit Error Rate</i>)
C-MEC	Computação de Borda de Múltiplos Acessos Assistida por Nuvem (<i>Cloud-assisted Multi-access Edge Computing</i>)
C-V2X	<i>Cellular Vehicle-to-Everything</i>
CPU	Unidade Central de Processamento (<i>Central Processing Unit</i>)
CTSRAO	Otimização de Alocação de Recursos e Divisão de Tarefas Nuvem-Terminal (<i>Cloud-Terminal Task Splitting and Resource Allocation Optimization</i>)
CU	Unidade Central (<i>Central Unit</i>)
D3QN	Rede Q Profunda Dupla com Dueling (<i>Dueling Double Deep Q-Network</i>)
DDPG	Gradiente de Política Determinístico Profundo (<i>Deep Deterministic Policy Gradient</i>)
DDQN	Redes Q-Deep Duplas (<i>Double Deep Q-Network</i>)
DES	Simulação de Eventos Discretos (<i>Discrete Event Simulation</i>)
DNN	Redes Neurais Profundas (<i>Deep Neural Networks</i>)
DQN	Redes Q-Deep (<i>Deep Q-Network</i>)
DRL	Aprendizado por Reforço Profundo (<i>Deep Reinforcement Learning</i>)
DSRC	<i>Dedicated Short-Range Communications</i>
DSRC/IEEE	<i>Dedicated Short-Range Communications/Institute of Electrical and Electronics Engineers</i>
DT	Gêmeo Digital (<i>Digital Twin</i>)
DU	Unidade Distribuída (<i>Distributed Unit</i>)

DVFS	Escalonamento Dinâmico de Tensão e Frequência (<i>Dynamic Voltage and Frequency Scaling</i>)
eMBB	Banda Larga Móvel Melhorada (<i>enhanced Mobile Broadband</i>)
FCC	Comissão Federal de Comunicações (<i>Federal Communications Commission</i>)
FL	Aprendizado Federado (<i>Federated Learning</i>)
FOFF	<i>Fuzzy Offloading</i>
FR1	Faixa de Frequência 1 (<i>Frequency Range 1</i>)
FR2	Faixa de Frequência 2 (<i>Frequency Range 2</i>)
Fuzzy-TOPSIS	<i>Fuzzy-Technique for Order of Preference by Similarity to Ideal Solution</i>
GAN	Redes Adversárias Generativas (<i>Generative Adversarial Networks</i>)
gNB	Nó B de Próxima Geração (<i>Next-Generation Node B</i>)
GNN	Redes Neurais em Grafos (<i>Graph Neural Networks</i>)
GPU	Unidade de Processamento Gráfico (<i>Graphics Processing Unit</i>)
IA	Inteligência Artificial (<i>Artificial Intelligence</i>)
IoT	Internet das Coisas (<i>Internet of Things</i>)
IoV	Internet dos Veículos (<i>Internet of Vehicles</i>)
ISM	Industrial, Científica e Médica (<i>Industrial, Scientific and Medical</i>)
ITS	Sistemas de Transporte Inteligentes (<i>Intelligent Transport Systems</i>)
ITU	União Internacional de Telecomunicações (<i>International Telecommunication Union</i>)
JCT	Tempo de Conclusão de Tarefa (<i>Job Completion Time</i>)
L-MADDPG	MADDPG com Otimização de Lyapunov (<i>Lyapunov-based Multi-Agent Deep Deterministic Policy Gradient</i>)
LLM	Grande Modelo de Linguagem (<i>Large Language Model</i>)
MAC	Controle de Acesso ao Meio (<i>Medium Access Control</i>)
MADDPG	Ator-Crítico Determinístico Profundo Multiagente (<i>Multi-Agent Deep Deterministic Policy Gradient</i>)
MANET	Rede Móvel Ad hoc (<i>Mobile Ad hoc NETWORK</i>)
MARL	Aprendizado por Reforço Multiagente (<i>Multi-Agent Reinforcement Learning</i>)
MATD3	Multiagente Twin Delayed DDPG (<i>Multi-Agent Twin Delayed Deep Deterministic Policy Gradient</i>)
MCDM	Tomada de Decisão Multicritério (<i>Multiple-Criteria Decision-Making</i>)
MCS	Esquema de Modulação e Codificação (<i>Modulation and Coding Scheme</i>)

MDP	Processo de Decisão de Markov (<i>Markov Decision Process</i>)
MEC	Computação de Borda de Múltiplos Acessos (<i>Multi-access Edge Computing</i>)
MIMO	Múltiplas Entradas, Múltiplas Saídas (<i>Multiple-Input Multiple-Output</i>)
ML	Aprendizado de Máquina (<i>Machine Learning</i>)
MLP	Perceptron Multicamadas (<i>Multilayer Perceptron</i>)
mMTC	Comunicações Massivas do Tipo Máquina (<i>Massive Machine Type Communications</i>)
mmWave	Ondas Milimétricas (<i>Millimeter Wave</i>)
NG-RAN	Rede de Acesso por Rádio de Próxima Geração (<i>Next Generation Radio Access Network</i>)
NIS	Solução Ideal Negativa (<i>Negative Ideal Solution</i>)
NR	Novo Rádio (<i>New Radio</i>)
OBU	Unidade de Bordo (<i>On-Board Unit</i>)
OFDMA	Acesso Múltiplo por Divisão de Frequências Ortogonais (<i>Orthogonal Frequency-Division Multiple Access</i>)
P-DQN	Rede Q Profunda Parametrizada (<i>Parametrized Deep Q-Network</i>)
PC5	Interface de Comunicação Direta PC5
PDCP	Protocolo de Convergência de Protocolo de Dados (<i>Packet Data Convergence Protocol</i>)
PDQKM	DQN Preditivo e Emparelhamento de Kuhn-Munkres (<i>Predictive DQN and Kuhn-Munkres scheme</i>)
PHY	Camada Física (<i>Physical Layer</i>)
PIS	Solução Ideal Positiva (<i>Positive Ideal Solution</i>)
POMDP	Processo de Decisão de Markov Parcialmente Observável (<i>Partially Observable Markov Decision Process</i>)
PPO	Otimização de Política Proximal (<i>Proximal Policy Optimization</i>)
PRB	Bloco de Recurso Físico (<i>Physical Resource Block</i>)
PSCCH	Canal Físico de Controle do Sidelink (<i>Physical Sidelink Control Channel</i>)
PSFCH	Canal Físico de Feedback do Sidelink (<i>Physical Sidelink Feedback Channel</i>)
PSO	Otimização por Enxame de Partículas (<i>Particle Swarm Optimization</i>)
PSSCH	Canal Físico Compartilhado do Sidelink (<i>Physical Sidelink Shared Channel</i>)
QAM	Modulação de Amplitude em Quadratura (<i>Quadrature Amplitude Modulation</i>)

QDPG	Gradiente de Política com Q-Learning e DDPG (<i>Q-learning and Deep Deterministic Policy Gradient</i>)
QoS	Qualidade de Serviço (<i>Quality of Service</i>)
RB	Bloco de Recurso (<i>Resource Block</i>)
RIS	Superfície Inteligente Reconfigurável (<i>Reconfigurable Intelligent Surface</i>)
RL	Aprendizado por Reforço (<i>Reinforcement Learning</i>)
RLC	Controle de Enlace de Rádio (<i>Radio Link Control</i>)
RLHF	Aprendizado por Reforço a partir de Feedback Humano (<i>Reinforcement Learning from Human Feedback</i>)
RRC	Controle de Recursos de Rádio (<i>Radio Resource Control</i>)
RSU	Unidade de Borda de Estrada (<i>Road Side Unit</i>)
SAC	Ator-Crítico Suave (<i>Soft Actor-Critic</i>)
SARSA	<i>State-Action-Reward-State-Action</i>
SBA	Arquitetura Baseada em Serviços (<i>Service Based Architecture</i>)
SDAP	Protocolo de Adaptação de Dados de Serviço (<i>Service Data Adaptation Protocol</i>)
SDN	Redes Definidas por Software (<i>Software-Defined Networking</i>)
SIDDPG	DDPG com Iteração de Estados Internos Sequenciais (<i>Sequential Internal-state Deep Deterministic Policy Gradient</i>)
SINR	Relação Sinal-Interferência-e-Ruído (<i>Signal-to-Interference-plus-Noise Ratio</i>)
SMF	Função de Gerenciamento de Sessão (<i>Session Management Function</i>)
SOE	Eficiência Operacional do Sistema (<i>System Operation Efficiency</i>)
SUMO	Simulation of Urban MObility
TD2PG	<i>Task Descriptor Policy Gradient</i>
TD3	<i>Twin Delayed Deep Deterministic Policy Gradient</i>
TFN	Número Fuzzy Triangular (<i>Triangular Fuzzy Number</i>)
TL	Aprendizado por Transferência (<i>Transfer Learning</i>)
TOPSIS	<i>Technique for Order of Preference by Similarity to Ideal Solution</i>
UAV	Veículo Aéreo Não Tripulado (<i>Unmanned Aerial Vehicle</i>)
UE	Equipamento de Usuário (<i>User Equipment</i>)
UPF	Função de Plano de Usuário (<i>User Plane Function</i>)
URLLC	Comunicações Ultraconfiáveis de Baixa Latência (<i>Ultra-Reliable Low-Latency Communications</i>)
Uu	Interface de Rádio Celular

V2I	Veículo-para-Infraestrutura (<i>Vehicle-to-Infrastructure</i>)
V2N	Veículo-para-Rede (<i>Vehicle-to-Network</i>)
V2P	Veículo-para-Pedestre (<i>Vehicle-to-Pedestrian</i>)
V2V	Veículo-para-Veículo (<i>Vehicle-to-Vehicle</i>)
V2X	Veículo-para-Tudo (<i>Vehicle-to-Everything</i>)
VANET	Rede Veicular Ad hoc (<i>Vehicular Ad hoc NETwork</i>)
VCC	Computação em Nuvem Veicular (<i>Vehicular Cloud Computing</i>)
VEC	Computação de Borda Veicular (<i>Vehicular Edge Computing</i>)
VFC	Computação em Névoa Veicular (<i>Vehicular Fog Computing</i>)
WSM	Método de Soma Ponderada (<i>Weighted Sum Method</i>)

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Contextualização	15
1.2	Motivação	17
1.3	Definição do Problema	17
1.4	Questões de Pesquisa	19
1.5	Hipóteses	20
1.6	Objetivos	21
1.6.1	Objetivo Geral	21
1.6.2	Objetivos Específicos	21
1.7	Contribuições Esperadas	22
1.8	Publicação	23
1.9	Organização do Trabalho	23
2	FUNDAMENTAÇÃO TEÓRICA	25
2.1	Arquitetura e Modelagem 5G V2I e C-V2X	25
2.1.1	Interface Aérea 5G NR: Acesso Múltiplo por Divisão de Frequências Or- togonais (<i>Orthogonal Frequency-Division Multiple Access</i>) (OFDMA) e Adaptação de Enlace	27
2.1.2	Comunicação Veicular C-V2X e Interface <i>Sidelink</i> PC5	28
2.2	A Internet de Veículos (IoV) e as Redes Veiculares	30
2.3	Descarregamento de Tarefas	33
2.4	Aprendizado por Reforço	36
2.5	TOPSIS e Fuzzy-TOPSIS	45
3	REVISÃO DE LITERATURA	49
3.1	Padrão de Comunicação	49
3.1.1	Tecnologia	50
3.1.2	Servidor	50
3.2	Problema	54
3.2.1	Estratégia	54
3.3	Considerações Finais	62
4	PROPOSTA DE SOLUÇÃO	63
4.1	Ambiente	64

4.2	Cenários de Teste e Treinamento	65
4.3	Visão Geral da Arquitetura e Formulação do Problema	67
4.4	Complexidade Computacional	71
4.5	Estudo de Ablação	71
4.6	Análise dos Resultados	74
4.6.1	Métrica de Efetividade: <i>Success-Weighted Quality</i> (SWQ)	75
4.7	Por que o TOPSIS agrega valor ao SAC mas não ao Otimização de Política Proximal (<i>Proximal Policy Optimization</i>) (PPO)?	76
4.7.1	O problema do aprendizado redundante no PPO-TOPSIS	77
4.7.2	Colapso de política induzido pelo sinal TOPSIS	77
4.7.3	Assimetria de eficiência amostral com cenário único	78
4.7.4	Síntese: complementaridade estrutural SAC–TOPSIS	79
4.8	Avaliação Comparativa de Políticas	79
5	CRONOGRAMA DE PESQUISA E PLANO DE PUBLICAÇÃO	81
5.1	Cronograma de Pesquisa	81
5.2	Plano de Publicação	81
6	CONCLUSÕES	82
6.1	Considerações Finais	82
	REFERÊNCIAS	83
	APÊNDICE A – CONVERSÃO DE ESCALA DE TEMPO DO <i>TRACE</i> ALIBABA	91

1 INTRODUÇÃO

Este capítulo apresenta uma visão geral do trabalho, descrevendo o contexto e os desafios que o motivaram (Seção 1.1 e Seção 1.2). A partir disso, formaliza-se o problema de pesquisa (Seção 1.3), as questões norteadoras (Seção 1.4) e as hipóteses elencadas (Seção 1.5). Por fim, são traçados os objetivos (Seção 1.6), as contribuições esperadas (Seção 1.7), as publicações decorrentes da pesquisa (Seção 1.8) e a forma como o documento está estruturado (Seção 1.9).

1.1 Contextualização

A convergência entre tecnologias de comunicação, eletrificação veicular e condução autônoma tem moldado um novo paradigma para o setor de transportes. O conceito de IoV transforma veículos em sistemas computacionais sofisticados, capazes de viabilizar diversas aplicações de Sistemas de Transporte Inteligentes (*Intelligent Transport Systems*) (ITS) (RANI; SHARMA, 2023).

Veículos modernos são equipados com múltiplos sensores, unidades de processamento e interfaces de comunicação, permitindo a coleta e o processamento de grandes volumes de dados. Contudo, a complexidade das aplicações e a quantidade crescente de dados a serem processados podem exceder a capacidade computacional das Unidade de Bordo (*On-Board Unit*) (OBU) instaladas.

Uma solução imediata seria aumentar a capacidade computacional dessas OBUs, possibilitando maior processamento local. No entanto, o atual período de transição tecnológica resulta em cenários heterogêneos, no qual coexistem OBUs com diferentes capacidades computacionais (LUO *et al.*, 2024) e distintas tecnologias de comunicação (ZHENG *et al.*, 2015). Essa diversidade representa um desafio para a implementação de aplicações veiculares em larga escala, especialmente no contexto das cidades inteligentes (*Smart Cities*) (LUO *et al.*, 2024).

Nesse ambiente fortemente dinâmico e caótico, muitas aplicações, em particular aquelas voltadas à condução autônoma, impõem requisitos rigorosos de latência (HE *et al.*, 2025). A execução integral do processamento nos dispositivos embarcados frequentemente inviabiliza o atendimento a tais requisitos, devido às limitações físicas de energia e capacidade computacional (LUO *et al.*, 2024).

Para superar essas restrições, destaca-se a VEC, um paradigma onde servidores computacionalmente robustos são posicionados próximos aos veículos, reduzindo de forma

significativa a latência de processamento (LIU *et al.*, 2021). Tecnologias relacionadas, como a Computação em Nuvem Veicular (*Veicular Cloud Computing*) (VCC) (WHAIDUZZAMAN *et al.*, 2014) e a Computação em Névoa Veicular (*Veicular Fog Computing*) (VFC) (KESHARI; SINGH; MAURYA, 2022), ampliam o escopo do processamento distribuído, oferecendo suporte adicional em diferentes níveis de proximidade e escalabilidade.

O descarregamento de tarefas surge como o mecanismo central para operacionalizar a comunicação e o processamento colaborativo entre veículos e essas arquiteturas distribuídas. Essa técnica consiste em migrar tarefas computacionais para nós mais capazes, como servidores de borda ou outros veículos com maior poder de processamento (WANG *et al.*, 2022), possibilitando que veículos com recursos limitados executem aplicações veiculares complexas.

O principal desafio, entretanto, está em decidir quando e para onde descarregar as tarefas, considerando múltiplos fatores conflitantes, como tempo limite de processamento, custo energético, balanceamento de carga e qualidade do enlace de comunicação. Abordagens determinísticas, baseadas exclusivamente em heurísticas, apresentam desempenho limitado diante da alta variabilidade espacial e temporal do ambiente veicular (AHMED *et al.*, 2022). Mobilidade intensa, flutuações do canal e mudanças abruptas na carga computacional tornam as decisões estáticas rapidamente obsoletas, comprometendo a eficiência do sistema.

Essa limitação tem impulsionado a busca por soluções adaptativas e mais robustas, baseadas em aprendizado de máquina e sistemas inteligentes (MAO *et al.*, 2017; ZHANG *et al.*, 2018; ZHOU *et al.*, 2019; HUANG *et al.*, 2020; SHI; CHEN; ZHU, 2023; NIETO *et al.*, 2024). Diante desse cenário, este trabalho de qualificação propõe uma solução para o descarregamento de tarefas em redes VEC que integra ranqueamento multicritério por TOPSIS (HWANG; YOON, 1981) com DRL baseado no algoritmo SAC. O TOPSIS avalia multicritérios como latência estimada, ocupação de fila, taxa de canal e consumo energético, reduzindo o conjunto variável de candidatos ao descarregamento — RSU e veículos vizinhos — a um vetor de estado compacto e de dimensão fixa (6D), contornando o problema da entrada variável nas redes neurais. Com o melhor candidato já identificado pelo TOPSIS, o SAC aprende uma política de decisão binária (descarregar para esse candidato ou executar localmente), beneficiando-se da regularização por entropia máxima para manter a exploração em ambientes não-estacionários. Como extensão planejada, o TOPSIS será substituído por *Fuzzy-TOPSIS* (CHEN, 2000), incorporando incerteza linguística aos critérios de avaliação dos candidatos.

1.2 Motivação

A motivação para esta pesquisa fundamenta-se em quatro pilares críticos que representam lacunas tecnológicas e científicas no estado da arte de redes veiculares:

Primeiramente, existe a necessidade de viabilizar soluções de descarregamento de tarefas em cenários de alta heterogeneidade computacional. O atual período de transição tecnológica resulta na coexistência de veículos com OBU de capacidades distintas, exigindo modelos de decisão que operem de forma eficiente independentemente do *hardware* disponível (ZHENG *et al.*, 2015; LUO *et al.*, 2024).

Em segundo lugar, identifica-se um importante *gap* de pesquisa relacionado à escalabilidade do DRL em cenários de mundo real. Como o DRL utiliza redes neurais que dependem de entradas de dimensão fixa, a maioria das soluções existentes exige o retreinamento do agente sempre que o número de veículos vizinhos (agentes) varia (UDDIN; ZHANG; SAKR, 2025; TANG *et al.*, 2024). Desenvolver uma abordagem que lide com essa variação sem necessidade de retreinamento é um dos principais desafios abordados neste trabalho (MA *et al.*, 2025).

Adicionalmente, a realização de reengenharia de estado e de recompensa em DRL é frequentemente descrita como um processo empírico e complexo devido à alta dimensionalidade e à falta de estruturação dos dados de entrada (SHI; CHEN; ZHU, 2023). Esta pesquisa propõe mitigar esse problema ao utilizar o método de ranqueamento multicritério TOPSIS (HWANG; YOON, 1981) como um facilitador da engenharia de estado, reduzindo a complexidade da entrada da rede neural e estruturando a tomada de decisão (VIEIRA; SOUZA; JÚNIOR, 2025).

Por fim, este trabalho de qualificação propõe uma abordagem evolutiva: inicialmente, emprega-se o TOPSIS auxiliando o agente de decisão para validar a arquitetura proposta. Como etapa subsequente da pesquisa de doutorado, planeja-se a evolução para a lógica *Fuzzy*-TOPSIS, visando incorporar a capacidade de lidar com a incerteza e a imprecisão inerentes aos cenários veiculares reais, aumentando a robustez do modelo final.

1.3 Definição do Problema

Diante das limitações de processamento local e da elevada variabilidade do ambiente veicular, o problema central investigado neste trabalho pode ser formulado da seguinte maneira:

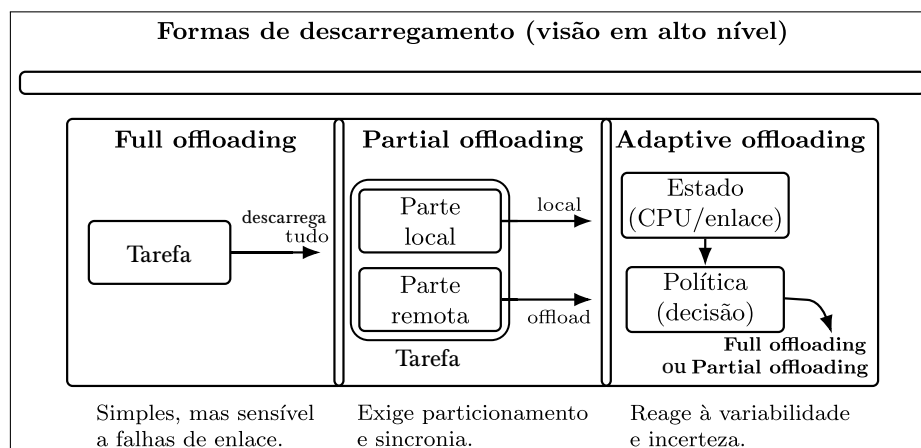
Como tomar decisões de descarregamento de tarefas em redes VEC/V2X, com número variável de candidatos (RSU e veículos vizinhos), de forma confiável e

adaptativa, cumprindo restrições de prazo e minimizando latência e consumo energético, sem necessidade de retreinamento diante de variações na densidade veicular?

Estudos recentes reforçam que a alta mobilidade, a intermitência dos enlaces e a heterogeneidade de recursos tornam esse desafio ainda mais complexo (LIU *et al.*, 2021; MAO *et al.*, 2017; SHI; CHEN; ZHU, 2023; NIETO *et al.*, 2024). Para delimitar o escopo da solução proposta, é importante destacar que o problema de descarregamento de tarefas, em sua essência, configura-se como um problema de otimização *NP-hard* (MAO *et al.*, 2017; WANG *et al.*, 2017). Isso implica que soluções exatas em tempo polinomial são inviáveis para cenários dinâmicos como o veicular, o que direciona a busca por métodos de baixa complexidade computacional e boa capacidade de adaptação.

O processo de descarregamento de tarefas pode ser classificado em três categorias principais: total (*full offloading*), parcial (*partial offloading*) e adaptativo (*adaptive offloading*) (WANG *et al.*, 2022), conforme ilustrado na Figura 1. A escolha entre essas abordagens impacta diretamente a complexidade da formulação do problema, especialmente em ambientes dinâmicos.

Figura 1 – Formas de descarregamento de tarefas em VEC: total (*full*), parcial (*partial*) e adaptativo (*adaptive*), com seus trade-offs de confiabilidade e complexidade.



Fonte: Elaborado pelo autor

Além disso, as estratégias de decisão variam entre abordagens que processam conjuntos de tarefas em lote (*batch*) ou aquelas que decidem dinamicamente para cada tarefa individual, o que impacta diretamente a arquitetura e a complexidade das soluções propostas. O ambiente alvo desta pesquisa caracteriza-se por uma rede VEC altamente heterogênea e volátil, operando

sob cobertura 5G NR para comunicação com infraestruturas de borda e interfaces V2X *Side-link* para cooperação entre veículos vizinhos. A principal dificuldade reside na modelagem de um cenário onde a densidade veicular e a topologia da rede mudam abruptamente, alterando o número de candidatos disponíveis para o descarregamento e a estabilidade dos enlaces de comunicação (HE *et al.*, 2025; LUO *et al.*, 2024).

Nesse contexto, aplicações com alta demanda computacional e baixa carga de dados — como visão computacional e algoritmos de navegação — impõem restrições de latência que superam a capacidade de processamento das OBU locais. Para viabilizar tais aplicações, a tomada de decisão deve integrar, em tempo real, múltiplos indicadores conflitantes (latência, energia, carga de Unidade Central de Processamento (*Central Processing Unit*) (CPU) e qualidade do canal) sob forte incerteza de medição (LIU *et al.*, 2021).

Embora a literatura apresente métodos baseados em programação linear ou meta-heurísticas (LI; ZHU, 2020; YOU; TANG, 2021), tais abordagens operam majoritariamente sobre instantes isolados da rede (*snapshots*). Essa natureza míope limita sua eficácia em cenários veiculares, pois falham em considerar a evolução futura da topologia e o impacto de longo prazo das decisões sequenciais. Por outro lado, soluções de *Deep Reinforcement Learning* (DRL) puro, embora capazes de otimizar recompensas futuras, enfrentam o problema da entrada de dimensão variável. Assim, a integração de métodos de ranqueamento multicritério (TOPSIS) com DRL surge como a estratégia necessária para estruturar esse ambiente caótico em uma representação de estado compacta e invariante à densidade, permitindo decisões robustas e escaláveis.

1.4 Questões de Pesquisa

Este trabalho busca responder às seguintes questões de pesquisa, formuladas a partir das limitações inerentes ao ambiente veicular, da complexidade do problema de descarregamento de tarefas e da necessidade de garantir confiabilidade, eficiência energética e baixa latência em sistemas VEC/V2X:

- QP1. **Como realizar o descarregamento de tarefas em ambientes VEC/V2X de maneira confiável e adaptativa, considerando a alta variabilidade da mobilidade veicular, do canal de comunicação e da carga computacional?**
- QP2. **De que forma a integração de ranqueamento multicritério por TOPSIS com DRL baseado em SAC pode melhorar a tomada de decisão de descarregamento em cenários veiculares dinâmicos, contornando o problema da**

entrada variável nas redes neurais e mantendo estabilidade de aprendizado independentemente da densidade veicular?

- QP3. Em que medida a substituição do TOPSIS clássico por *Fuzzy*-TOPSIS — incorporando incerteza linguística nos critérios de avaliação dos candidatos ao descarregamento — aprimora a robustez da representação de estado e o desempenho do agente SAC em cenários de alta mobilidade e informação imprecisa?**
- QP4. Quais são os impactos do uso conjunto de comunicação 5G NR e V2X, modelados no simulador NS-3 versão 3.42 com o módulo 5G-LENA, no desempenho da solução SAC-*Fuzzy*-TOPSIS, especialmente na redução da latência, no consumo energético e na taxa de cumprimento de prazo (*deadline*)?**

1.5 Hipóteses

A partir das questões acima e da fundamentação teórica apresentada, formulam-se as seguintes hipóteses de pesquisa:

- H1. A combinação de TOPSIS com SAC é capaz de produzir um modelo de decisão adaptativo que supera métodos puramente determinísticos ou heurísticos em cenários VEC/V2X altamente dinâmicos, graças à representação de estado compacta (6D) e à regularização por entropia máxima do SAC.**
- H2. O ranqueamento multicritério por TOPSIS reduz o impacto da variabilidade do número de candidatos ao descarregamento, permitindo que o agente SAC generalize para cenários com densidades veiculares distintas sem necessidade de retreinamento.**
- H3. A substituição do TOPSIS clássico por *Fuzzy*-TOPSIS aprimora a robustez da representação de estado ao incorporar incerteza linguística nos critérios de avaliação, resultando em maior taxa de cumprimento de prazo e melhor função objetivo $J(\pi)$ em cenários de alta mobilidade e informação imprecisa.**
- H4. A validação da arquitetura SAC-*Fuzzy*-TOPSIS no simulador NS-3 versão 3.42, com modelagem realista de 5G NR e comunicação V2X (*sidelink*), confirmará a viabilidade da solução proposta em condições de rede mais próximas do ambiente real, preservando as vantagens de generalização**

observadas na etapa intermediária.

1.6 Objetivos

A fim de formalizar o propósito desta pesquisa e as etapas necessárias para sua execução, esta seção detalha os objetivos pretendidos. As metas são apresentadas seguindo uma abordagem *top-down*, partindo de uma visão consolidada da solução proposta até o detalhamento das etapas técnicas e de validação.

1.6.1 Objetivo Geral

Desenvolver e avaliar uma solução completa de descarregamento de tarefas em redes VEC/V2X baseada na arquitetura SAC-*Fuzzy*-TOPSIS, validada no simulador NS-3 versão 3.42 com modelagem realista de 5G NR e comunicação V2X. Como etapa intermediária — objeto desta qualificação — propõe-se e valida-se a combinação de ranqueamento multicritério por TOPSIS com DRL baseado no algoritmo SAC, produzindo um vetor de estado compacto (6D) e invariante à escala que permite generalização *zero-shot* a cenários com densidades veiculares variáveis. A conclusão do trabalho prevê a substituição do TOPSIS clássico por *Fuzzy*-TOPSIS, incorporando incerteza linguística nos critérios de avaliação dos candidatos ao descarregamento, e a migração da validação para o NS-3 3.42.

1.6.2 Objetivos Específicos

- OE1. Realizar uma revisão sistemática de algoritmos de descarregamento de tarefas em cenários VEC, focando em abordagens de DRL, métodos multicritério e técnicas de engenharia de estado.**
- OE2. Projetar e implementar um módulo TOPSIS para ranqueamento multicritério dos candidatos ao descarregamento (RSU e veículos vizinhos), reduzindo o espaço de estado a um vetor 6D normalizado e invariante à escala, independentemente da densidade veicular — constituindo a etapa intermediária desta qualificação.**
- OE3. Desenvolver e treinar um agente SAC que utilize o vetor de estado produzido pelo TOPSIS para aprender uma política de descarregamento que otimize simultaneamente latência, consumo energético e taxa de cumprimento de prazo (*deadline*), validando a arquitetura SAC-TOPSIS em simulador custo-**

mizado com comunicação V2X.

- OE4. Estender o módulo de ranqueamento para *Fuzzy-TOPSIS*, incorporando incerteza linguística nos critérios de avaliação dos candidatos, e integrar a arquitetura SAC-*Fuzzy-TOPSIS* ao simulador NS-3 versão 3.42, com 5G-LENA e suporte a V2X, para validação final e comparação sistemática com heurísticas e demais algoritmos de DRL.

1.7 Contribuições Esperadas

Espera-se que este trabalho produza contribuições relevantes tanto do ponto de vista científico quanto do ponto de vista prático, sobretudo no contexto de sistemas veiculares inteligentes e computação de borda. As principais contribuições previstas são:

- CE1. **Proposição e validação intermediária da arquitetura SAC-TOPSIS, que combina ranqueamento multicritério com DRL de entropia máxima, demonstrando tomada de decisão adaptativa e generalização *zero-shot* a cenários de densidades veiculares distintas.** Essa etapa fundamenta e antecede a contribuição final: a arquitetura SAC-*Fuzzy-TOPSIS* validada no NS-3 3.42.
- CE2. **Modelagem detalhada do ambiente de comunicação e processamento veicular, incluindo enlaces 5G NR (com alinhamento às diretrizes do 3GPP Release 16), comunicação *Sidelink* V2X, parâmetros de mobilidade e heterogeneidade computacional entre veículos e servidores de borda.** Essa modelagem fornece uma plataforma de simulação realista para decisões de descarregamento.
- CE3. **Desenvolvimento do módulo TOPSIS para avaliação multicritério dos candidatos ao descarregamento, considerando critérios como latência estimada, ocupação de fila, taxa de canal e consumo energético; e, como etapa final da dissertação, desenvolvimento do módulo *Fuzzy-TOPSIS*, que incorporará incerteza linguística para maior robustez em cenários de alta mobilidade e informação imprecisa.**
- CE4. **Treinamento e avaliação do agente SAC, cujo mecanismo de regularização por entropia máxima previne o colapso de política em ambientes não-estacionários e cuja aprendizagem *off-policy* confere eficiência amostral**

superior à de algoritmos *on-policy* como o PPO.

- CE5. **Implementação e avaliação da arquitetura SAC-TOPSIS em simulador customizado com comunicação V2X (etapa intermediária), e implementação final da arquitetura SAC-Fuzzy-TOPSIS no NS-3 versão 3.42 com 5G-LENA, avaliando métricas como taxa de cumprimento de prazo, *Success-Weighted Quality* (SWQ) e função objetivo $J(\pi)$ em cenários de simulação distintos.**
- CE6. **Comparação sistemática com métodos de referência da literatura, incluindo heurísticas tradicionais e outras famílias de algoritmos de DRL, destacando os ganhos obtidos pela arquitetura SAC-TOPSIS na etapa intermediária e pela arquitetura SAC-Fuzzy-TOPSIS na validação final.**
- CE7. **Produção de um conjunto de diretrizes práticas para pesquisadores e engenheiros interessados em projetar sistemas de descarregamento de tarefas veicular, evidenciando cenários onde a arquitetura SAC-Fuzzy-TOPSIS apresenta maior vantagem em relação a heurísticas e algoritmos de DRL sem ranqueamento multicritério.**

1.8 Publicação

Como parte do desenvolvimento deste trabalho, foi submetido e aceito um artigo no periódico internacional *Journal of Cloud Computing* (Springer), classificado como Q1 nas bases *Scopus/SJR* e posicionado no percentil superior da área de *Computer Networks and Communications*. O artigo, intitulado “*FOFF: descarregamento de tarefas eficiente energeticamente em redes veiculares baseadas em VEC usando fuzzy TOPSIS*” (VIEIRA; SOUZA; JÚNIOR, 2025), apresenta uma versão preliminar do modelo híbrido de descarregamento de tarefas proposto nesta dissertação, incluindo a arquitetura conceitual e os primeiros resultados derivados de experimentos em simulação. A publicação serviu tanto como validação da relevância científica da abordagem quanto como etapa intermediária para o refinamento metodológico, a ampliação dos cenários simulados e o aprofundamento das análises apresentadas nos capítulos seguintes.

1.9 Organização do Trabalho

Este documento está estruturado partindo dos fundamentos teóricos até o delineamento da solução proposta e o planejamento para a finalização da pesquisa. A organização segue o seguinte formato:

- **Capítulo 1 — Introdução:** apresenta o contexto geral, a motivação, a definição do problema, as questões de pesquisa, hipóteses e objetivos, além das contribuições esperadas e publicações decorrentes.
- **Capítulo 2 — Fundamentação Teórica:** discute os conceitos essenciais que sustentam este trabalho, abrangendo arquiteturas 5G NR e *Sidelink* V2X, o paradigma de descarregamento de tarefas em ambientes VEC, fundamentos de *Deep Reinforcement Learning* (DRL) e a lógica *fuzzy*.
- **Capítulo 3 — Revisão da Literatura:** realiza uma análise crítica de trabalhos recentes, explorando as principais técnicas e tendências voltadas ao descarregamento de tarefas em cenários veiculares altamente dinâmicos.
- **Capítulo 4 — Proposta:** detalha a arquitetura SAC-*Fuzzy*-TOPSIS desenvolvida, descrevendo a integração entre o algoritmo Soft *Actor-Critic* e o ranqueamento multicritério, bem como a modelagem do ambiente de simulação.
- **Capítulo 5 — Cronograma:** descreve o planejamento das atividades futuras para a conclusão do doutorado, estabelecendo os prazos para experimentos finais, redação da tese e defesa.
- **Capítulo 6 — Conclusão:** sintetiza as principais descobertas desta etapa de qualificação, discute as limitações atuais e aponta os próximos passos para a validação final do modelo.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos teóricos fundamentais que sustentam esta pesquisa, estabelecendo o embasamento necessário para a compreensão da proposta desenvolvida. Inicialmente, a Seção 2.1 detalha a arquitetura 5G NR e a modelagem das comunicações *Sidelink* V2X. Em seguida, a Seção 2.2 aborda a evolução das redes veiculares sob a perspectiva da Internet dos Veículos (IoV). O paradigma de descarregamento de tarefas (*task offloading*) e seus requisitos em ambientes de computação de borda são discutidos na Seção 2.3. Por fim, as bases do Aprendizado por Reforço Profundo (DRL) e da Lógica *Fuzzy*, combinadas ao método de decisão multicritério TOPSIS, são apresentadas nas Seções 2.4 e 2.5, respectivamente.

2.1 Arquitetura e Modelagem 5G V2I e C-V2X

A quinta geração de comunicações móveis celulares, o 5G, representou um avanço tecnológico disruptivo, projetado para revolucionar não apenas o setor de telecomunicações, mas também a sociedade como um todo (STALLINGS, 2021). Suas especificações foram fundamentadas pelo padrão internacional IMT-2020, sob avaliação e monitoramento da União Internacional de Telecomunicações (*International Telecommunication Union*) (ITU) (ITU-R, 2015; ITU-R, 2017; ITU-R, 2021), e formalizadas pelo 3GPP em sucessivos *Releases* (ASPLUND *et al.*, 2025; COX, 2025).

O 5G foi idealizado para operar em três grandes cenários de uso. O primeiro, Banda Larga Móvel Melhorada (*enhanced Mobile Broadband*) (eMBB), atende serviços que exigem altas taxas de transferência de dados, suportando picos de até 20 Gbps no *downlink* (STALLINGS, 2021). O segundo, Comunicações Massivas do Tipo Máquina (*Massive Machine Type Communications*) (mMTC), foi formulado para interconectar eficientemente o ecossistema da Internet das Coisas (*Internet of Things*) (IoT), suportando um grande número de dispositivos com baixo consumo energético (MORAIS, 2025; RAGHAVAN *et al.*, 2025). O terceiro, Comunicações Ultraconfiáveis de Baixa Latência (*Ultra-Reliable Low-Latency Communications*) (URLLC), é essencial para aplicações que demandam atrasos na casa de frações de milissegundos e taxas de erro de 10^{-5} , como controle robótico e ITS, incluindo casos de uso avançados como o *platooning*¹ e a direção autônoma de Nível 4 e 5 (SAE International, 2021; RAGHAVAN *et al.*, 2025; SHEIKH; BHAT; MASOODI, 2026).

¹ Refere-se à aplicação de ITS que vincula eletronicamente dois ou mais veículos para que trafeguem de forma coordenada, mantendo uma distância mínima e constante entre si.

Para suportar tais exigências, a arquitetura do 5G foi subdividida em dois grandes blocos: (i) a Rede de Acesso por Rádio de Próxima Geração (*Next Generation Radio Access Network*) (NG-RAN), responsável pelo acesso de rádio, e (ii) o 5GC, que implementa o núcleo da rede por meio do modelo de Arquitetura Baseada em Serviços (*Service Based Architecture*) (SBA) (STALLINGS, 2021), conforme ilustrado na Figura 2. No núcleo, as Função de Gerenciamento de Acesso e Mobilidade (*Access and Mobility Management Function*) (AMF) e Função de Gerenciamento de Sessão (*Session Management Function*) (SMF) gerenciam mobilidade, sessões e sinalização, enquanto a UPF é responsável pelo roteamento efetivo dos pacotes de dados.

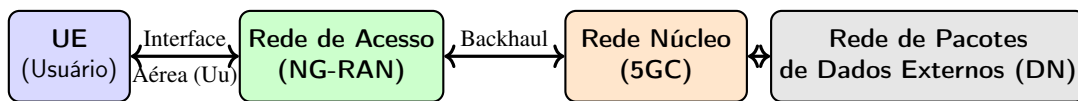


Figura 2 – Modelo de sistema básico 3GPP evidenciando os dois grandes blocos operacionais do 5G.

Fonte: Baseado no modelo de sistema 3GPP (FOWDUR *et al.*, 2025).

Para garantir baixas latências, foi necessário distribuir as funções de processamento e armazenamento da nuvem para dispositivos mais próximos do cliente do serviço. Na prática, isso significou que as aplicações não precisariam mais enviar seus dados para *datacenters* distantes, evitando latências elevadas. Essa solução foi materializada com a integração do MEC à rede móvel 5G, onde a UPF é distribuída localmente para rotear os dados diretamente aos servidores periféricos, possibilitando a resposta em tempo real. A Figura 3 ilustra a arquitetura do 5GC com a integração do MEC.

Na rede de acesso 5G, a estação base Nó B de Próxima Geração (*Next-Generation Node B*) (gNB) adotou uma arquitetura modular que separa suas funções em duas unidades. A Unidade Central (*Central Unit*) (CU) processa serviços em tempo não real, hospedando os protocolos de maior nível como Controle de Recursos de Rádio (*Radio Resource Control*) (RRC), Protocolo de Convergência de Protocolo de Dados (*Packet Data Convergence Protocol*) (PDCP) e Protocolo de Adaptação de Dados de Serviço (*Service Data Adaptation Protocol*) (SDAP). Já as Unidade Distribuída (*Distributed Unit*)s (DUs) operam localmente próximas às antenas, executando serviços de tempo real como agendamento de recursos de rádio, correção de erros e processamento de sinais — responsabilidade dos protocolos Controle de Enlace de Rádio (*Radio Link Control*) (RLC), Controle de Acesso ao Meio (*Medium Access Control*) (MAC) e da camada Física (Camada Física (*Physical Layer*) (PHY)) (FOWDUR *et al.*, 2025; COX, 2025).

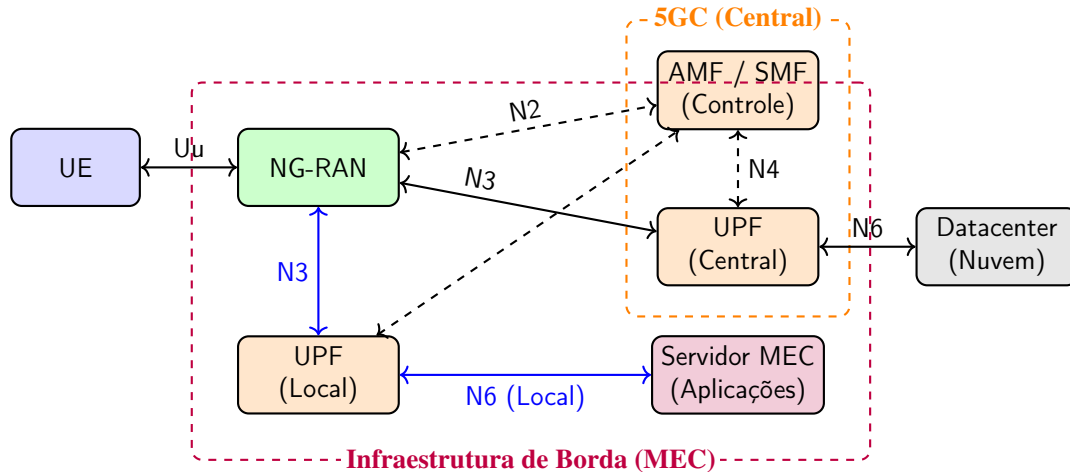


Figura 3 – Integração do MEC à rede 5G distribuindo a função UPF localmente visando reduzir a latência de tráfego sensível.

Fonte: Elaborado pelo autor.

2.1.1 Interface Aérea 5G NR: OFDMA e Adaptação de Enlace

Os sinais transmitidos pela interface aérea do 5G NR utilizam o esquema de OFDMA. Esse esquema divide a largura de banda total em múltiplas subportadoras ortogonais, eliminando a interferência entre elas. Os recursos são gerenciados como uma matriz bidimensional de tempo e frequência, agrupados em Bloco de Recurso (*Resource Block*)s (RBs) com 12 subportadoras cada, permitindo que diferentes fatias do espectro sejam alocadas simultaneamente a diferentes usuários (STALLINGS, 2021; ASPLUND *et al.*, 2025). O 5G NR introduziu ainda uma numerologia escalável, com espaçamento de subportadoras $\Delta f = 15 \times 2^\mu$ kHz para $\mu \in \{0, 1, 2, 3, 4, 5, 6\}$, permitindo adaptar a latência e a capacidade de acordo com a faixa de frequência e o cenário de uso (COX, 2025). O espectro suportado abrange a faixa Faixa de Frequência 1 (*Frequency Range 1*) (FR1) (sub-6 GHz, até 100 MHz de largura de banda) e a faixa Faixa de Frequência 2 (*Frequency Range 2*) (FR2) ou Ondas Milimétricas (*Millimeter Wave*) (mmWave) (24,25 GHz a 71 GHz, até 400 MHz), com o emprego de *Massive* Múltiplas Entradas, Múltiplas Saídas (*Multiple-Input Multiple-Output*) (MIMO) e *beamforming* para compensar a atenuação nas frequências milimétricas (STALLINGS, 2021; ASPLUND *et al.*, 2025).

Na camada física, a informação digital é mapeada em ondas eletromagnéticas por meio de esquemas de Modulação de Amplitude em Quadratura (*Quadrature Amplitude Modulation*) (QAM). O 3GPP especifica tabelas de Esquema de Modulação e Codificação (*Modulation and Coding Scheme*) (MCS) que associam um índice a um esquema de modulação específico (QPSK, 16-QAM, 64-QAM ou 256-QAM) e à sua respectiva taxa de codificação. Cada esquema

empacota uma quantidade crescente de bits por símbolo transmitido — de 2 bits no QPSK a 8 bits no 256-QAM —, elevando diretamente a vazão de dados (COX, 2025). Conforme a ordem de modulação aumenta, os pontos no diagrama de constelação tornam-se mais densos, exigindo uma Relação Sinal-Interferência-e-Ruído (*Signal-to-Interference-plus-Noise Ratio*) (SINR) proporcionalmente superior para manter a Taxa de Erro de Bit (*Bit Error Rate*) (BER) aceitável.

Para contornar a degradação, a rede emprega o mecanismo de adaptação de enlace (*Link Adaptation*): avalia continuamente a qualidade do canal e regrida dinamicamente para esquemas de modulação mais simples (como QPSK) em condições adversas, preservando a confiabilidade da entrega dos pacotes (COX, 2025; STALLINGS, 2021). É essa dinâmica de seleção do MCS que determina a taxa de canal instantânea — um dos critérios multicritério avaliados pelo TOPSIS na seleção do melhor candidato ao descarregamento de tarefas neste trabalho.

2.1.2 Comunicação Veicular C-V2X e Interface *Sidelink* PC5

O próximo passo natural, após essa evolução na arquitetura da comunicação móvel, foi o aprimoramento da comunicação direta entre os Equipamento de Usuário (*User Equipment*)s (UEs). No que tange à comunicação veicular, a tecnologia de quinta geração consolidou o paradigma C-V2X (GARCIA; AL., 2021). Ao herdar a robustez do 5G NR, o C-V2X supera as limitações de tecnologias baseadas em redes *ad-hoc* (como o *Dedicated Short-Range Communications/Institute of Electrical and Electronics Engineers* (DSRC/IEEE) 802.11p), garantindo a latência e a confiabilidade necessárias para aplicações críticas de segurança e condução autônoma.

O C-V2X opera fundamentalmente sobre duas interfaces: (i) a Uu, conectando o veículo à infraestrutura da rede celular (V2I/V2N), e (ii) a PC5 (*Sidelink*), que estabelece um enlace direto veículo-veículo (V2V) e veículo-pedestre (V2P) sem que os dados precisem transitar pelas antenas ou pelo núcleo da rede 5GC. No *Release* 16 do 3GPP, o *Sidelink* do 5G NR foi aprimorado para suportar não apenas transmissões abertas (*broadcast*), mas também comunicações *unicast* e *groupcast*, garantindo maior controle de Qualidade de Serviço (*Quality of Service*) (QoS) em manobras cooperativas, como a formação de pelotões (*platooning*).

O gerenciamento do acesso ao meio e a reserva de recursos de rádio no PC5 *Sidelink* operam sob dois modos distintos. No Modo 1, a alocação de recursos é centralizada: a gNB

agenda os recursos para os veículos através da Uu, garantindo baixa ocorrência de colisões, mas exigindo que os veículos estejam sob cobertura celular ininterrupta. No Modo 2, os próprios veículos gerenciam os recursos de rádio de forma autônoma por meio de sensoriamento prévio do canal (*Channel Sensing*), sendo a base para a cooperação distribuída em zonas sem cobertura celular. A Figura 4 apresenta essas conexões.

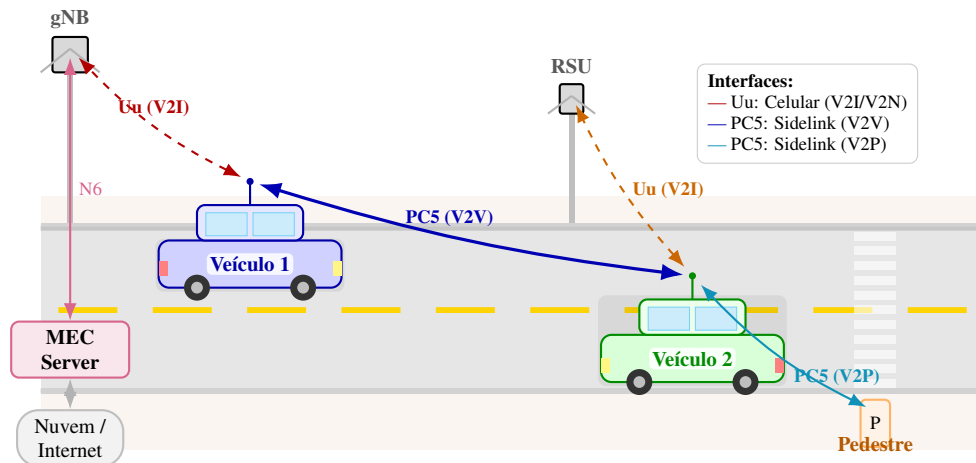


Figura 4 – Arquitetura das comunicações 5G C-V2X: coexistência da interface celular Uu (V2I/V2N) com a interface direta PC5 Sidelink (V2V/V2P), integrando o núcleo 5GC, o servidor MEC de borda e os elementos veiculares e pedestres.

Fonte: Elaborado pelo autor com base em (GARCIA; AL., 2021; COX, 2025).

Para viabilizar essa comunicação direta, a camada física da PC5 organiza a troca de dados em três canais físicos complementares que operam sequencialmente dentro de um mesmo *slot* de transmissão: o PSCCH, o PSSCH, que transporta efetivamente a carga útil de dados, e o PSFCH. A Figura 5 ilustra a coexistência e a multiplexação temporal desses canais dentro de um *slot* do 5G NR Sidelink.

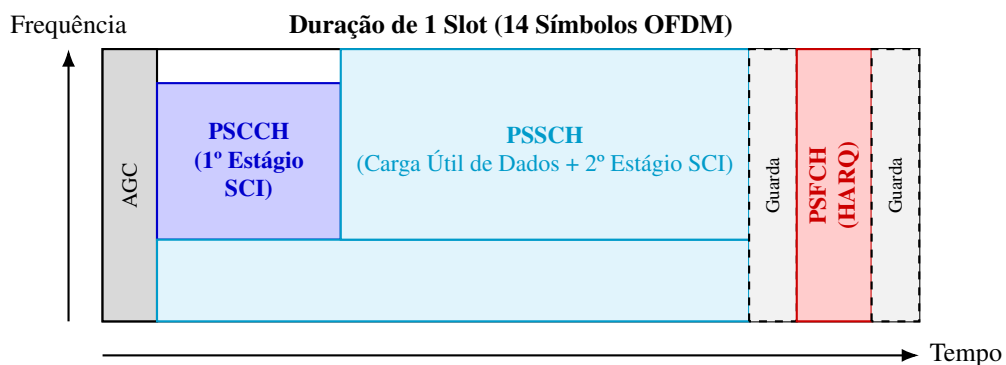


Figura 5 – Estrutura temporal básica da camada física em um *slot* do 5G NR Sidelink, ilustrando a multiplexação dos canais físicos PSCCH, PSSCH e PSFCH (COX, 2025; GARCIA; AL., 2021).

O PSCCH anuncia publicamente a reserva de recursos de tempo e frequência (Bloco de Recurso Físico (*Physical Resource Block*)s (PRBs)), permitindo que veículos vizinhos realizem o sensoriamento do meio para evitar colisões de dados e encontrar blocos de rádio livres para transmissão. Em seguida, o canal compartilhado (PSSCH) transmite a carga útil da aplicação e as instruções para o destinatário processar e responder às requisições. Essas instruções incluem o identificador do emissor, um indicador que avisa ao receptor se o pacote contém novos dados ou é uma retransmissão, o tipo de comunicação (*broadcast*, *unicast* ou *groupcast*), dados geográficos e uma requisição ao receptor sobre as condições físicas do enlace. Por último, o canal de *feedback* (PSFCH) confirma, quase instantaneamente, o sucesso ou a falha da recepção do pacote (ACK/NACK).

2.2 A Internet de Veículos (IoV) e as Redes Veiculares

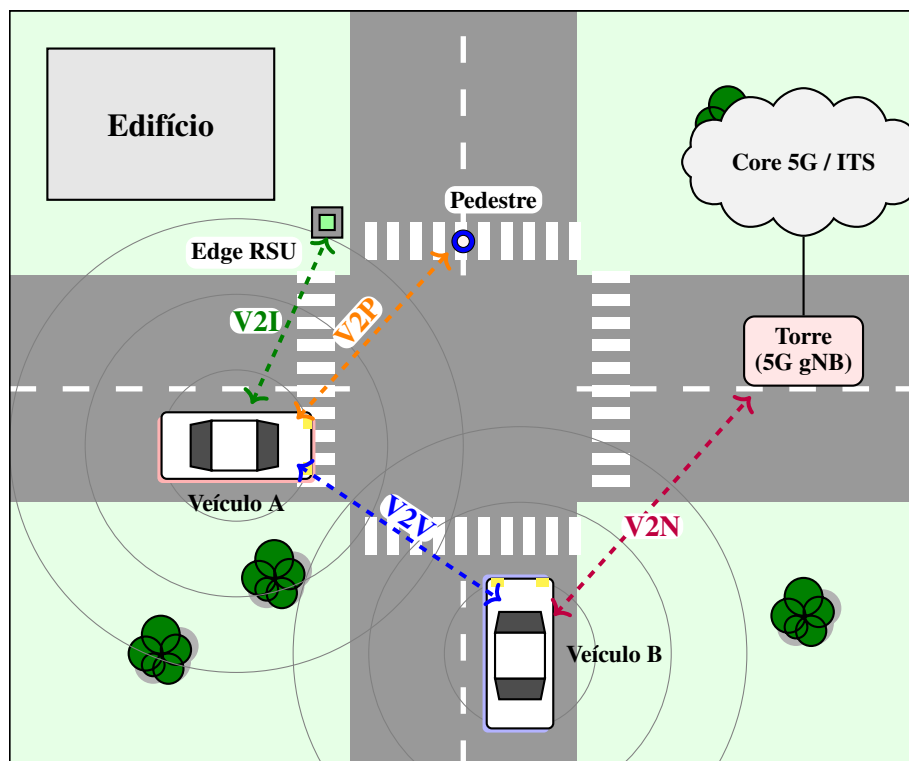
A evolução das Redes de Comunicação Veicular nas últimas décadas é marcada por uma transição clara entre paradigmas tecnológicos. Inicialmente, o foco concentrava-se nas Rede Veicular Ad hoc (*Vehicular Ad hoc NETWORK*)s (VANETs), priorizando a conectividade direta entre os veículos. Com o amadurecimento da IoT, iniciou-se um processo de convergência que resultou no atual ecossistema da IoV (PARRA *et al.*, 2025; ABD-ELNABY *et al.*, 2024). Enquanto o estágio inicial das VANETs visava a comunicação básica, a IoV contemporânea integra sensores embarcados, Inteligência Artificial (*Artificial Intelligence*) (IA) e Redes Definidas por Software (*Software-Defined Networking*) (SDN) para viabilizar o ITS verdadeiramente autônomos e conectados.

As VANETs emergiram como uma evolução das Rede Móvel Ad hoc (*Mobile Ad hoc NETWORK*)s (MANETs). No entanto, seu grande diferencial é a alta mobilidade dos nós, o que impõe desafios rigorosos em termos de conectividade e latência. Diferente das MANETs, os enlaces nas redes veiculares surgem e desaparecem rapidamente. Para aplicações de segurança crítica, como alertas de colisão, atrasos na comunicação são intoleráveis. Consequentemente, novas tecnologias de comunicação foram desenvolvidas para atender às exigências das VANETs e da futura IoV, destacando-se o DSRC e o C-V2X.

À medida que as redes veiculares evoluíam, os sistemas de IoT se expandiam pelas infraestruturas urbanas. Cada vez mais, dispositivos com sensores foram instalados em semáforos, estações meteorológicas, equipamentos públicos e até mesmo sendo carregados por pedestres (ABD-ELNABY *et al.*, 2024). A IoV surge da consolidação deste cenário, sendo

definida como uma rede heterogênea de grande escala que possibilita a comunicação entre veículos, infraestrutura, pedestres e a nuvem (ANSARI, 2020). O compartilhamento destas informações amplia a observação do ambiente (NAIK *et al.*, 2019), otimiza a orquestração do tráfego (SÁNCHEZ *et al.*, 2025) e aprimora os sistemas de segurança (ABD-ELNABY *et al.*, 2024).

Figura 6 – Abstração do Ecossistema IoV: V2V, V2I, V2N e V2P em Cruzamento



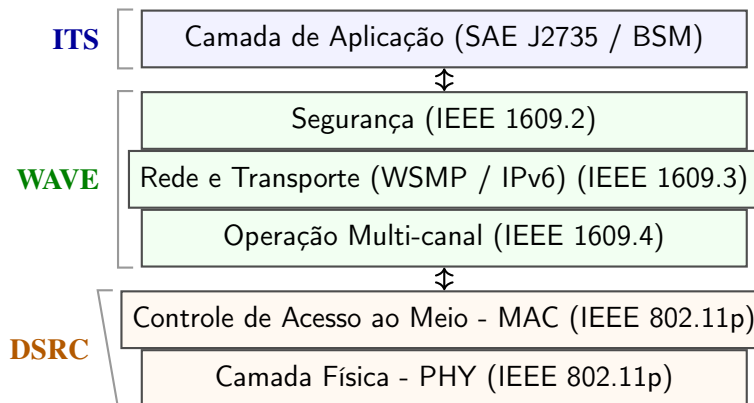
Fonte: Elaborado pelo autor

O ecossistema IoV, referenciado como V2X, é estruturado por quatro interfaces de comunicação que determinam a topologia e o alcance da troca de dados. A interface V2V viabiliza a comunicação direta entre veículos, possibilitando sistemas de segurança ativa e alertas de colisão iminente. A comunicação V2I conecta o veículo aos equipamentos da infraestrutura viária (RSU), provendo tanto dados locais de tráfego quanto o acesso a recursos de computação robustos com baixa latência. Para cenários que demandam coordenação global ou acesso a um poder de armazenamento e processamento massivo, a interface V2N estabelece o enlace celular com a rede núcleo da operadora, encaminhando o tráfego para a nuvem em detrimento do aumento da latência. Por fim, a interface V2P integra pedestres à rede por meio de dispositivos móveis, ampliando a consciência situacional do sistema de transporte.

Neste cenário, a viabilização da IoV é intrinsecamente dependente da maturidade de

tecnologias de rádio robustas. Historicamente, o marco inicial no desenvolvimento de sistemas de comunicação de baixa latência ocorreu no final da década de 1990, com a alocação de 75MHz do espectro na banda 5.9GHz para sistemas de transporte inteligente (ZHENG *et al.*, 2015). A motivação na época, residia na incapacidade das redes celulares em atender aos requisitos de segurança viária (latências proibitivas e instabilidade de conexão), somada às limitações de interferência das bandas não licenciadas (Industrial, Científica e Médica (*Industrial, Scientific and Medical*) (ISM)). Sob uma perspectiva técnica, o DSRC consolidou-se não como um protocolo isolado, mas como um arcabouço regulatório e tecnológico que, ao integrar a camada física IEEE 802.11p à arquitetura *Wireless Access in Vehicular Environments* (WAVE) — responsável por definir como as mensagens de segurança são estruturadas, criptografadas e um conjunto de padrões de interface de comunicação V2V e V2I — definiu os parâmetros fundamentais para aplicações de segurança ativa e sistemas de tarifação eletrônica (Figura 7).

Figura 7 – Arquitetura de Protocolos DSRC / WAVE (IEEE 1609 / 802.11p)



Fonte: Adaptado de (ZHENG *et al.*, 2015)

A partir do legado do DSRC, a evolução tecnológica da comunicação veicular ramificou-se em dois eixos técnicos distintos. O primeiro, o padrão IEEE 802.11bd (*Next Generation V2X* - NGV), surge como uma melhoria incremental do IEEE 802.11p. Conforme (NAIK; CHOUDHURY; PARK, 2019), a demanda por maior vazão de dados (*throughput*) e a necessidade de robustez contra o desvanecimento rápido em cenários de alta mobilidade foram os principais motivadores desta evolução, integrando técnicas para suporte a velocidades de até 250 km/h.

O segundo eixo representa uma mudança de paradigma com o surgimento do C-V2X e sua posterior transição para o 5G NR-V2X e o horizonte do Sexta Geração de Redes Móveis (*Sixth Generation*) (6G) (RODRÍGUEZ-PIÑEIRO *et al.*, 2025). As limitações do DSRC em

termos de escalabilidade e custo de implantação impulsionaram o desenvolvimento do C-V2X pelo 3GPP (CHEN; AL., 2017). Sob uma perspectiva regulatória atual, a disputa tecnológica foi amplamente resolvida em mercados-chave; nos Estados Unidos, a Comissão Federal de Comunicações (*Federal Communications Commission*) (FCC) estabeleceu o C-V2X como padrão obrigatório para segurança ITS com descontinuação oficial do DSRC em dezembro de 2026 (Federal Communications Commission (FCC), 2024), enquanto no Brasil a Agência Nacional de Telecomunicações (ANATEL), através da Resolução nº 772/2025, harmonizou os requisitos técnicos nacionais com os padrões globais celular e radar automotivo.

As redes veiculares evoluíram com a integração à IoV, transitando da incerteza tecnológica para uma realidade onde a superioridade do C-V2X é um fato regulatório consolidado (ANSARI, 2020). Embora o DSRC possua validação extensiva em segurança básica, a indústria convergiu para o ecossistema celular em função de sua projeção escalonável em direção ao 5G NR em frequências milimétricas (mmWave, FR2) e aos futuros padrões 6G operando em faixas Terahertz (THz) (NAIK *et al.*, 2019).

2.3 Descarregamento de Tarefas

O descarregamento de tarefas (*task offloading* ou *computation offloading*) é uma estratégia que visa transferir a responsabilidade de computar tarefas complexas de dispositivos com recursos limitados para nós de processamento remoto mais poderosos. O objetivo é contornar as restrições locais de bateria, armazenamento e processamento, com o intuito de reduzir o consumo de energia, minimizar a latência e melhorar o desempenho e a qualidade do serviço (QoS) para o usuário.

Em termos de infraestrutura, geralmente o descarregamento de tarefas é implementado em uma hierarquia de três camadas (SOFLA *et al.*, 2022; GARAI *et al.*, 2023). A Figura 8 ilustra essa arquitetura, onde a Camada 1 (Dispositivos) é composta pelas unidades finais que geram as demandas computacionais, como *smartphones*, sensores IoT, *drones* (Veículo Aéreo Não Tripulado (*Unmanned Aerial Vehicle*) (UAV)) e veículos conectados (VEC). Estes dispositivos possuem restrições severas de bateria e processamento, sendo o ponto de origem das tarefas que necessitam de descarregamento.

A Camada 2 (Borda) atua como um intermediário estratégico. Composta por servidores de borda, como estações rádio base (gNB) e unidades de beira de estrada (RSU), esta camada oferece recursos computacionais com baixíssima latência devido à sua proximidade

física com os dispositivos da Camada 1. Um diferencial importante nesta camada, destacado pela linha pontilhada, é a capacidade de colaboração horizontal, onde diferentes nós de borda podem compartilhar carga de trabalho para otimizar o uso de recursos locais.

Por fim, a Camada 3 (Nuvem) representa o nível superior da hierarquia, consistindo em servidores centralizados com capacidades de processamento (CPU/Unidade de Processamento Gráfico (*Graphics Processing Unit*) (GPU)) e armazenamento virtualmente ilimitadas (JANSEN *et al.*, 2022). Embora ofereça o maior poder computacional, a comunicação com esta camada é caracterizada por uma latência mais elevada e maior consumo de largura de banda nos *links* de longa distância, sendo ideal para tarefas extremamente complexas que não possuem restrições rígidas de tempo real.

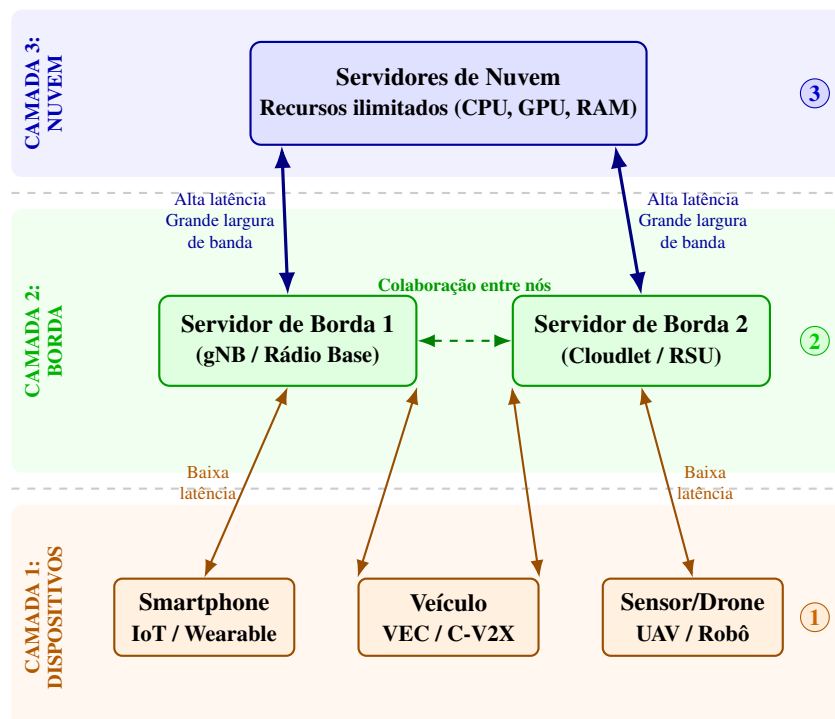


Figura 8 – Arquitetura hierárquica em três camadas para computação em borda e nuvem.

A Figura 9 apresenta o processo de funcionamento do descarregamento de tarefas em uma sequência de etapas. Após a geração da tarefa (Fase 1), quando a estratégia de descarregamento parcial é adotada, as aplicações são particionadas e dividem a tarefa em duas categorias: componentes não descarregáveis, que obrigatoriamente devem ser executados localmente (como interfaces de usuário ou acesso a sensores), e componentes computacionalmente intensivos (como processamento de vídeo de alta resolução, inferência de modelos de aprendizado de máquina ou cálculos matemáticos complexos), que podem ser descarregados para execução em servidores remotos (MACH; BECVAR, 2017; ZHANG *et al.*, 2023).

Em seguida, realiza-se a Fase 2 de decisão, frequentemente apontada como a mais complexa, onde um algoritmo avalia, segundo sua política de funcionamento, se é vantajoso descarregar a tarefa para processamento remoto ou mantê-la para computação local. Caso a decisão seja pelo processamento remoto, o dispositivo negocia a execução da tarefa através de protocolos de comunicação. Após a confirmação da alocação de recursos, o dispositivo transmite os dados para o servidor escolhido e aguarda o processamento (Fase 3). Por fim, os resultados são retornados e incorporados à aplicação original de forma transparente para o usuário (Fase 4) (BOUKERCHE; SOTO, 2020).

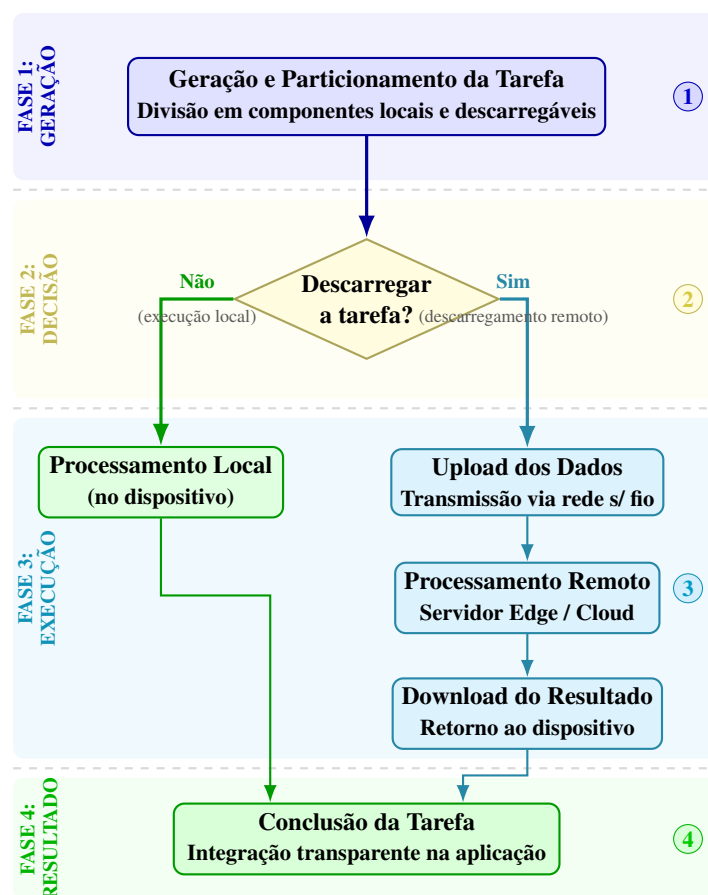


Figura 9 – Fluxograma do processo decisório e ciclo de vida do descarregamento de tarefas.

Vale ainda ressaltar que, durante o processo de geração e modelagem das tarefas, a aplicação pode especificar parâmetros essenciais como a complexidade computacional exigida (frequentemente medida em ciclos de CPU) e o prazo máximo tolerável (*deadline* ou restrição de latência) (KUMAR; PAL, 2025; WANG *et al.*, 2024). Esse limite temporal é estipulado para garantir que todo o ciclo de descarregamento remoto — o qual engloba a transmissão de dados, a espera em filas de recursos (*queuing delay*), o processamento e o retorno dos resultados — seja

concluído sem causar degradação na qualidade do serviço ofertado ao usuário (ZHANG *et al.*, 2023).

2.4 Aprendizado por Reforço

Fundamentos

O RL é um paradigma de Aprendizado de Máquina (*Machine Learning*) (ML) onde o treinamento dos modelos se dá com a interação direta com o ambiente através de ações e a observação da recompensa ou penalização por tal ato (SUTTON; BARTO, 2018). Nela, o agente, aquele que procura aprender as relações intrínsecas do problema que se busca resolver, não tem acesso a nenhum supervisor que indique qual a melhor ação a ser tomada, e nem mesmo a um conjunto de dados de partida rotulados para que ele possa analisar e buscar reconhecer padrões (HU, 2024; SUTTON; BARTO, 2018).

De maneira geral, o aprendizado em problemas de RL ocorre por meio de um ciclo de interação contínua entre o agente e o ambiente, dinâmica que pode ser observada na Figura 10 (HU, 2024; SUTTON; BARTO, 2018). Embora esse mecanismo iterativo de tentativa e erro seja conceitualmente simples, ele serve de base para o desenvolvimento de modelos de IA capazes de resolver problemas complexos de tomada de decisão em áreas desafiadoras, como robótica, condução autônoma e jogos de alto desempenho (HU, 2024). Além disso, essa mesma interação tem sido fundamental para auxiliar no treinamento e alinhamento de Grande Modelo de Linguagem (*Large Language Model*s) (LLMs), empregando algoritmos de otimização de políticas aliados ao Aprendizado por Reforço a partir de Feedback Humano (*Reinforcement Learning from Human Feedback*) (RLHF) para refinar e instruir sistemas avançados, a exemplo do InstructGPT e do ChatGPT (OUYANG *et al.*, 2022; HU, 2024).

Processos de Decisão de Markov (MDP) e seus Componentes

Um MDP é uma estrutura matemática utilizada para modelar problemas de tomada de decisão sequencial sob incerteza (SUTTON; BARTO, 2018; ZHAO, 2024). Um problema modelado sob o paradigma MDP é frequentemente representado pela tupla (S, A, P, R, γ) (SUTTON; BARTO, 2018; HU, 2024). Nela, S representa o espaço de estados, contendo todas as configurações possíveis do ambiente, e A denota o espaço de ações disponíveis para o agente (HU, 2024). A função de transição, $P(s'|s, a)$, define a probabilidade de o ambiente transitar para um

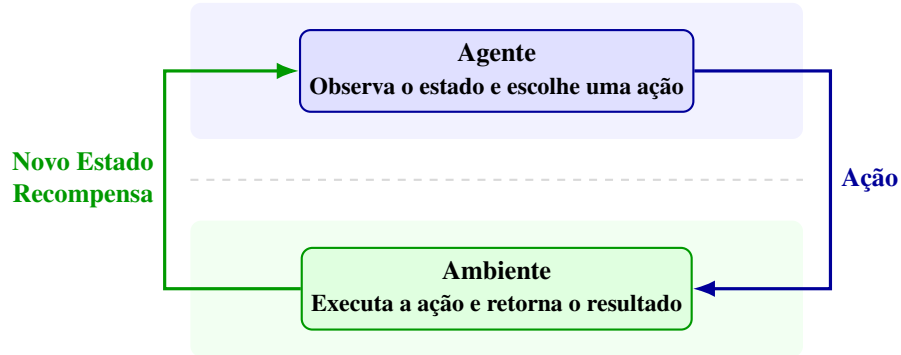


Figura 10 – Ciclo básico de interação Agente-Ambiente no RL. A cada passo, o agente observa o estado atual, escolhe uma ação, e o ambiente responde com uma recompensa e um novo estado.

Fonte: Elaborado pelo autor com base em (HU, 2024; SUTTON; BARTO, 2018).

estado sucessor s' dado que a ação a foi tomada no estado atual s (SUTTON; BARTO, 2018; ZHAO, 2024). Por sua vez, a Função de Recompensa (R) especifica a retribuição numérica imediata (que pode ser positiva ou negativa) indicando o ganho ou custo da transição (SUTTON; BARTO, 2018; HU, 2024). A recompensa esperada para um par estado-ação é formalmente expressa por $R(s, a) = \mathbb{E}[R_t | S_t = s, A_t = a]$ (HU, 2024). O parâmetro $\gamma \in [0, 1)$ atua como um fator de desconto, controlando o peso que o agente dará às recompensas futuras em comparação às imediatas (SUTTON; BARTO, 2018; ZHAO, 2024).

Além disso, o agente pode seguir uma estratégia para mapear estados e tomar suas ações, mecanismo este conhecido como política (π) (HU, 2024). A dinâmica desse sistema atende rigorosamente à Propriedade de Markov, a qual postula que o futuro independe do passado dado o estado e a ação presentes (SUTTON; BARTO, 2018; ZHAO, 2024). Por fim, em cenários onde o agente não observa o estado real, mas recebe apenas uma observação corrompida ou baseada em probabilidades, o modelo é generalizado para os Processos de Decisão de Markov Parcialmente Observáveis (Processo de Decisão de Markov Parcialmente Observável (*Partially Observable Markov Decision Process*) (POMDP)) (SUTTON; BARTO, 2018; HU, 2024).

A Figura 11 detalha o ciclo de interação na modelagem de MDP (SUTTON; BARTO, 2018). Primeiramente (Passo 1), o agente observa o estado (S_t) em que o ambiente se encontra (SUTTON; BARTO, 2018). Depois, guiado por sua política π , o agente seleciona uma ação A_t a ser tomada (Passo 2) (SUTTON; BARTO, 2018). Em seguida (Passo 3), essa ação é executada, e com base na probabilidade $P(s' | s, a)$, o estado atual muda de s para s' (SUTTON; BARTO, 2018). Como consequência dessa alteração, o ambiente devolve ao agente um sinal de recompensa imediato R_{t+1} (Passo 4), que beneficia ou penaliza a ação tomada frente ao objetivo do problema (SUTTON; BARTO, 2018). Por fim (Passo 5), o ambiente convergiu para um novo

estado sucessor S_{t+1} , e o agente captura a nova observação, reiniciando o ciclo iterativamente ao longo do tempo (SUTTON; BARTO, 2018).

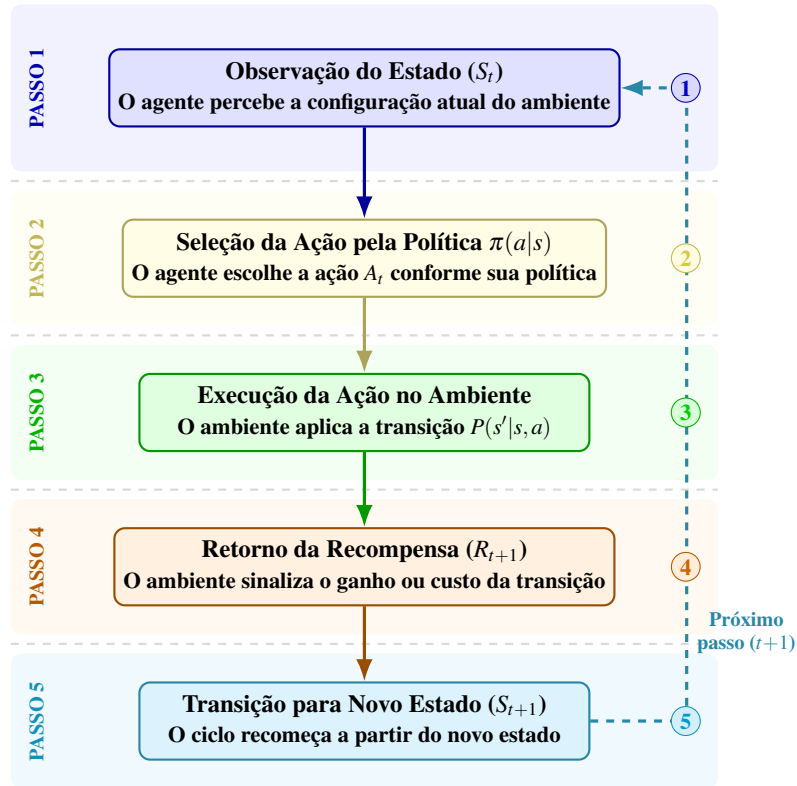


Figura 11 – Sequência de interação em um MDP.

Fonte: Elaborado pelo autor com base em (SUTTON; BARTO, 2018).

Retorno Esperado, Fator de Desconto e Horizontes

O MDP é capaz de modelar problemas que possuem um final natural bem definido, conhecidos na literatura como tarefas episódicas (Horizonte Finito), como uma partida de xadrez, ou problemas em que o agente continua interagindo com o ambiente perpetuamente, chamados de tarefas contínuas (Horizonte Infinito) (SUTTON; BARTO, 2018; HU, 2024). Em tarefas episódicas, que finalizam em um passo de tempo terminal T , o agente busca maximizar o retorno cumulativo simples, dado pela soma $G_t = R_{t+1} + R_{t+2} + \dots + R_T$ (SUTTON; BARTO, 2018).

No entanto, para tarefas contínuas (Horizonte Infinito), a simples soma das recompensas ao longo do tempo poderia divergir e tender ao infinito (HU, 2024; SUTTON; BARTO, 2018). Para solucionar esse problema, aplica-se o fator de desconto $\gamma \in [0, 1)$ (SUTTON; BARTO, 2018; ZHAO, 2024). O Retorno Descontado G_t , iniciado no passo t , é calculado como a soma infinita das recompensas futuras, onde as mais próximas são menos penalizadas que as mais

distantes (SUTTON; BARTO, 2018):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

Além disso, uma propriedade algébrica do retorno em RL é que a Equação 2.1 pode ser reescrita de forma exata como uma relação recursiva (HU, 2024; SUTTON; BARTO, 2018). Ela separa a recompensa imediatamente do retorno descontado do próximo passo de tempo (SUTTON; BARTO, 2018; HU, 2024):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1} \quad (2.2)$$

Esta decomposição recursiva forma a base matemática que torna possíveis as Equações de Bellman e a atualização iterativa das estimativas das funções de valor nos algoritmos de aprendizado (HU, 2024; SUTTON; BARTO, 2018). Em última análise, o objetivo central do agente é maximizar o retorno esperado $\mathbb{E}[G_t]$, ponderando tanto a dinâmica aleatória do ambiente quanto as ações definidas pela sua política, que pode ser determinística ($a = \pi(s)$) ou estocástica ($\pi(a|s) = P(A_t = a|S_t = s)$) (SUTTON; BARTO, 2018; HU, 2024).

Funções de Valor e Equações de Bellman

Em problemas de RL, o agente avalia o quão bom é o seu comportamento ao seguir uma determinada política por meio da estimativa do retorno esperado a longo prazo (SUTTON; BARTO, 2018). Para quantificar formalmente esse desempenho, a literatura estabelece duas abordagens principais: (i) estimar o valor de estar no estado atual (s), ou (ii) estimar o valor de selecionar uma ação específica (a) a partir desse estado (s) (HU, 2024; SUTTON; BARTO, 2018). A primeira estimativa é calculada utilizando a Função de Valor de Estado (V^π), enquanto a segunda é obtida por meio da Função de Valor de Ação (Q^π) (HU, 2024; SUTTON; BARTO, 2018). Ambas as funções fundamentam-se em uma propriedade de consistência recursiva, expressas matematicamente pelas equações de expectativa de Bellman (SUTTON; BARTO, 2018):

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')] \quad (2.3)$$

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right] \quad (2.4)$$

Para resolver o problema de RL, não basta apenas avaliar ações; é preciso encontrar o comportamento que gere a maior recompensa possível a longo prazo, ou seja, a política ótima

(π^*) (HU, 2024; SUTTON; BARTO, 2018). Para isso, o objetivo passa a ser encontrar as Funções de Valor Ótimas (V^* e Q^*), que representam o máximo retorno esperado que pode ser alcançado entre todas as políticas possíveis (HU, 2024; SUTTON; BARTO, 2018). Essas funções devem satisfazer as Equações de Otimidade de Bellman, que introduzem o operador *max* para selecionar ativamente a ação que maximiza o ganho (SUTTON; BARTO, 2018; HU, 2024):

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')] \quad (2.5)$$

$$Q^*(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a'} Q^*(s',a') \right] \quad (2.6)$$

Uma vez que o agente descobre a função de valor de ação ótima, a política ótima (π^*) pode ser facilmente determinada agindo de forma gulosa, ou seja, escolhendo sempre a ação que fornece o valor máximo através da relação $\pi^*(s) = \arg \max_a Q^*(s,a)$ (HU, 2024; SUTTON; BARTO, 2018). Além disso, a conexão entre as duas funções ótimas estabelece que o valor de um estado sob a política ótima é exatamente equivalente ao valor dessa melhor ação escolhida, o que é expresso matematicamente por $V^*(s) = \max_a Q^*(s,a)$ (HU, 2024; SUTTON; BARTO, 2018). Também é possível definir a Função de Vantagem (*Advantage*), que quantifica o quão melhor é tomar uma ação específica isolada em comparação ao retorno esperado caso o agente simplesmente seguisse a política atual naquele estado (WANG *et al.*, 2016):

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \quad (2.7)$$

Dilema Exploração-Exploração e Métodos de Aprendizado

De forma geral, durante o ciclo de interação para otimizar as funções de valor, o agente enfrenta o dilema exploração-exploração (*Exploration-Exploitation Dilemma*), precisando decidir entre explorar o ambiente para descobrir novas ações e estados, e explorar o conhecimento já adquirido para maximizar sua recompensa cumulativa (SUTTON; BARTO, 2018; HU, 2024). O balanceamento adequado entre essas duas estratégias é importante para evitar a convergência prematura para comportamentos subótimos e garantir a descoberta das melhores decisões (HU, 2024).

Para resolver o problema de como o agente descobre essas melhores decisões, a literatura divide os métodos em duas categorias principais: Métodos baseado em modelo (*Model-Based*) e livres de modelo (*Model-Free*) (SUTTON; BARTO, 2018; MOERLAND *et al.*, 2023). No primeiro método, o agente possui ou aprende as regras de funcionamento do

ambiente, permitindo o uso de técnicas de planejamento para simular trajetórias e antecipar ações (SUTTON; BARTO, 2018; HU, 2024). Já no segundo método, o agente não constrói um modelo das regras do jogo, mas aprende as funções de valor ou as políticas diretamente a partir da experiência obtida por tentativa e erro no ambiente (HU, 2024; SUTTON; BARTO, 2018).

Para resolver problemas *Model-Free*, existem duas estratégias principais (SUTTON; BARTO, 2018). Os Métodos de Monte Carlo esperam até o final de um episódio para atualizar as estimativas de valor, utilizando a soma real de todas as recompensas obtidas ao longo de toda a trajetória (SUTTON; BARTO, 2018; HU, 2024). Por outro lado, a Aprendizagem por Diferença Temporal (*TD Learning*) não espera o fim do episódio, ela atualiza as estimativas a cada passo, ou seja, o agente ajusta o valor do estado atual somando a recompensa imediata recebida com o valor que ele já estimava para o próximo estado (SUTTON; BARTO, 2018).

Dentro do *TD Learning*, destacam-se dois algoritmos clássicos (SUTTON; BARTO, 2018). O *State-Action-Reward-State-Action* (SARSA) realiza um aprendizado *On-policy*, o que significa que a política que o agente busca aprender (política alvo) e a política que ele usa para gerar experiências no ambiente são exatamente a mesma (HU, 2024; SUTTON; BARTO, 2018). Dessa forma, a atualização do valor esperado da ação é calculada de forma direta com base na próxima ação que o agente efetivamente escolheu e executou no ambiente (SUTTON; BARTO, 2018):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (2.8)$$

E o *Q-Learning* (WATKINS; DAYAN, 1992), que opera de forma *Off-policy*, aproximando diretamente a função de valor da política ótima independentemente da política comportamental utilizada para explorar o ambiente (HU, 2024; SUTTON; BARTO, 2018):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right] \quad (2.9)$$

Por fim, no *Q-Learning* clássico, a operação de máximo utiliza as mesmas amostras de dados tanto para determinar a melhor ação quanto para calcular o seu valor, o que frequentemente gera um viés de superestimação matemática (SUTTON; BARTO, 2018; HU, 2024). Para mitigar esse problema, introduziu-se o *Double Q-Learning*, que previne a superestimação ao dissociar a seleção da melhor ação da avaliação do seu valor real utilizando duas tabelas ou funções independentes, Q_1 e Q_2 (HASSELT, 2010; HU, 2024). Com probabilidade igual a cada passo, a atualização de Q_1 ocorre selecionando a melhor ação futura com base nela mesma, mas

avaliando seu valor real através de Q_2 (SUTTON; BARTO, 2018):

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1(S_t, A_t) \right] \quad (2.10)$$

O processo ocorre de forma simétrica caso Q_2 seja sorteada para a atualização, garantindo que as estimativas de ambas as funções permaneçam não enviesadas (SUTTON; BARTO, 2018).

Aprendizagem por Reforço Profundo (DRL)

A DRL consolida-se ao utilizar redes neurais profundas para aproximar a função de valor de ação em vez de empregar tabelas clássicas para armazenar os valores Q . Dessa forma, permite-se que o agente aprenda e generalize eficientemente em espaços de estados contínuos e de alta dimensão (MNIH *et al.*, 2015). O algoritmo Redes Q-Deep (*Deep Q-Network*) (DQN) estabiliza esse processo de aprendizado através de duas inovações estruturais (MNIH *et al.*, 2015).

A primeira é a utilização de um *Replay Buffer* (memória de repetição de experiências). Em vez de aprender com as amostras na ordem sequencial exata em que são geradas no ambiente, o agente armazena suas transições no buffer e sorteia mini-lotes aleatórios para o treinamento (MNIH *et al.*, 2015). Isso é necessário porque amostras consecutivas em uma trajetória são correlacionadas no tempo; a amostragem aleatória quebra essa correlação, reduz a variância das atualizações e melhora a eficiência do aprendizado, já que cada experiência pode ser reutilizada múltiplas vezes.

A segunda inovação é a introdução de uma rede alvo (θ^-) (MNIH *et al.*, 2015). No *Q-Learning* padrão com redes neurais, o alvo da atualização depende dos mesmos pesos que estão sendo continuamente ajustados. Isso cria um problema de “alvo móvel”, onde a rede tenta alcançar uma estimativa que também se altera a cada passo, o que frequentemente gera instabilidade, oscilações severas ou divergência da política. Para contornar isso, o algoritmo dissocia o cálculo do alvo da atualização da rede principal. A rede alvo possui a mesma arquitetura, mas seus pesos permanecem congelados e são sincronizados com a rede principal (θ) apenas periodicamente. Essa separação estabiliza o treinamento ao fixar os alvos por um determinado número de passos, minimizando o Erro TD através da seguinte função de perda:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.11)$$

A partir dessas ideias, outros algoritmos foram propostos para sanar limitações estruturais no aprendizado. Como a operação de máximo no DQN induz a um viés de superestimação, o *Double DQN* adapta o conceito de duplo aprendizado para redes neurais, utilizando a rede principal para selecionar a melhor ação futura e a rede alvo para avaliar o valor real dessa mesma ação (HASSELT, 2010; MNIH *et al.*, 2015).

Por fim, a arquitetura *Dueling DQN* aprimora a própria estrutura interna da rede neural, bifurcando a última camada oculta em duas ramificações independentes, uma responsável por estimar o valor do estado (V) e outra para estimar a vantagem relativa de cada ação (A) (WANG *et al.*, 2016). Ao agregar esses dois sinais apenas na camada de saída, o modelo consegue aprender o valor do estado independentemente das ações que são tomadas, o que acelera drasticamente a convergência em cenários onde múltiplas escolhas produzem resultados semelhantes (WANG *et al.*, 2016).

Gradientes de Política e REINFORCE

Diferente dos métodos baseados em valor, os métodos de Gradiente de Política parametrizam a política diretamente, $\pi_\theta(a|s)$, e a otimizam por ascensão de gradiente para maximizar o desempenho esperado $J(\theta)$. O Teorema do Gradiente de Política viabiliza essa otimização ao fornecer uma expressão exata para o gradiente sem exigir o conhecimento prévio das dinâmicas de transição do ambiente. Ao aplicar o truque da razão de verossimilhança, a formulação se transforma na Equação 2.12, que permite o uso de amostragens reais. O objetivo prático dessa equação é simples, ajustar a rede neural para aumentar a probabilidade de uma ação exatamente na proporção do retorno que ela gerou.

$$\nabla_\theta J(\theta) = \mathbb{E} \pi_\theta [\nabla_\theta \log \pi_\theta(A_t|S_t) Q^\pi(S_t, A_t)] \quad (2.12)$$

A partir dessa base teórica, derivou-se o algoritmo REINFORCE (WILLIAMS, 1992). Ele contorna a necessidade de calcular a função de valor $Q^\pi(S_t, A_t)$ estimando-a empiricamente. Ele utiliza o retorno real acumulado ao final do episódio (G_t) como um estimador não enviesado do valor daquela ação (SUTTON; BARTO, 2018). Consequentemente, o agente atualiza os parâmetros da política apenas após a conclusão de cada episódio, movendo os pesos matematicamente na direção que aumenta a probabilidade das ações que geraram altos retornos (G_t), ao mesmo tempo em que inibe aquelas associadas a retornos ruins (SUTTON; BARTO,

2018).

Ator-Crítico, PPO e SAC

As arquiteturas Ator-Crítico foram desenvolvidas para diminuir a alta variância dos métodos de gradiente de política tradicionais (SUTTON; BARTO, 2018). Nessas arquiteturas, o aprendizado é dividido em dois componentes simultâneos: o Ator ajusta as probabilidades de seleção de uma ação para um determinado estado ao seguir uma política $\pi(a|s)$, enquanto o Crítico estima a função de valor $V(s)$ (SUTTON; BARTO, 2018). Essas funções podem utilizar Redes Neurais Profundas (*Deep Neural Networks*) (DNN) de duas maneiras diferentes (HU, 2024). A primeira utiliza redes neurais separadas e independentes para o Ator e para o Crítico, o que confere maior flexibilidade na otimização de cada função, mas exige maior esforço computacional (HU, 2024). A segunda utiliza uma única rede neural com pesos compartilhados, onde as camadas ocultas iniciais extraem características comuns do estado e, em seguida, bifurcam-se em ramificações de saída independentes, uma para calcular a probabilidade de seleção da ação e outra para estimar a função de valor (HU, 2024).

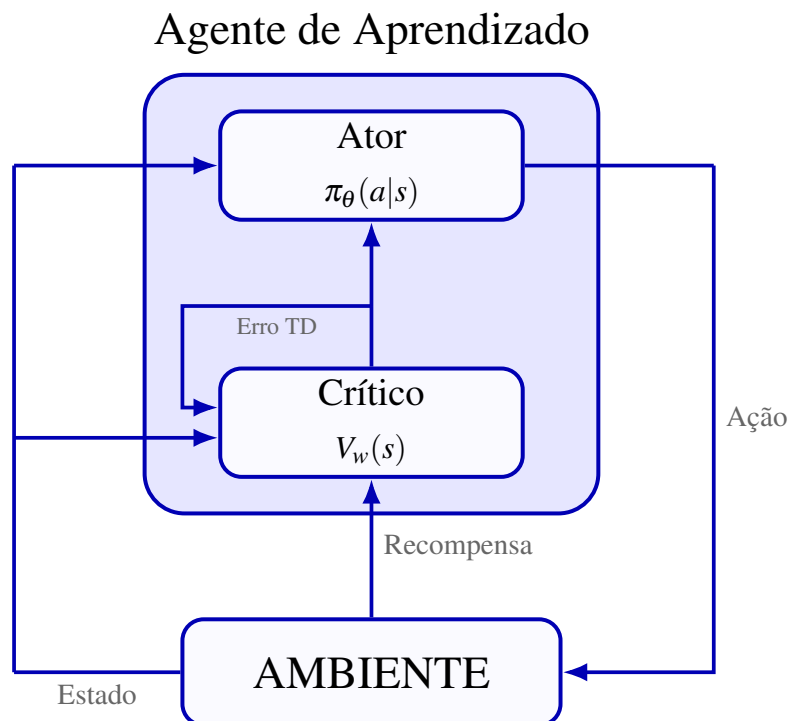


Figura 12 – Dinâmica Ator-Crítico.

Fonte: Elaborado pelo autor com base em (SUTTON; BARTO, 2018).

A Figura 12 ilustra a dinâmica clássica de interação em uma arquitetura Ator-Crítico. No início, o agente obtém do ambiente o estado atual do problema, que é passado para a entrada

da DNN em ambos os componentes do Agente de Aprendizado. Com base nesse estado, o Ator emprega sua política $\pi_{\theta}(a|s)$ para selecionar e aplicar uma ação no ambiente, o que causa a transição para um novo estado e a emissão de um sinal de recompensa. O Crítico utiliza essa nova informação para calcular o Erro TD ($\delta = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$), que mede a adequação da escolha realizada comparada ao que era esperado. Esse erro é então propagado tanto para atualizar os próprios parâmetros do Crítico – aprimorando suas futuras estimativas de valor $V_w(s)$ – quanto para escalar os gradientes e guiar a atualização dos pesos do Ator, reforçando ações que resultaram em retornos maiores que o esperado e desencorajando escolhas subótimas (SUTTON; BARTO, 2018).

2.5 TOPSIS e Fuzzy-TOPSIS

Nesta seção, serão apresentados os métodos de Tomada de Decisão Multicritério (*Multiple-Criteria Decision-Making*) (MCDM) TOPSIS e sua extensão *Fuzzy-TOPSIS*. Esses métodos são responsáveis por ranquear um conjunto de alternativas com base em múltiplos critérios simultaneamente. Como etapa intermediária para a validação da arquitetura proposta neste trabalho, o TOPSIS clássico foi inicialmente integrado ao algoritmo SAC para a resolução do problema de descarregamento de tarefas, permitindo a avaliação dos candidatos e a redução do espaço de observação. No entanto, o objetivo final da pesquisa consiste em incorporar o *Fuzzy-TOPSIS* para mitigar as incertezas e a imprecisão das medições inerentes aos cenários veiculares de alta mobilidade, modelando a incerteza linguística nos critérios de avaliação.

O Algoritmo TOPSIS

O TOPSIS é um método de MCDM que ranqueia alternativas pela proximidade relativa à Solução Ideal Positiva (*Positive Ideal Solution*) (PIS) (melhor valor possível em cada critério) e pela distância à Solução Ideal Negativa (*Negative Ideal Solution*) (NIS) (pior valor possível). Uma alternativa é preferível se estiver simultaneamente próxima da PIS e distante da NIS. A Figura 13 ilustra esse conceito de forma geométrica em um espaço bidimensional. Nela e na formulação matemática do método, a letra d representa a distância euclidiana entre os pontos, os sobrescritos sinalizam qual é a fronteira de referência sendo medida (onde $^+$ simboliza o cálculo em direção à PIS, e $^-$ em direção à NIS), e o subscrito numérico refere-se a qual candidato A_i está sendo avaliado. Observa-se que a alternativa A_1 , por exemplo, apresenta uma distância menor em direção à métrica ideal positiva (d_1^+) e uma distância maior para a solução

anti-ideal (d_1^-), tornando-se preferível em comparação direta com a alternativa A_3 .

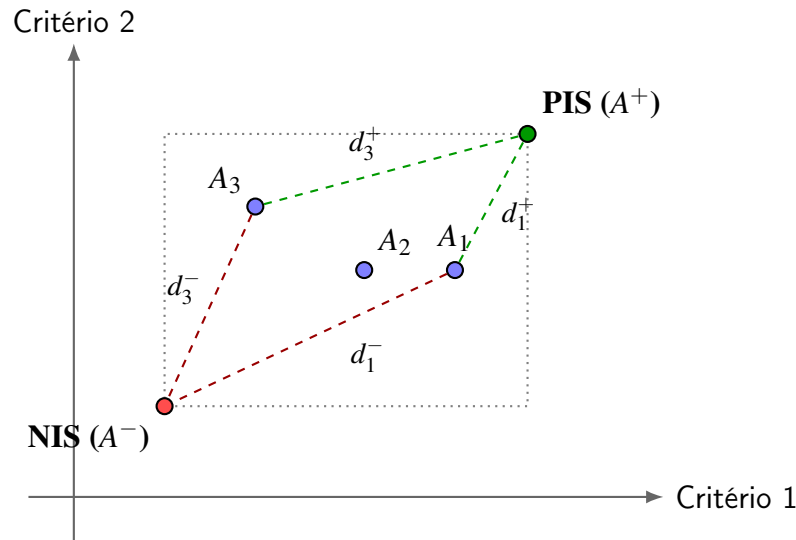


Figura 13 – Representação geométrica bidimensional do ranqueamento TOPSIS baseada em critérios de benefício.

Fonte: Elaborado pelo autor.

De forma geral, o TOPSIS organiza os dados em formato de matriz ($\mathbb{R}^{m \times n}$) e ranqueia m alternativas candidatas simultaneamente considerando n critérios através de um conjunto de passos. Tal procedimento, está abstraído no Algoritmo 1. Ele funciona em três fases principais. Inicialmente, os dados de entrada são normalizados por coluna (linha 3), o que unifica as métricas para que possuam a mesma escala, permitindo a comparação de critérios com naturezas e unidades diferentes. Em seguida, os valores normalizados são multiplicados por um vetor de pesos $\mathbf{w}$ que define a prioridade de cada critério (linha 4).

Com os dados padronizados, o método identifica os pontos extremos. Para cada critério de benefício, busca-se o valor máximo; para critérios de custo, o valor mínimo. O agrupamento dos melhores valores forma a PIS, enquanto os piores formam a NIS (linhas 7–11).

Por último, o TOPSIS calcula a distância Euclidiana de cada candidato até essas duas soluções ideais (linhas 15 e 16). O coeficiente de proximidade $C_i^* \in [0, 1]$ é obtido calculando a razão entre a distância à NIS e a distância total para ambos os extremos (linha 17). As alternativas são então ranqueadas ordenando C_i^* decrescentemente (linha 19). O uso específico da distância Euclidiana garante que alternativas com falhas simultâneas em vários critérios sofram penalidades maiores, valorizando soluções balanceadas.

Algoritmo 1: Processo de Avaliação Multi-critério TOPSIS

Entrada : Matriz de decisão $\mathbf{X} \in \mathbb{R}^{m \times n}$ com m alternativas em n critérios; vetor normalizado de pesos $\mathbf{w}$.

Saída : Vetor de proximidade $\mathbf{C}^*$ e alternativas ranqueadas decorrentemente.

// Passos 1 e 2: Normalização Vetorial e Ponderação

```

1 for j = 1 to n do
2   for i = 1 to m do
3      $r_{ij} \leftarrow x_{ij} / \sqrt{\sum_{k=1}^m x_{kj}^2}$ ;
4      $v_{ij} \leftarrow w_j \cdot r_{ij}$ ;
5   end
6   // Passo 3: Dimensionamento das Fronteiras PIS ( $A^+$ ) e NIS ( $A^-$ )
7   if critério j é de benefício then
8      $A_j^+ \leftarrow \max_i v_{ij}$ ;
9      $A_j^- \leftarrow \min_i v_{ij}$ ;
10  else
11     $A_j^+ \leftarrow \min_i v_{ij}$ ;           // Critério de custo (minimizar)
12     $A_j^- \leftarrow \max_i v_{ij}$ ;
13  end
14 end

// Passos 4 e 5: Distâncias Euclidianas e Coeficiente de Similaridade
14 for i = 1 to m do
15    $d_i^+ \leftarrow \sqrt{\sum_{j=1}^n (v_{ij} - A_j^+)^2}$ ;
16    $d_i^- \leftarrow \sqrt{\sum_{j=1}^n (v_{ij} - A_j^-)^2}$ ;
17    $C_i^* \leftarrow d_i^- / (d_i^+ + d_i^-)$ ;
18 end
19 return Ordenação decrescente de  $\mathbf{C}^*$ ;

```

Extensão Fuzzy do TOPSIS

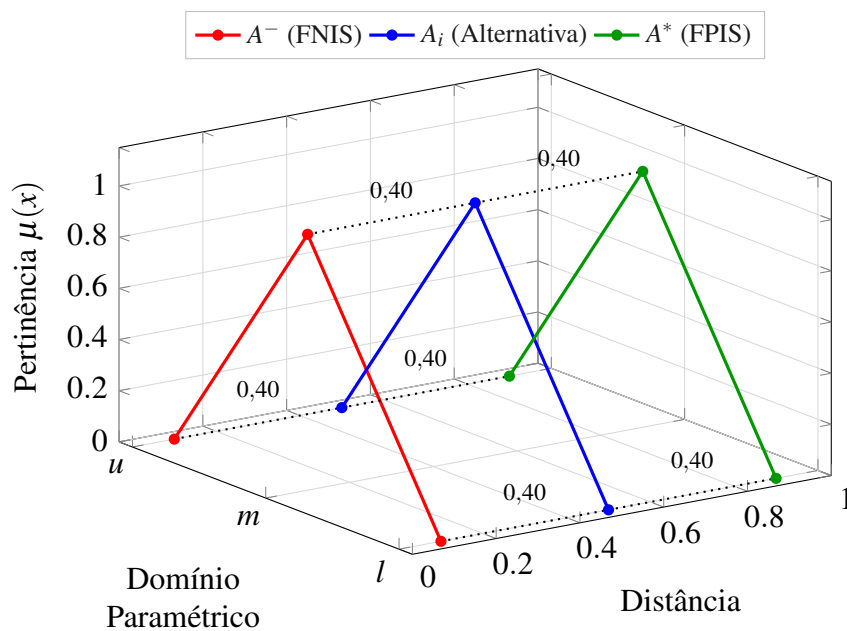
A lógica *fuzzy* (ou lógica difusa) fornece uma estrutura matemática para tratar a incerteza e a imprecisão inerentes aos fenômenos físicos e a informações qualitativas ou incompletas (ZADEH, 1965; CRUZ, 2014). Nela, diferentemente da lógica clássica, que é rígida e restringe os resultados a valores absolutos de verdadeiro (1) ou falso (0), a lógica *fuzzy* utiliza valores no intervalo contínuo $[0, 1]$ para representar o grau de adequação (pertinência) de uma variável, permitindo que se trabalhe de forma suave com as chamadas “meias-verdades” (CRUZ, 2014).

No contexto do TOPSIS, o método determinístico exige valores escalares precisos. Contudo, em situações onde os dados de entrada possuem incertezas ou ruídos de medição, o

uso de valores pontuais pode levar a instabilidades no ranqueamento. A extensão *Fuzzy-TOPSIS* substitui os valores escalares por Número *Fuzzy Triangular* (*Triangular Fuzzy Number*)s (TFNs), permitindo que o modelo considere intervalos de confiança em vez de pontos fixos.

O processo de cálculo segue a lógica do TOPSIS convencional, mas adaptado para operações com números *fuzzy*. As distâncias entre as alternativas e as soluções ideais (PIS e NIS) são calculadas utilizando métricas de distância espacial iterando sobre os vértices dos conjuntos geométricos, conforme ilustrado pela abstração na Figura 14. Ao final, o valor *fuzzy* resultante é defuzzificado para um escalar, permitindo a ordenação das alternativas. Esta abordagem torna o ranqueamento mais robusto a variações nos dados de entrada, uma vez que a incerteza é explicitamente modelada durante todo o processo de decisão.

Figura 14 – Distância de Alternativas para as Soluções Ideais no *Fuzzy-TOPSIS*



Fonte: Elaborado pelo autor

3 REVISÃO DE LITERATURA

Este capítulo tem como objetivo apresentar uma revisão bibliográfica aprofundada dos trabalhos recentes e relevantes para o desenvolvimento desta proposta. Os critérios de seleção englobaram pesquisas focadas na interseção entre comunicação veicular emergente e soluções de aprendizado de máquina adaptativo voltadas para o descarregamento de tarefas dinâmico.

Para classificar e relacionar o estado da arte das soluções de descarregamento de tarefas em ambientes de VEC e MEC, adotou-se uma versão adaptada da taxonomia proposta em (SOUZA *et al.*, 2020), sendo ela dividida em duas vertentes estruturais: Padrão de Comunicação, que avalia as tecnologias de rede e os servidores utilizados; e Problema, que analisa as estratégias com ênfase em modelos DRL. Dessa maneira, os artigos relacionados foram classificados conforme a Figura 15.

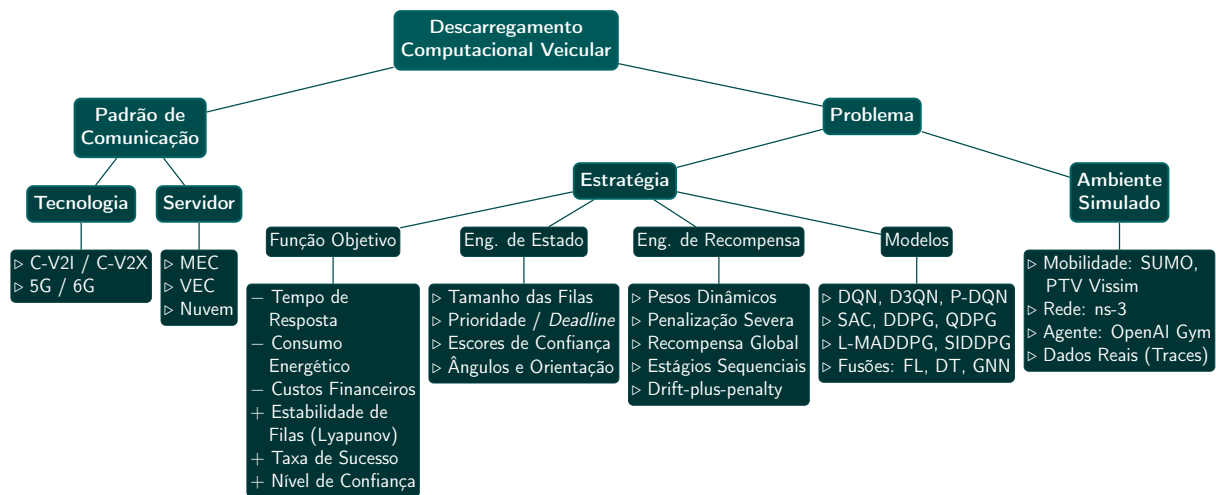


Figura 15 – Taxonomia adaptada para classificação dos trabalhos relacionados em descarregamento computacional veicular, com ênfase na modelagem de Aprendizado por Reforço Profundo (DRL) e ambiente de simulação.

Fonte: Adaptado de Souza *et al.* (2020).

3.1 Padrão de Comunicação

No descarregamento de tarefas, o cenário de comunicação celular vem ganhando bastante destaque. Neste quesito, suas tecnologias vêm evoluindo rapidamente e ditam a maioria das pesquisas aqui apresentadas. A seguir são apresentados alguns trabalhos que utilizam tecnologias 5G e além (Além do 5G (*Beyond 5G*) (B5G)/6G) para atender às exigentes demandas

relacionadas a aplicações veiculares.

3.1.1 Tecnologia

A evolução das tecnologias de comunicação veicular destaca a transição de padrões tradicionais, como o DSRC e o IEEE 802.11bd, para a comunicação celular aliada à IA. Essa mudança visa, sobretudo, possibilitar baixas latências e alta vazão de dados em cenários de alta mobilidade, contribuindo para resolver o problema de descarregamento de tarefas. Nesse contexto, diversos trabalhos, como os de (MOGHADDASI *et al.*, 2023; SIDWINI *et al.*, 2024; LI *et al.*, 2025; MEGHAMALA; PAUL; KUMAR, 2025), utilizam interfaces C-V2X e arquiteturas de borda para o compartilhamento de informações em aplicações veiculares avançadas.

O estudo de (MOGHADDASI *et al.*, 2023) propõe um algoritmo de DRL, especificamente o Redes Q-Deep Duplas (*Double Deep Q-Network*) (DDQN), em redes 5G para solucionar o problema de descarregamento de tarefas utilizando a comunicação V2I para alcançar eficiência energética e a diminuição da latência. O trabalho de (SIDWINI *et al.*, 2024) explora o uso de algoritmos avançados de DRL, como o SAC, para otimizar o descarregamento dinâmico entre dispositivos locais, servidores de borda e a nuvem, equilibrando o consumo de energia e o atraso na execução. Em um cenário direcionado aos sistemas 6G, (LI *et al.*, 2025) introduz um esquema inteligente baseado em DQN em redes V2X assistidas por MEC para otimizar conjuntamente a associação aos servidores, a taxa de descarregamento de tarefas e a aceleração dos veículos. Essa abordagem visa minimizar o atraso do sistema e evitar colisões de trânsito, utilizando uma função de recompensa segmentada que respeita restrições de atraso, energia e segurança. Por fim, (MEGHAMALA; PAUL; KUMAR, 2025) propõe um *framework* de Aprendizado por Reforço Multiagente (*Multi-Agent Reinforcement Learning*) (MARL) para redes 5G que combina o algoritmo SAC com arquiteturas hierárquicas (Multiagente Twin Delayed DDPG (*Multi-Agent Twin Delayed Deep Deterministic Policy Gradient*) (MATD3) e *Task Descriptor Policy Gradient* (TD2PG)). Essa solução visa a alocação dinâmica de subcanais, potência de transmissão e recursos computacionais em servidores MEC, levando em consideração a mobilidade dos usuários e o isolamento das fatias de rede (*network slicing*).

3.1.2 Servidor

Quando há necessidade de transferência de responsabilidade de processamento de tarefas, existem três opções viáveis: processar as tarefas em (i) um dispositivo de infraestrutura

de borda, (ii) recursos ociosos das unidades processadoras dos veículos vizinhos, ou (iii) uma robusta infraestrutura de nuvem, que apresenta maior capacidade, mas latência consideravelmente superior. A seguir, é descrito como as principais opções locais foram modeladas.

MEC

Servidores MEC contribuem significativamente para atender exigências de URLLC em 5G NR. Eles ficam localizados na borda da rede e podem ser constituídos de equipamentos robustos, capazes de auxiliar no processamento pesado de tarefas delegadas por outros dispositivos com recursos limitados.

Nesse contexto, esses servidores frequentemente estão conectados a RSU e estações base para oferecer suporte computacional de baixa latência aos veículos. Aproveitando essa infraestrutura de borda, o estudo de (ZHAO *et al.*, 2025) propõe o esquema DQN Preditivo e Emparelhamento de Kuhn-Munkres (*Predictive DQN and Kuhn-Munkres scheme*) (PDQKM) para otimizar o descarregamento de múltiplas tarefas, permitindo reduzir a latência total do sistema. Por outro lado, o trabalho de (WU *et al.*, 2025) utiliza Redes Adversárias Generativas (*Generative Adversarial Networks*) (GAN) para expandir amostras de tarefas evitando o desequilíbrio de dados, criando um modelo de Aprendizado Federado (*Federated Learning*) (FL) para realizar o descarregamento de tarefas de forma energeticamente eficiente em servidores MEC. O estudo de (MOHAMMAD *et al.*, 2025) apresenta o *framework* EMEC-IoVTOF, que integra DRL com a técnica de Otimização por Enxame de Partículas (*Particle Swarm Optimization*) (PSO) para otimizar a alocação de recursos em sistemas IoV. A solução calcula custos de descarregamento para evitar a convergência para ótimos locais, conseguindo reduções significativas de latência e consumo de energia, além de contornar restrições de largura de banda em cenários veiculares dinâmicos. Por sua vez, o estudo de (LIU *et al.*, 2024) utiliza o método de aproximação de Bernstein para converter restrições probabilísticas de interferência em formato tratável, minimizando uma função de utilidade que equilibra atrasos na transmissão e na computação para o Computação de Borda de Múltiplos Acessos Assistida por Nuvem (*Cloud-assisted Multi-access Edge Computing*) (C-MEC).

VEC

VEC distribui poder computacional formando micro-nuvens entre os próprios veículos na estrada, empregando recursos ociosos de suas unidades embarcadas (OBU) para auxiliar

vizinhos próximos através de canais de comunicação direta, frequentemente desassociados de infraestruturas estáticas da pista. O estudo de (XIE *et al.*, 2025) propõe utilizar Superfície Inteligente Reconfigurável (*Reconfigurable Intelligent Surface*) (RIS) juntamente com VEC para reduzir custos de implantação, ao mesmo tempo que busca maximizar Eficiência Operacional do Sistema (*System Operation Efficiency*) (SOE) via algoritmo PPO, otimizando conjuntamente decisões de descarregamento, alocação de recursos computacionais e vetores de mudança de fase. Por sua vez, (WU; GU; LIANG, 2025) utiliza FL e Ator-Crítico Determinístico Profundo Multiagente (*Multi-Agent Deep Deterministic Policy Gradient*) (MADDPG), empregando mecanismos de incentivo baseados em reputação para determinar a proporção ideal de recursos cedidos por veículos vizinhos, visando minimizar latência e consumo de energia e garantindo alta taxa de conclusão em tarefas computacionalmente intensivas. O trabalho de (LONG *et al.*, 2025) desenvolve o algoritmo Otimização de Alocação de Recursos e Divisão de Tarefas Nuvem-Terminal (*Cloud-Terminal Task Splitting and Resource Allocation Optimization*) (CTSRAO), que integra dispositivos finais, borda veicular e nuvem para realizar descarregamento parcial de tarefas, separando o problema em subproblemas convexos para otimizar de forma colaborativa o *task splitting* (taxa de divisão de tarefas) e a alocação de recursos computacionais, alcançando equilíbrio de desempenho global do sistema. Em (ABBAS; XU; ZHANG, 2024), propõe-se um algoritmo baseado em aprendizado descentralizado colaborativo, que orquestra a execução concorrente de tarefas fragmentadas pelos veículos presentes na área de cobertura, otimizando conjuntamente a alocação de canais sem fio com os recursos computacionais. Dessa forma, minimiza-se a latência total e o consumo de energia em ambientes veiculares dinâmicos.

Discussão

Analisando a Tabela 2, constata-se forte tendência de padronização de tecnologias celulares avançadas para comunicação veicular. Grande parte dos trabalhos recentes analisados concentra-se na utilização de conectividade V2I via redes 5G e 6G. Em relação à comunicação entre os nós, embora o padrão C-V2X englobe diferentes interfaces, diversos trabalhos referem-se estritamente à utilização das interfaces *Sidelink* destas comunicações para troca direta de dados (V2V). Já em relação à infraestrutura, a adoção de servidores MEC centralizados em unidades RSU ainda é predominante, mas observa-se tendência crescente de pesquisas que exploram VEC, tratando os próprios veículos como provedores de recursos. Trabalhos como os de (LIU *et al.*, 2024) e (LONG *et al.*, 2025) adotam arquiteturas colaborativas integrando MEC com nuvem e

Tabela 2 – Resumo dos aspectos de Padrão de Comunicação, Servidor e Condições de Ambiente dos trabalhos relacionados

Referência	Tecnologia		Servidor			Observabilidade Parcial (Incerteza de Estado)	Canal Perfeito	Carga de Trabalho (Diferentes regimes)
	V2I	<i>Sidelink</i> (V2V)	MEC	VEC	Nuvem			
(MOGHADDASI <i>et al.</i> , 2023)	✓		✓				✓	✓
(SIDWINI <i>et al.</i> , 2024)	✓		✓		✓		✓	
(LI <i>et al.</i> , 2025)		✓	✓				✓	
(MEGHAMALA; PAUL; KUMAR, 2025)	✓		✓				✓	
(ZHAO <i>et al.</i> , 2025)	✓		✓				✓	
(WU <i>et al.</i> , 2025)	✓		✓				✓	
(MOHAMMAD <i>et al.</i> , 2025)		✓	✓				✓	
(LIU <i>et al.</i> , 2024)	✓		✓		✓			
(XIE <i>et al.</i> , 2025)	✓			✓			✓	
(WU; GU; LIANG, 2025)		✓		✓			✓	
(LONG <i>et al.</i> , 2025)	✓	✓		✓	✓		✓	
(ABBAS; XU; ZHANG, 2024)	✓			✓			✓	

Fonte: Elaborado pelo autor

VEC com nuvem, respectivamente.

Outro aspecto identificado foi a ausência de tratamento para a observabilidade parcial, caracterizada na tabela pela coluna "Incerteza de Estado". Todos os trabalhos identificados na tabela modelam o ambiente assumindo conhecimento perfeito do cenário, como a disponibilidade de CPU dos servidores no exato momento da tomada de decisão. Dessa forma, agentes independentes podem decidir descarregar tarefas para um mesmo servidor simultaneamente com base em informações desatualizadas. Além disso, não há menção nos trabalhos sobre imperfeições do canal de comunicação. Portanto, assumir tais características cria políticas de decisão pouco robustas frente à dinâmica real.

Outra limitação identificada diz respeito à modelagem de cargas de trabalho. Com exceção do estudo de (MOGHADDASI *et al.*, 2023), os demais trabalhos avaliam seus algoritmos sob regimes de tráfego constantes ou com taxa de chegada fixa por episódio. Em cenários de tráfego reais, eventos imprevisíveis podem causar rajadas instantâneas (*bursts*) no envio de pacotes críticos, alterando a distribuição de chegada das tarefas. Modelos DRL treinados exclusivamente sem variação dinâmica de carga tendem a sofrer degradação de desempenho com tais mudanças de regime.

Nesse contexto, o modelo proposto neste trabalho visa endereçar essas lacunas ao modelar, desde a concepção do ambiente, tais dificuldades e estocasticidades inerentes a redes veiculares reais, conforme a adoção das tecnologias supramencionadas.

3.2 Problema

Com a infraestrutura de comunicação estabilizada, a escolha de abordagens apropriadas constituem um passo crítico para a garantia dos requisitos no ambiente veicular. A formulação de soluções para o descarregamento de tarefas dinâmico e em tempo real apresenta elevada complexidade. Para contornar essa dificuldade, diversos trabalhos na literatura optam por simplificar o escopo, desenvolvendo abordagens focadas em uma única métrica, como a minimização exclusiva da latência ou do consumo de energia (FENG *et al.*, 2017; HOU *et al.*, 2016; REN *et al.*, 2018; REN *et al.*, 2019; ZHANG *et al.*, 2019). No entanto, o estudo de (QIN *et al.*, 2020) enfatiza que otimizar um único objetivo isoladamente não reflete a realidade das redes, enquanto (ZHANG; YI; MA, 2024) reitera que a busca por um compromisso dinâmico entre tempo de resposta e consumo energético transforma a alocação em um problema incerto de otimização multiobjetivo NP-difícil. Diante dessa constatação, observa-se a necessidade do desenvolvimento de políticas que considerem múltiplos critérios simultaneamente, com o intuito de projetar propostas robustas e capazes de atender às demandas operacionais e funcionais de longo prazo.

A modelagem de um cenário realista precisa lidar fundamentalmente com incertezas espaciais da rede. A alta velocidade dos veículos altera significativamente a infraestrutura veicular, mascarando os estados reais e comprometendo a acurácia dos dados divulgados pelos nós de computação (LIU *et al.*, 2024). A somatória entre a imprevisibilidade de processos no interior da borda (ABBAS; XU; ZHANG, 2024) e o natural decréscimo de precisão proveniente do atraso por telemetria exige uma abordagem maleável. Decisões sustentadas integralmente por lógicas determinísticas tornam-se insatisfatórias, contribuindo para que os modelos não atendam adequadamente os requisitos de qualidade no descarregamento de tarefas (KANDALI *et al.*, 2025; ZHAO *et al.*, 2025).

3.2.1 Estratégia

Para contornar grande parte dos vieses supracitados, a literatura contemporânea aponta uma considerável substituição do uso de heurísticas e meta-heurísticas por soluções mais sofisticadas e adaptativas. A respeito desta transformação, destaca-se a utilização de DRL, por conta de sua capacidade de aprimorar sequências de ações de longo prazo extraídas através de intensas interações com ambiente simulado.

De fato, o escopo científico atual tem explorado ativamente o uso de diversas

estratégias integradas ao DRL. Uma delas é a integração com aprendizado FL, que permite o treinamento colaborativo de modelos DRL entre múltiplos agentes, preservando a privacidade dos dados locais (CHEN *et al.*, 2025). Outra frente mescla esquemas DRL com tecnologia Gêmeo Digital (*Digital Twin*) (DT), possibilitando que soluções sejam testadas e simuladas virtualmente antes da execução no ambiente físico (WEI *et al.*, 2024). Para lidar com o vasto espaço de estados e com cenários altamente variáveis, pesquisas recentes fundem arquiteturas DRL com redes Neurais em Grafos (*Graph Neural Networks*) (GNN) para capturar de forma eficiente as relações e as dependências espaciais entre os nós veiculares (WANG *et al.*, 2024; TANG *et al.*, 2024). Adicionalmente, métodos Aprendizado por Transferência (*Transfer Learning*) (TL) têm sido utilizados para acelerar a adaptação de veículos recém-conectados em infraestruturas baseadas em SDN, onde a separação entre os planos de controle e de dados facilita a gerência global (BAKER *et al.*, 2024). Por fim, destaca-se a incorporação da teoria de otimização de Lyapunov acoplada a métodos DRL para garantir matematicamente a estabilidade contínua das filas de tarefas a longo prazo (KUMAR; ZHAO; FERNANDO, 2023).

Função Objetivo

Além do compromisso entre a minimização da latência de processamento e a redução do consumo de energia, a literatura recente incorpora a otimização de custos operacionais e financeiros da rede (MA *et al.*, 2025). A estabilidade das filas de tarefas (*queue stability*) a longo prazo também é um objetivo considerado, frequentemente abordado por meio da otimização de Lyapunov acoplada a técnicas de aprendizado (KUMAR; ZHAO; FERNANDO, 2023). Outros estudos buscam maximizar a taxa de sucesso das tarefas sob restrições de prioridade e da capacidade dos nós de computação (KUMARI *et al.*, 2025). Adicionalmente, o gerenciamento de confiança (*trust management*) dos nós de borda desponta como um objetivo crítico em infraestruturas SDN para evitar descarregamentos falhos ou maliciosos (BAKER *et al.*, 2024).

Engenharia de Estado e Recompensa

No MDP, a engenharia do espaço de estado não busca prever matematicamente as transições do sistema, uma vez que algoritmos DRL se destacam justamente por gerenciar o controle em ambientes estocásticos e dinâmicos sem conhecimento prévio dessas transições (FAN; CAI, 2024). Em vez disso, o estado é projetado para permitir que o agente avalie diretamente a dinâmica da rede com base nos resultados observados. Para isso, o espaço de estado deve

utilizar métricas além das básicas, incorporando variáveis que capturem a dinamicidade dos nós de computação (KUMAR; ZHAO; FERNANDO, 2023) ou até mesmo escores de confiança dos dispositivos candidatos (BAKER *et al.*, 2024). A literatura recente expande ainda mais essa abstração ao integrar as características das tarefas — englobando requisitos de recursos, níveis de prioridade e prazos máximos toleráveis (*deadlines*) (FAN; CAI, 2024; WEI *et al.*, 2024). Além disso, para refinar a percepção física, as pesquisas incluem o ângulo de desvio e a orientação direcional dos veículos em relação aos servidores. Essa consciência espacial de granularidade fina complementa o estado atual, otimizando a escolha do nó alvo de forma proativa (FARIMANI *et al.*, 2024).

A respeito da função de recompensa, é uma boa prática (SUTTON; BARTO, 2018) realizar o dimensionamento adequado (*scaling*) das métricas e a normalização dos dados para evitar que uma única variável domine o processo de convergência. Além disso, alguns trabalhos utilizam pesos dinâmicos em vez de estáticos, ajustando-os em tempo real de acordo com as condições momentâneas da rede e com o nível de urgência das tarefas (KUMARI *et al.*, 2025; FAN; CAI, 2024; SHI; CHEN; ZHU, 2023). Já a penalização para quando um agente escolhe uma ação ruim — como a violação do tempo máximo tolerável (*deadline*) ou quebras de comunicação por evasão de cobertura — é integrada à recompensa para guiar o agente a priorizar o cumprimento dos requisitos de latência e confiabilidade. Também se observa uma forte transição para mecanismos de recompensa globais, nos quais os agentes colaboram buscando a eficiência sistêmica da rede, mesmo que em detrimento do custo ótimo individual imediato (FARIMANI *et al.*, 2024). Em cenários de descarregamento parcial, a recompensa é formulada dividindo a ação em múltiplos estágios sequenciais, avaliando o impacto do particionamento da tarefa de forma independente da alocação de recursos (TIAN *et al.*, 2025). Por fim, a teoria de Lyapunov é frequentemente utilizada na recompensa por meio de funções *drift-plus-penalty*, garantindo matematicamente que o agente seja penalizado caso a redução excessiva do consumo energético coloque em risco a estabilidade das filas do sistema a longo prazo (KUMAR; ZHAO; FERNANDO, 2023).

Modelos

No que tange aos modelos, existem diversos trabalhos focados em otimizar a forma como o agente observa o estado e seleciona ações eficientemente. Variantes baseadas em valor, como redes DQN aprimoradas, realizam a seleção de servidores reconhecendo que nem todas

as tarefas computacionais geradas possuem a mesma importância ou urgência (KUMARI *et al.*, 2025). Para espaços de ação contínuos, arquiteturas Ator-Crítico dominam o cenário. O algoritmo SAC é empregado para evitar a convergência prematura para ótimos locais, utilizando a maximização da entropia estocástica para explorar melhor as opções de descarregamento e *caching* (BAO *et al.*, 2025). Para lidar com o problema de espaços de ação híbridos — nos quais decisões discretas de seleção de servidor são combinadas com a alocação contínua de recursos computacionais —, alguns trabalhos propõem métodos como o algoritmo Rede Q Profunda Parametrizada (*Parametrized Deep Q-Network*) (P-DQN) (MA *et al.*, 2025) ou mesmo fusões entre modelos, como o Gradiente de Política com Q-Learning e DDPG (*Q-learning and Deep Deterministic Policy Gradient*) (QDPG), que une as arquiteturas Gradiente de Política Determinístico Profundo (*Deep Deterministic Policy Gradient*) (DDPG) e Rede Q Profunda Dupla com Dueling (*Dueling Double Deep Q-Network*) (D3QN) aliadas a algoritmos de clusterização *K-Means* (WEI *et al.*, 2024). Em sistemas compostos por múltiplos agentes, o modelo MADDPG com Otimização de Lyapunov (*Lyapunov-based Multi-Agent Deep Deterministic Policy Gradient*) (L-MADDPG) utiliza a otimização de Lyapunov para garantir matematicamente a estabilidade do sistema a longo prazo (KUMAR; ZHAO; FERNANDO, 2023), enquanto abordagens como o modelo DDPG com Iteração de Estados Internos Sequenciais (*Sequential Internal-state Deep Deterministic Policy Gradient*) (SIDDPG) iteram múltiplos estados internamente para refinar o particionamento de sub-tarefas de forma sequencial (TIAN *et al.*, 2025). A Tabela 3 mapeia alguns algoritmos DRL ao tipo de espaço de ação adequado.

Tabela 3 – Classificação dos algoritmos clássicos de DRL quanto à adequação ao espaço de ação

Algoritmo DRL	Espaço de Ação	Referência Base
DQN (<i>Deep Q-Network</i>)	Discreto	(UDDIN; ZHANG; SAKR, 2025; FARIMANI <i>et al.</i> , 2024)
Double DQN	Discreto	(UDDIN; ZHANG; SAKR, 2025)
Dueling DQN	Discreto	(UDDIN; ZHANG; SAKR, 2025)
DDPG (<i>Deep Deterministic Policy Gradient</i>)	Contínuo	(UDDIN; ZHANG; SAKR, 2025; FARIMANI <i>et al.</i> , 2024)
<i>Twin Delayed Deep Deterministic Policy Gradient</i> (TD3) (<i>Twin Delayed DDPG</i>)	Contínuo	(UDDIN; ZHANG; SAKR, 2025)
PPO (<i>Proximal Policy Optimization</i>)	Discreto e Contínuo	(UDDIN; ZHANG; SAKR, 2025)
TRPO (<i>Trust Region Policy Optimization</i>)	Discreto e Contínuo	(UDDIN; ZHANG; SAKR, 2025)
SAC (<i>Soft Actor-Critic</i>)	Contínuo	(UDDIN; ZHANG; SAKR, 2025; FARIMANI <i>et al.</i> , 2024)

Fonte: Elaborado pelo autor

Ambiente Simulado

O treinamento e a validação de algoritmos de aprendizado DRL voltados para redes veiculares exigem ambientes simulados capazes de replicar a natureza não estacionária e volátil do mundo real. Diferente de aplicações computacionais estáticas, os modelos precisam interagir com um ecossistema que engloba tanto a física da movimentação quanto a complexidade das comunicações sem fio e da carga de processamento (UDDIN; ZHANG; SAKR, 2025).

Para modelar a mobilidade com precisão, simuladores de tráfego microscópico, como o *SUMO* (Simulation of Urban MObility) (LOPEZ *et al.*, 2018) e o *PTV Vissim* (PTV Group, 2020), são amplamente adotados na literatura (SOUZA *et al.*, 2021; KUMAR; ZHAO; FERNANDO, 2023). Ferramentas desse tipo permitem a criação de malhas rodoviárias e urbanas e simulam o comportamento dinâmico dos veículos por meio de modelos matemáticos, capturando variações de velocidade, aceleração e respostas a semáforos (LI *et al.*, 2025). Em paralelo, a dinâmica das comunicações — que inclui o desvanecimento do canal, perdas de propagação e bloqueios físicos (condições de visada *LoS/NLoS*) — é frequentemente emulada por simuladores de rede de eventos discretos, destacando-se o ns-3, o qual costuma ser integrado ao SUMO para formar plataformas de co-simulação (SOUZA *et al.*, 2021; UDDIN; ZHANG; SAKR, 2025).

Em relação ao treinamento do modelo de inteligência artificial, o ambiente de simulação é abstraído matematicamente para que o agente observe os estados e tome decisões. Para viabilizar esse treinamento, trabalhos recentes constroem ambientes customizados baseados em padrões como o *OpenAI Gym* (recentemente sucedido pelo *Gymnasium*) (BROCKMAN *et al.*, 2016), que gerenciam o ciclo de observação do estado, execução da ação e recebimento da recompensa (UDDIN; ZHANG; SAKR, 2025) para treinar e otimizar os pesos das redes neurais (MOGHADDASI; RAJABI; GHAREHCHOPOGH, 2024; KUMAR; ZHAO; FERNANDO, 2023; WU *et al.*, 2025).

Por fim, para evitar que o treinamento ocorra de forma puramente teórica, a literatura atual enfatiza o uso de dados do mundo real (*real-world traces*) dentro do ambiente de simulação (FARIMANI *et al.*, 2024). Algumas abordagens incorporam trajetórias extraídas de frotas reais ou bases de dados de servidores comerciais — como o *Google Cluster Dataset* — para emular as flutuações estocásticas na geração de tarefas e a exigência de recursos de CPU, Memória e Largura de Banda (LONG *et al.*, 2025). Essa imersão de dados reais no ambiente simulado garante que as políticas de descarregamento aprendidas pelo agente DRL

sejam aplicáveis e estáveis diante das dinâmicas de arquiteturas VEC no mundo físico.

Discussão

Tabela 4 – Análise comparativa dos trabalhos relacionados por Função Objetivo, Estado, Recompensa e Ambiente Simulado

Referência	Função Objetivo				Estado				Recompensa				Ambiente Simulado				
	Latência	Energia	Utilidade	Financeiro	Tarefa	Recursos	Tendências	Escore	Escalonamento	Pesos		Penalização		Mobilidade		DES	
										Estáticos	Dinâmicos	Discreta	Contínua	SUMO	Vissim	Customizado	Simulador
(BAKER <i>et al.</i> , 2024)	✓	✓	✓			✓		✓	✓		✓				✓		
(WEI <i>et al.</i> , 2024)	✓	✓			✓	✓			✓	✓			✓			✓	
(CHEN <i>et al.</i> , 2025)	✓					✓	✓			✓		✓				✓	
(MA <i>et al.</i> , 2025)	✓	✓		✓		✓			✓	✓		✓				✓	
(KUMAR; ZHAO; FERNANDO, 2023)	✓	✓	✓			✓	✓		✓				✓	✓	✓	✓	
(KUMARI <i>et al.</i> , 2025)	✓	✓	✓		✓	✓			✓		✓	✓				✓	
(TIAN <i>et al.</i> , 2025)	✓	✓				✓			✓	✓		✓	✓			✓	
(BAO <i>et al.</i> , 2025)	✓	✓				✓			✓	✓			✓			✓	
(FAN; CAI, 2024)	✓	✓			✓	✓					✓	✓				✓	
(FARIMANI <i>et al.</i> , 2024)	✓	✓	✓				✓					✓				✓	
(SHI; CHEN; ZHU, 2023)	✓	✓									✓					✓	

Fonte: Elaborado pelo autor

Segundo (MIRIYALA; CHIRRA, 2026), apesar dos notáveis avanços proporcionados pela integração do DRL no descarregamento de tarefas, a literatura recente aponta limitações que dificultam que os modelos teóricos sejam utilizados no mundo real. Uma análise rigorosa conduzida por (MIRIYALA; CHIRRA, 2026) revela que a maioria das abordagens atuais sofre de simplificações excessivas na modelagem do ambiente. Frequentemente, os trabalhos utilizam representações de estado baseadas em vetores planos (*flat vectors*), o que ignora as complexas relações estruturais e topológicas entre os veículos e a infraestrutura. Essa lacuna evidencia a necessidade de incorporar arquiteturas diferentes, como as GNN para extrair representações topológicas antes da tomada de decisão. Além disso, os ambientes simulados costumam assumir canais de comunicação idealizados e conectividade persistente, negligenciando falhas físicas comuns nas redes veiculares, como quedas intermitentes de conexão (*dropouts*) (MIRIYALA; CHIRRA, 2026).

Outra lacuna está relacionada a carga computacional imposta pelos próprios modelos de aprendizado de máquina. Grande parte das pesquisas assume que os servidores de borda possuem recursos substanciais de *hardware*, deixando um vácuo no estudo onde dispositivos possuem severas restrições energéticas, de memória e de processamento (teto operacional de *FLOPS*). Para contornar esse obstáculo e viabilizar a implantação, (MIRIYALA; CHIRRA, 2026) destacam que pesquisas futuras devem focar no desenvolvimento de arquiteturas e representações mais leves, aplicando técnicas de compressão de modelos.

No que tange à adaptabilidade dos algoritmos, observa-se que as políticas DRL são tradicionalmente treinadas e permanecem imutáveis durante a implantação em produção. Contudo, em cenários veiculares não estacionários, onde o ambiente sofre alterações constantes devido a mudanças de distribuição das cargas (*distribution shifts*), tais políticas estáticas divergem e rapidamente se tornam obsoletas e ineficientes (MIRIYALA; CHIRRA, 2026). Identifica-se que há espaço na exploração de mecanismos de adaptação *online* de modo a evoluir a política computada pelo modelo.

Por fim, a transição destas propostas para sistemas reais esbarra na falta de avaliação comparativa confiável (*benchmarking*). A disparidade entre os estudos dificulta extrapolações e comparações parciais justas sobre as potenciais eficiências obtidas. (MIRIYALA; CHIRRA, 2026) criticam o fato de que a literatura tende a se atentar demasiadamente com as constatações de performance baseadas no treinamento ideal dos agentes.

3.3 Considerações Finais

Conforme discutido ao longo deste capítulo, as abordagens baseadas em DRL têm se solidificado como a direção mais promissora para o gerenciamento de recursos em arquiteturas de borda veicular. A integração com técnicas emergentes demonstra que há flexibilidade suficiente para abarcar os altos desafios impostos pelos cenários veiculares práticos. No entanto, as lacunas destacadas — como a predominância de simulações com observabilidade perfeita e cenários determinísticos — salientam a necessidade crítica de propostas que incorporem incerteza, variação de rajadas e carga de trabalho. Este alinhamento direciona o desenvolvimento da solução apresentada nos capítulos seguintes, projetada com vistas a preencher estas falhas e entregar maior robustez.

4 PROPOSTA DE SOLUÇÃO

Para lidar com o problema de descarregamento de tarefas em ambiente veicular, este capítulo apresenta os fundamentos desta proposta. De forma geral, nosso trabalho estende a solução desenvolvida em (VIEIRA; SOUZA; JÚNIOR, 2025), adicionando a ela o DRL. Em (VIEIRA; SOUZA; JÚNIOR, 2025), o *Fuzzy-TOPSIS* era o principal mecanismo decisor de descarregamento; ou seja, por meio de regras estáticas, o *Fuzzy Offloading* (FOFF) era encarregado de decidir quando e como o descarregamento era realizado, seguindo critérios de latência e consumo energético. Diferentemente disso, nossa proposta utiliza o algoritmo SAC para lidar com a complexidade inerente de ambientes altamente dinâmicos e decidir apenas se o processamento será local ou remoto. O sistema atua em sinergia — tanto na fase de treinamento quanto na de implantação — com o *Fuzzy-TOPSIS*, que é responsável por escolher o destino da tarefa. Além disso, enquanto em (VIEIRA; SOUZA; JÚNIOR, 2025) o FOFF apenas ranqueava os candidatos para seleção, em nosso caso ele também comprime a representação do estado em um vetor fixo de seis dimensões, adaptando-se a diferentes cenários quanto ao número de nós e cargas de trabalho. Por último, a proposta prioriza o atendimento dos requisitos de prazo em detrimento do consumo energético para buscar garantir a qualidade de serviço em ambientes críticos, como no caso de veículos autônomos.

No desenvolvimento da proposta, o trabalho de (MIRIYALA; CHIRRA, 2026) fundamentou a identificação de requisitos para uma solução de DRL capaz de mitigar lacunas críticas em ecossistemas inteligentes. A arquitetura proposta utiliza o *Fuzzy-TOPSIS* para comprimir N nós candidatos em um vetor de estado de seis dimensões invariante à escala, resolvendo a limitação de representações estáticas frente a topologias variáveis. Para lidar com a não estacionariedade e a alta mobilidade, o algoritmo SAC provê adaptação contínua por meio da regularização por entropia máxima, que mantém a exploração ativa e previne o colapso prematuro da política (HAARNOJA *et al.*, 2018). Adicionalmente, o uso do *replay buffer* permite a reutilização de experiências passadas e a quebra de correlações temporais, mecanismo essencial para estabilizar a otimização e evitar o esquecimento catastrófico na rede neural (MNIH *et al.*, 2015). Além disso, a normalização do estado via lógica nebulosa reduz a sensibilidade a ruídos e perturbações adversariais. Por fim, a viabilidade em dispositivos de borda extrema é garantida por uma rede MLP leve.

Nas seções seguintes, a proposta será detalhada. Inicialmente, é descrito o ambiente de rede veicular considerado e os cenários de treinamento e teste. Em seguida, formula-se o

problema e apresenta-se a arquitetura que integra o método multicritério ao aprendizado profundo. Cabe ressaltar que a solução definitiva fará uso do *Fuzzy-Technique for Order of Preference by Similarity to Ideal Solution* (Fuzzy-TOPSIS); contudo, como um passo intermediário, as formulações e os experimentos analisados neste capítulo empregam o TOPSIS tradicional para validar a eficácia da abordagem. Por fim, os resultados experimentais demonstram a viabilidade da solução, incluindo um estudo detalhado de ablação que justifica o *design* dos componentes.

4.1 Ambiente

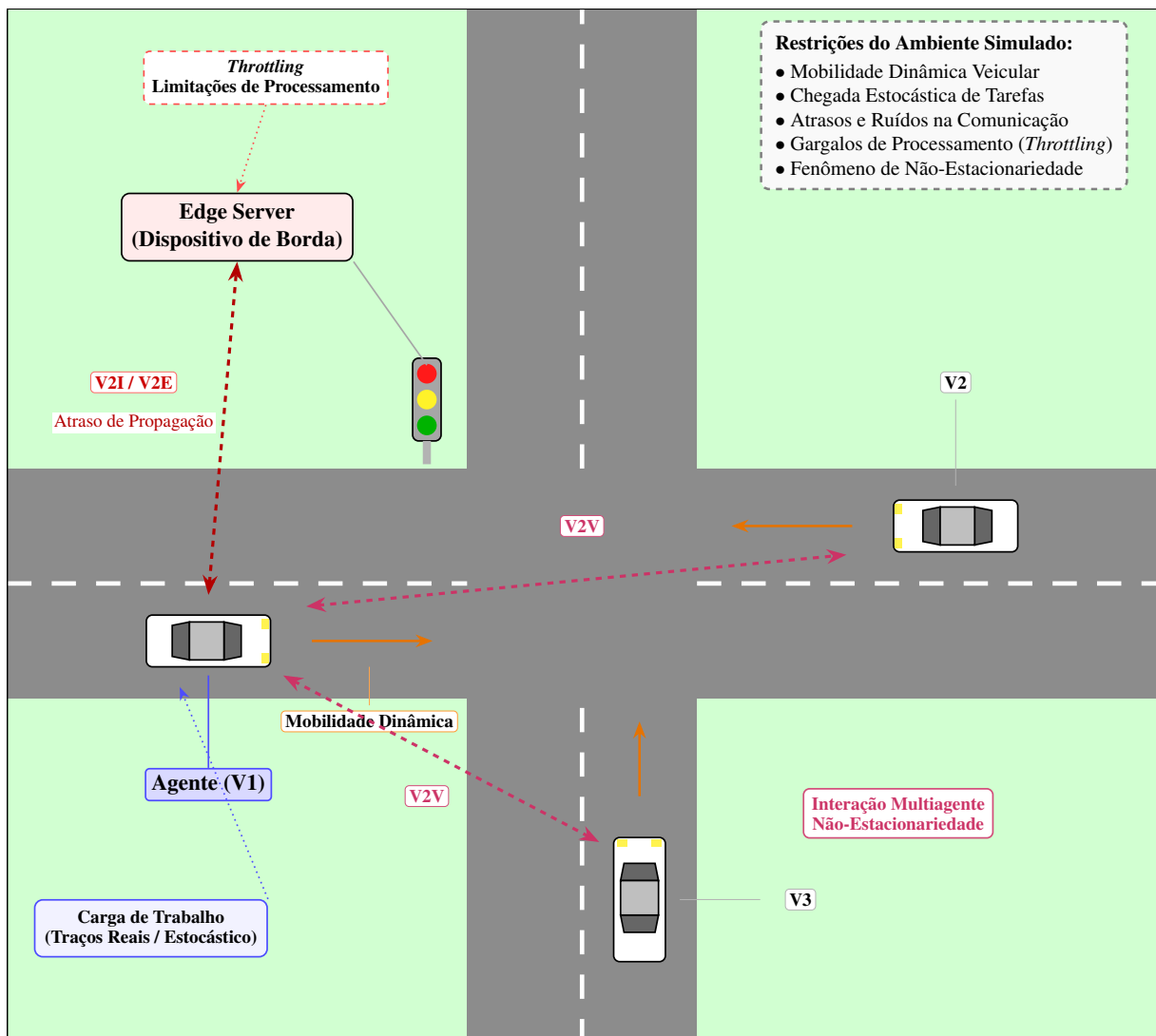


Figura 16 – Representação do ambiente de simulação: um cruzamento veicular contemplando comunicação V2I, V2V e os principais componentes do cenário estocástico.

O ambiente de treinamento e avaliação, nesta fase, é implementado como um Simulação de Eventos Discretos (*Discrete Event Simulation*) (DES) desenvolvido em C++20

com co-rotinas, utilizando a biblioteca `simcpp20` (SCHUETZ, 2025). Cada tarefa gerada pelos veículos é modelada como um processo concorrente que: (i) aguarda o instante de chegada t_j ; (ii) gera o vetor de estado percebido pelos agentes; (iii) executa a política de escalonamento; (iv) atualiza as filas dos servidores; e (v) registra as métricas reais de Tempo de Conclusão de Tarefa (*Job Completion Time*) (JCT) e consumo energético.

O servidor VEC é modelado utilizando um processador com Escalonamento Dinâmico de Tensão e Frequência (*Dynamic Voltage and Frequency Scaling*) (DVFS) e *clock rate* de $F_{\text{VEC}} = 1,2\text{GHz}$, com fila compartilhada de capacidade $Q_{\text{max}} = 100$ tarefas. Os veículos possuem frequências heterogêneas, amostradas no intervalo $[0,3; 1,5]\text{GHz}$. A comunicação V2I utiliza largura de banda $B = 20\text{MHz}$, potência de transmissão $P_{\text{tx}} = 0,2\text{W}$ (23 dBm) e modelo de atenuação de canal do 3GPP TR 38.901 (UMi Street Canyon), consistindo nestes os valores-padrão recomendados pelo 3GPP para avaliações de cenários veiculares urbanos (3GPP, 2020). Para redes V2V, adotam-se as diretrizes do documento técnico TR 37.885 com parâmetros análogos de canal, além de altura de antena de 1,5 m e fator de ruído de 9 dB (3GPP, 2019).

4.2 Cenários de Teste e Treinamento

Os cenários de treinamento foram gerados utilizando o SUMO (LOPEZ *et al.*, 2018) para a mobilidade veicular e para carga de trabalho utilizou-se um *trace* real de demandas computacionais do Alibaba (CHENG; CHAI; ANWAR, 2018), realizando as conversões de escala de tempo para adequação ao cenário veicular (detalhadas no Apêndice A). Já para o cenário de testes, foi utilizada a mesma mobilidade e foram criados 658 cenários a partir do *trace* original com carga de trabalho distribuídos conforme a Tabela 5.

Tabela 5 – Distribuição dos 658 cenários de treinamento/teste por taxa de chegada.

Taxa (tps)	Qtd. cenários	Tarefas/cenário
10	59	100
15	90	150
20	120	200
25	150	250
30	180	300
45	59	450
Total	658	—

Cada cenário define, para cada tarefa, suas características, o instante de chegada e o

veículo de origem. O ambiente compartilha um conjunto de parâmetros físicos fixos entre todos os cenários, calibrados para refletir as restrições operacionais de redes veiculares reais, como: frequências de processamento heterogêneas, alcances de comunicação limitados, potência de transmissão controlada e capacidade finita de filas. A Tabela 6 apresenta esses parâmetros.

Tabela 6 – Parâmetros físicos do ambiente simulado.

Parâmetro	Descrição	Valor
F_{VEC}	Frequência de clock da VEC	1,2 GHz
F_{veh}	Freq. dos veículos (heterogênea)	[0,3; 1,5] GHz
$B_{V2I/V2V}$	Largura de banda dos canais	20 MHz
P_{tx}	Potência de transmissão V2I e V2V	0,2 W
R_{max}	Raio de cobertura VEC	300 m
V_{V2V}	Alcance V2V	300 m
Q_{max}	Capacidade máxima das filas	100 tarefas

Protocolo de Treinamento e Hiperparâmetros

O treinamento de cada política DRL é conduzido interagindo com o simulador, sem nenhum acesso aos cenários de teste. Um único cenário de 100 tarefas a 10 tps (tarefas por segundo) é utilizado durante a fase de treinamento. A generalização para outras cargas de trabalho é avaliada nas 658 instâncias de teste. Cada episódio corresponde a uma simulação completa do cenário (100 transições). O treinamento dura 400 episódios (40 000 transições no total). A Tabela 7 lista os hiperparâmetros do SAC utilizados em todos os experimentos.

Tabela 7 – Hiperparâmetros do SAC compartilhados por todas as políticas DRL.

Hiperparâmetro	Descrição	Valor
η (lr)	Taxa de aprendizado (Adam)	3×10^{-4}
γ	Fator de desconto	0,99
τ	Taxa de atualização <i>soft</i>	0,005
α_{ent}	Temperatura de entropia	0,10
$ \mathcal{B} $	Capacidade do <i>replay buffer</i>	50 000
B	Tamanho do mini-lote	1 024
<i>warmup</i>	Transições antes do 1º update	1 024
Neurônios por camada	Arquitetura MLP (2 camadas ocultas)	64

A configuração dos hiperparâmetros adotada baseia-se em padrões empíricos e teóricos consolidados na literatura de Aprendizado por Reforço Profundo para garantir a estabilidade da otimização (SUTTON; BARTO, 2018). A taxa de aprendizado definida em 3×10^{-4} é o

valor amplamente recomendado para o otimizador Adam, promovendo uma convergência segura sem oscilações destrutivas nos gradientes (KINGMA; BA, 2014). O fator de desconto $\gamma = 0,99$ reflete uma modelagem com visão de longo prazo, o que é essencial para que o agente pondere os impactos futuros de suas decisões de descarregamento sobre o congestionamento das filas e o cumprimento dos prazos. Os parâmetros estruturais do SAC, como a taxa de atualização *soft* $\tau = 0,005$ e a temperatura de entropia $\alpha_{\text{ent}} = 0,10$, seguem as diretrizes originais do algoritmo para estabilizar a convergência das redes alvo e manter a exploração estocástica ativa, prevenindo colapsos prematuros da política (HAARNOJA *et al.*, 2018). Adicionalmente, o emprego de um *replay buffer* com capacidade para 50.000 transições, processado em mini-lotes de 1.024 amostras para reduzir a variância durante o treinamento (MNIH *et al.*, 2015). Por fim, a arquitetura de rede neural, composta por apenas duas camadas ocultas de 64 neurônios, garante um custo computacional baixo, um requisito indispensável frente às restrições de *hardware* e latência da computação de borda veicular (MIRIYALA; CHIRRA, 2026).

4.3 Visão Geral da Arquitetura e Formulação do Problema

O problema de descarregamento de tarefas nesta proposta é modelado como um MDP, no qual o objetivo principal consiste em maximizar a taxa de sucesso dentro do prazo estipulado (*deadline*), ponderada pelo custo normalizado de latência e energia. Dessa forma, busca-se atender aos rigorosos requisitos de QoS inerentes às redes veiculares, estabelecendo a premissa fundamental de que a violação de prazos em aplicações críticas é estritamente mais severa do que o incremento no consumo energético operacional. Formalmente, o objetivo da política de decisão é formulado da seguinte maneira:

$$\max_{\pi} J(\pi) = \mathbb{E}_{\pi} \left[w_s \cdot \underbrace{\mathbf{1}(\widehat{\text{JCT}}_t \leq D_t)}_{\text{taxa de sucesso}} - w_c \cdot \underbrace{\frac{1}{2} \left(\frac{\widehat{\text{JCT}}_t}{\text{JCT}_t^{\max}} + \frac{E_t}{E_t^{\max}} \right)}_{\text{custo normalizado} \in [0,1]} \right] \quad (4.1)$$

onde $w_s = 0,60 > w_c = 0,40$ refletem a maior importância dada à taxa de sucesso. O custo normalizado é a média aritmética de dois termos, ambos obtidos por normalização *min-max* com mínimo zero e máximo calculado dinamicamente no instante t . As Equações 4.2 e 4.3 denotam o pior caso de latência e de energia, respectivamente, entre os modos de descarregamento viáveis para a tarefa t .

$$\text{JCT}_t^{\max} = \max(\widehat{\text{JCT}}_t^{\ell}, \widehat{\text{JCT}}_t^{\text{edge}} \cdot \mathbf{1}_{\text{viável}}, \widehat{\text{JCT}}_t^{\text{v2v}} \cdot \mathbf{1}_{\text{viável}}) \in [0, 1] \quad (4.2)$$

$$E_t^{\max} = \max(E_t^{\ell}, E_t^{\text{edge}} \cdot \mathbf{1}_{\text{viável}}, E_t^{\text{v2v}} \cdot \mathbf{1}_{\text{viável}}) \in [0, 1] \quad (4.3)$$

Para resolver esse problema, a arquitetura proposta inicialmente combina TOPSIS e SAC em um *pipeline* de decisão unificado. A cada nova tarefa, o módulo TOPSIS avalia os nós de computação disponíveis segundo os critérios de latência, energia e distância, e identifica o melhor candidato (rank_1). O resultado é condensado em um vetor de estado compacto de 6 dimensões $\mathbf{S}_t \in \mathbb{R}^6$, invariante ao número de candidatos presentes. Uma MLP treinada via SAC recebe esse vetor e emite a decisão binária, $a_t \in \{0, 1\}$, executar localmente ou descarregar para o rank_1 .

Módulo TOPSIS e a Engenharia de Estado Compacto

Um dos desafios no desenvolvimento de soluções em cenários VEC é a variabilidade do número de candidatos ao descarregamento ao longo do tempo. Abordagens baseadas em Grafos (GNN) resolvem isso ao custo de alta complexidade computacional (MIRIYALA; CHIRRA, 2026). Neste trabalho, para lidar com esse problema de forma menos custosa, propõe-se extrair as características invariantes à escala através do TOPSIS, reduzindo os candidatos a um único vetor de estado de dimensão fixa. Isto é feito, avaliando os nós candidatos por meio de três critérios, conforme a Tabela 8.

Tabela 8 – Critérios e pesos do módulo TOPSIS.

Critério	Descrição	Tipo	Peso
C_1	Tempo de conclusão estimado ($\widehat{\text{JCT}}$ em milisegundos)	Custo	$w_1 = 0,40$
C_2	Energia total estimada (processamento + transmissão em Joules)	Custo	$w_2 = 0,30$
C_3	Distância estimada entre o veículo requisitor e o nó de computação (metros)	Custo	$w_3 = 0,30$

O procedimento segue os passos clássicos do TOPSIS: (i) normalização vetorial da matriz de decisão por norma L_2 de cada coluna; (ii) ponderação por w_j ; (iii) identificação das soluções ideal positiva A^+ (mínimo de cada critério) e ideal negativa A^- (máximo), ambas calculadas por minimização; (iv) cálculo das distâncias de cada alternativa a A^+ e A^- ; (v) escore

de proximidade. Independentemente de existirem 2 ou 50 candidatos, o estado entregue à rede neural é sempre um vetor fixo de 6 dimensões construído a partir do resultado do TOPSIS (Equação 4.4)

$$\mathbf{S}_t = \left[\Delta_{\text{obj}}, \frac{\widehat{\text{JCT}}_t^\ell}{D_t}, \frac{\widehat{\text{JCT}}_t^{\text{rank}_1}}{D_t}, \frac{|Q_t^{\text{rank}_1}|}{Q_{\text{max}}}, \frac{|Q_t^\ell|}{Q_{\text{max}}}, \frac{C_t}{C_{\text{max}}} \right] \in \mathbb{R}^6 \quad (4.4)$$

Cada dimensão carrega um valor semântico relativo para a decisão, de forma que $\Delta_{\text{obj}} = (\widehat{\text{JCT}}_t^\ell - \widehat{\text{JCT}}_t^{\text{rank}_1})/D_t$ denota a diferença de latência normalizada entre a execução local e o candidato rank_1 . Um valor positivo ($\Delta_{\text{obj}} > 0$) indica que o *offload* é mais rápido, enquanto um valor negativo ($\Delta_{\text{obj}} < 0$) indica que a execução local é mais rápida. A dimensão $\widehat{\text{JCT}}_t^\ell/D_t$ calcula a razão entre o tempo de conclusão estimado localmente e o prazo, de modo que valores $> 1,0$ sinalizam violação iminente e $\widehat{\text{JCT}}_t^{\text{rank}_1}/D_t$ é a razão análoga para o melhor candidato. Já $|Q_t^{\text{rank}_1}|/Q_{\text{max}}$ representa a ocupação normalizada da fila do nó classificado em primeiro lugar pelo TOPSIS e $|Q_t^\ell|/Q_{\text{max}}$ informa a ocupação da fila local. Por fim, C_t/C_{max} normaliza o número de ciclos de CPU da tarefa. Em conjunto, as seis dimensões fornecem ao agente os sinais necessários para avaliar, a cada instante, a vantagem relativa do *offload* frente à execução local.

Seleção da Ação de Descarregamento

O agente SAC opera sobre um espaço de ação discreto binário $a_t \in \{0, 1\}$. O mapeamento entre ação e modo de execução é mediado pelo resultado do TOPSIS. Quando $a_t = 1$, a tarefa é encaminhada ao candidato melhor ranqueado, que pode ser o servidor VEC ou um veículo vizinho. Caso nenhum candidato de *offload* seja viável ($N = 0$), $a_t = 1$ é tratado como LOCAL por padrão. Essa separação de responsabilidades, TOPSIS decide para onde descarregar, SAC decide quando descarregar, reduz o espaço de busca do agente de $N + 1$ para 2 ações, independente de N .

Como a representação do estado desta proposta comprime qualquer conjunto de N candidatos em um único vetor $\mathbb{R}^6$ constante, a política treinada interpola naturalmente para cenários com topologias distintas sem retreinamento. A Figura 17 detalha como o vetor $\mathbf{S}_t$ alimenta a rede *Actor* do SAC e como a camada de saída produz a decisão binária.

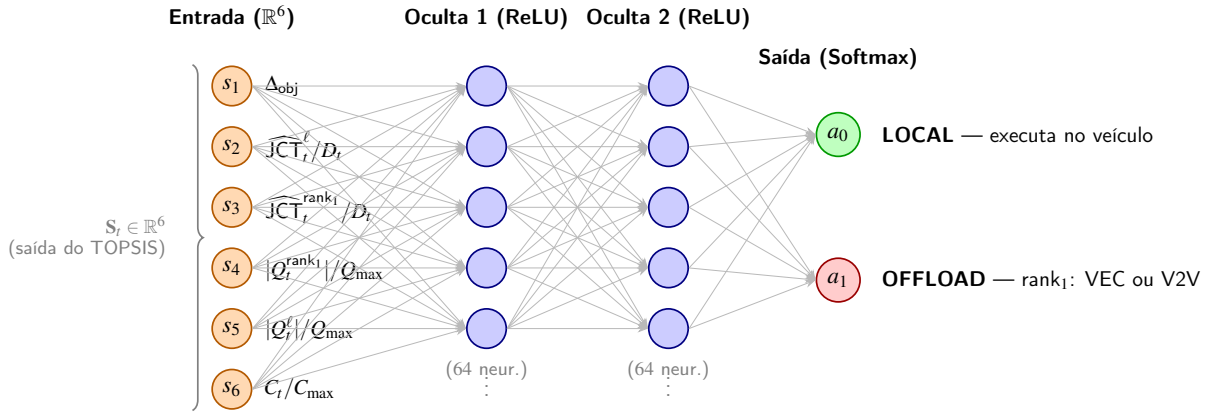


Figura 17 – Arquitetura da rede Actor (MLP 6 → 64 → 64 → 2): o vetor de estado compacto S_t alimenta duas camadas ocultas com ativação ReLU; a camada Softmax final emite as probabilidades das ações a_0 (LOCAL) e a_1 (OFFLOAD $rank_1$).

Modelagem da Recompensa

A função de recompensa é formulada para alinhar o sinal de aprendizado diretamente com o objetivo da Equação 4.1, isto é, priorizar a taxa de sucesso e, secundariamente, minimizar o custo operacional. Para isso, a recompensa por tarefa é estruturada conforme a Equação 4.5.

$$r_t = \begin{cases} 1,0 - \frac{1}{4} \left(\frac{\widehat{JCT}_t}{D_t} + \frac{E_t}{E_t^{\max}} \right), & \text{se } \widehat{JCT}_t \leq D_t \quad (\in [0,5, 1,0]) \\ -1,0 - \min \left(1,0, \frac{\widehat{JCT}_t - D_t}{D_t} \right), & \text{se } \widehat{JCT}_t > D_t \quad (\in [-2,0, -1,0]) \end{cases} \quad (4.5)$$

onde $E_t^{\max}$ (Equação 4.3) é a energia máxima entre os modos viáveis para a tarefa i , calculada por tarefa e por instante, não é uma constante global, pois depende de quais modos estão viáveis no momento da decisão.

A formulação da recompensa introduz intencionalmente uma lacuna mínima de 1,5 unidades entre o pior cenário de sucesso e o melhor cenário de falha. Essa descontinuidade atua como uma estratégia de *reward shaping* para estabelecer que cumprir o prazo é sempre preferível a qualquer falha. Sem essa separação, o agente poderia aprender a tolerar violações leves de prazo em troca de maior eficiência energética (NG; HARADA; RUSSELL, 1999). Além disso, essa transição abrupta na fronteira do prazo justifica a necessidade de adotar algoritmos de aprendizado estabilizados por entropia, como o SAC (HAARNOJA *et al.*, 2018), visto que abordagens puramente baseadas em valor enfrentariam problemas severos de superestimação nessa região (MIRIYALA; CHIRRA, 2026).

4.4 Complexidade Computacional

Ao evitar o uso de GNNs e optar pelo condicionamento via TOPSIS, a complexidade assintótica da tomada de decisão a cada instante é regida pelas operações de avaliação e ordenação. A avaliação dos N candidatos sob os M critérios possui complexidade $O(N \times M)$, enquanto a ordenação final das alternativas opera em $O(N \log N)$. Como o número de critérios M é constante, a complexidade dominante do pré-processamento escalar cresce de forma log-linear com a densidade veicular. Em seguida, a inferência da MLP (*Actor network*), com duas camadas ocultas de 64 neurônios e entrada compacta de 6 dimensões, opera com complexidade $O(1)$ em relação ao número de candidatos. Essa arquitetura de rede requer um custo paramétrico extremamente baixo. Dessa forma, a abordagem torna-se altamente escalável e compatível com os requisitos de latência e com as restrições de *hardware* menos robustos.

4.5 Estudo de Ablação

Para isolar a contribuição de cada componente do SAC-TOPSIS, realizou-se um estudo de ablação controlado com quatro variantes treinadas e avaliadas nas mesmas 658 instâncias de cenário. Cada variante remove ou substitui um único componente de design em relação à proposta completa, mantendo os demais fixos.

SAC-Base — Referência

O SAC-Base é o ponto de partida do estudo e serve de referência para medir o ganho de cada componente. Utiliza um estado bruto de 11 dimensões que concatena diretamente as métricas do ambiente sem nenhum pré-processamento multicritério:

$$\mathbf{S}^{\text{base}} = [C_i, D_i, d_{\text{VEC}}, f_{\text{dvfs}}, Q_{\text{VEC}}, T_{\text{cpu}}, f_{\text{local}}, Q_{\text{local}}, f_{\text{V2V}}, Q_{\text{V2V}}, d_{\text{V2V}}] \quad (4.6)$$

onde d denota distância, f frequência de processamento, Q ocupação de fila e T_{cpu} temperatura do processador. Embora possam existir múltiplos vizinhos V2V no alcance, o estado expõe apenas um, o veículo com maior frequência de *clock* dentro do raio V2V, selecionado por uma varredura gulosa no pré-processamento do cenário. Quando nenhum vizinho está disponível, as três últimas componentes recebem o valor sentinela -1 para distinguir ausência de V2V de V2V com métrica zero.

O espaço de ação é ternário $a_t \in \{0 = \text{LOCAL}, 1 = \text{VEC}, 2 = \text{V2V}\}$, ou seja, o agente deve aprender simultaneamente a decisão de descarregar e a escolha do destino. A recompensa é

calculada pelo Método de Soma Ponderada (WSM) com pesos iguais sobre três critérios de custo — JCT estimado ($j = 1$), energia total ($j = 2$) e ocupação de fila do executor escolhido ($j = 3$):

$$r_{t+1}^{\text{wsm}} = 2 \max(0, \min(1, 1 - \bar{x}_a)) - 1, \quad \bar{x}_a = \frac{1}{3} \sum_{j=1}^3 \tilde{x}_{aj}, \quad \tilde{x}_{aj} = \frac{x_{aj}}{\max_{a'} x_{a'j}} \quad (4.7)$$

onde $\tilde{x}_{aj} = x_{aj} / \max_{a'} x_{a'j} \in [0, 1]$ normaliza o critério j da alternativa a pelo valor máximo desse critério entre todas as alternativas $a' \in \{\text{LOCAL}, \text{VEC}, \text{V2V}\}$; $\bar{x}_a \in [0, 1]$ é a média dos três custos normalizados da alternativa escolhida; e a recompensa é obtida pela transformação $2(1 - \bar{x}_a) - 1$, que inverte o custo e o remapeia linearmente para $[-1, 1]$. Esse sinal de recompensa não distingue violações de *deadline* de execuções lentas porém viáveis. Uma ação que ultrapassa o prazo com custo moderado pode receber recompensa positiva, desorientando o aprendizado em ambientes com prazos rígidos.

SAC-A1 — Redução do Espaço de Ação sem TOPSIS

A variante A1 mantém inalterados o estado 11D (Equação 4.6) e a recompensa WSM (Equação 4.7), alterando *apenas* o espaço de ação para binário $a_t \in \{0 = \text{LOCAL}, 1 = \text{OFFLOAD}\}$. Quando $a_t = 1$, o destino de *offload* é determinado por uma regra de prioridade estática — VEC primeiro (se viável), V2V caso contrário — sem qualquer análise multicritério:

$$\text{modo}_t^{\text{A1}} = \begin{cases} \text{LOCAL} & \text{se } a_t = 0 \\ \text{VEC} & \text{se } a_t = 1 \text{ e VEC viável} \\ \text{V2V} & \text{se } a_t = 1, \text{ VEC inviável e V2V viável} \\ \text{LOCAL} & \text{se } a_t = 1 \text{ e nenhum offload viável (fallback)} \end{cases} \quad (4.8)$$

O objetivo é isolar o efeito de reduzir o espaço de busca de 3 para 2 ações, sem nenhum critério inteligente de escolha de destino. A regra $\text{VEC} \succ \text{V2V}$ é estática e ignora as condições dinâmicas de fila e distância no momento da decisão.

SAC-S1 — Estado Compacto 6D Manual sem TOPSIS

A variante S1 mantém o espaço de ação ternário e a recompensa Método de Soma Ponderada (*Weighted Sum Method*) (WSM) do *baseline*, mas substitui o estado 11D por um vetor compacto de 6 dimensões considerando as características de maior relevância:

$$\mathbf{S}^{S1} = \left[\frac{C_i}{C_{\max}}, \min\left(1, \frac{\widehat{\text{JCT}}_{\text{local}}}{D_i}\right), \frac{d_{\text{VEC}}}{d_{\max}}, \min\left(1, \frac{|Q_{\text{VEC}}|}{Q_{\max}}\right), \min\left(1, \frac{|Q_{\text{local}}|}{Q_{\max}}\right), \frac{f_{\text{local}}}{f_{\text{VEC}}}\right] \quad (4.9)$$

O primeiro componente calcular o valor relativo dos ciclos da tarefa. O segunda componente, urgência relativa $\widehat{\text{JCT}}_{\text{local}}/D_i$, verifica se a execução local da tarefa vai estourar o prazo. Nessa formulação, não é incluída informações de V2V (frequência, fila, distância do vizinho), forçando o agente a decidir entre o servidor VEC e V2V apenas observando o comportamento do ambiente veicular.

SAC-R1 — Recompensa Deadline-Aware sem Mudança de Estado ou Ação

A variante R1 mantém o estado 11D (Equação 4.6) e o espaço de ação ternário idênticos ao *baseline*, alterando apenas a função de recompensa para a formulação sensível ao prazo conforme a Equação 4.5.

SAC-T1 — TOPSIS como Feature de Estado com Ação de 3 Classes

A variante T1 combina o estado compacto 6D calculado *via* TOPSIS (idêntico ao da proposta, Equação 4.4) com a recompensa *deadline-aware* (Equação 4.5), mas mantém o espaço de ação ternário $a_t \in \{0 = \text{LOCAL}, 1 = \text{VEC}, 2 = \text{V2V}\}$. O TOPSIS calcula internamente os escores de todos os candidatos e usa o resultado para construir o vetor de estado — incluindo a vantagem objetiva Δ_{obj} e o JCT normalizado do rank_1 — mas a decisão final de escolher VEC ou V2V permanece com o agente SAC. Essa variante permite quantificar separadamente quanto do ganho do SAC-TOPSIS provém do TOPSIS como compressor de estado semântico versus o TOPSIS como responsável pela decisão de destino do descarregamento.

Decomposição Causal

A decomposição causal consiste em medir o impacto isolado de cada mudança no desempenho. A Equação 4.10 sintetiza o caminho sequencial percorrido do SAC-Base ao SAC-TOPSIS, onde cada seta quantifica a variação absoluta em pontos percentuais (pp) da taxa de sucesso devida àquela transição.

Tabela 9 – Resultados das variantes do estudo de ablação avaliadas em 658 cenários de teste. A proposta completa é destacada em negrito.

Variante	Estado	Ação	Recompensa	Taxa de Sucesso(%)	SWQ	JCT (s)
SAC-Base	11D bruto	Ternária	WSM	4,52	0,0119	57,075
SAC-A1	11D bruto	Binária	WSM	39,11	0,1256	13,722
SAC-S1	6D	Ternária	WSM	28,69	0,1122	39,298
SAC-R1	11D bruto	Ternária	<i>Deadline-aware</i>	38,26	0,1223	13,353
SAC-T1	6D TOPSIS	Ternária	<i>Deadline-aware</i>	32,96	0,1270	11,487
SAC-TOPSIS	6D TOPSIS	Binária	<i>Deadline-aware</i>	45,21	0,1484	6,547

$$\underbrace{4,52\%}_{\text{SAC-Base}} \xrightarrow{+24,17\text{pp}} \underbrace{28,69\%}_{\text{SAC-S1}} \xrightarrow{+4,27\text{pp}} \underbrace{32,96\%}_{\text{SAC-T1}} \xrightarrow{+12,25\text{pp}} \underbrace{45,21\%}_{\text{SAC-TOPSIS}} \quad (4.10)$$

O caminho revela que as três modificações não contribuem de forma uniforme. A maior parcela do ganho provém da compressão de estado (primeiro salto, +24,17 pp), seguida pelo uso do TOPSIS como responsável pela decisão de destino (terceiro salto, +12,25 pp). A incorporação do TOPSIS como *feature* de estado combinada com a recompensa *deadline-aware* (segundo salto) contribui de forma mais modesta (+4,27 pp), pois o agente ainda precisava resolver por conta própria a ambiguidade VEC/V2V.

4.6 Análise dos Resultados

Três achados principais emergem do estudo. Primeiro, a variante SAC-A1 (39,2%) demonstra que binarizar a ação *sem* critério de seleção de destino já supera o *baseline* em +34,6 pp — a simples redução do espaço de busca tem efeito expressivo — mas não alcança a proposta (45,3%) porque a regra estática VEC > V2V ignora condições dinâmicas de fila e distância no momento da decisão. Segundo, a variante SAC-T1 (33,1%) mostra que usar o TOPSIS apenas como *feature* de estado contribui com +4,3 pp sobre S1, enquanto usar o TOPSIS como *prior* de decisão (SAC-TOPSIS) adiciona +12,2 pp sobre T1: a eficácia do TOPSIS é quase três vezes maior quando ele assume o controle do destino do que quando fornece apenas informação semântica ao agente. Terceiro, o JCT médio do SAC-TOPSIS (6,6 s) é 8,6× menor que o do SAC-Base (57,1 s), indicando que a combinação TOPSIS+estado compacto+reward *deadline-aware* produz não apenas mais tarefas concluídas no prazo, mas também latências substancialmente menores.

Em suma, a hierarquia de contribuição dos componentes é:

$$\Delta_{\text{estado}} > \Delta_{\text{TOPSIS-decisão}} > \Delta_{\text{reward+feature}} \gg \Delta_{\text{ação-binária-estática}}$$

confirmando que a *compressão de estado via TOPSIS* e o *TOPSIS como prior de destino* são os dois pilares do ganho do SAC-TOPSIS, e que a recompensa *deadline-aware*, embora poderosa isoladamente, tem contribuição marginal reduzida quando combinada com um estado já semanticamente rico.

4.6.1 Métrica de Efetividade: *Success-Weighted Quality* (SWQ)

A taxa de *deadline* ($\hat{\tau}$) empregada até aqui é uma métrica binária: conta quantas tarefas foram concluídas no prazo, mas ignora *quão bem* essas tarefas foram atendidas. Uma política que cumpre o prazo com folga de 1% é tratada da mesma forma que uma que o cumpre com folga de 40%. De maneira análoga, o JCT médio e a energia total acumulam contribuições de tarefas bem-sucedidas e de tarefas que violaram o prazo, tornando difícil separar eficiência de falha.

Para reunir ambas as dimensões em um único número interpretável, define-se o *Success-Weighted Quality* (SWQ):

$$\text{SWQ}(\pi) = \hat{\tau}(\pi) \times \overline{\left(\frac{D_j - \text{JCT}_j}{D_j} \right)}_{\mathcal{S}} \quad (4.11)$$

onde $\hat{\tau}(\pi) \in [0, 1]$ é a taxa de sucesso da política π e o segundo fator é a folga normalizada média calculada exclusivamente sobre o conjunto $\mathcal{S}$ de tarefas que cumpriram o prazo:

$$\overline{\text{folga}}_{\mathcal{S}} = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \frac{D_j - \text{JCT}_j}{D_j} \in (0, 1]$$

A folga normalizada vale 1 quando a tarefa é concluída instantaneamente e decresce para 0 quando o JCT se aproxima do prazo. Uma política que jamais erra o prazo e conclui todas as tarefas com folga máxima teria $\text{SWQ} = 1,0$; uma que sempre viola teria $\text{SWQ} = 0$. O produto das duas componentes penaliza simultaneamente políticas com baixa taxa de sucesso e políticas que cumprem o prazo por margem ínfima — características indesejadas em contextos veiculares com prazos rígidos.

A Figura 18 apresenta o SWQ médio das vinte políticas avaliadas sobre os 661 cenários de teste. O SAC-TOPSIS alcança o maior valor ($\text{SWQ} = 0,1484$), seguido pelas políticas *Always-Local*, PPO-Base e PPO-Compact (todas em 0,1418). Nota-se que o *greedy-unfair* — que possui conhecimento perfeito das filas em tempo de execução — obtém $\text{SWQ} = 0,1261$, inferior ao SAC-TOPSIS: embora o oráculo minimize o JCT por decisão, sua natureza míope

acumula pior qualidade agregada do que a política aprendida. Por outro lado, *Always-VEC* e *SAC-Base* apresentam os menores SWQs (0,003 e 0,012, respectivamente), reflexo de taxas de sucesso irrisórias (1,9% e 4,6%).

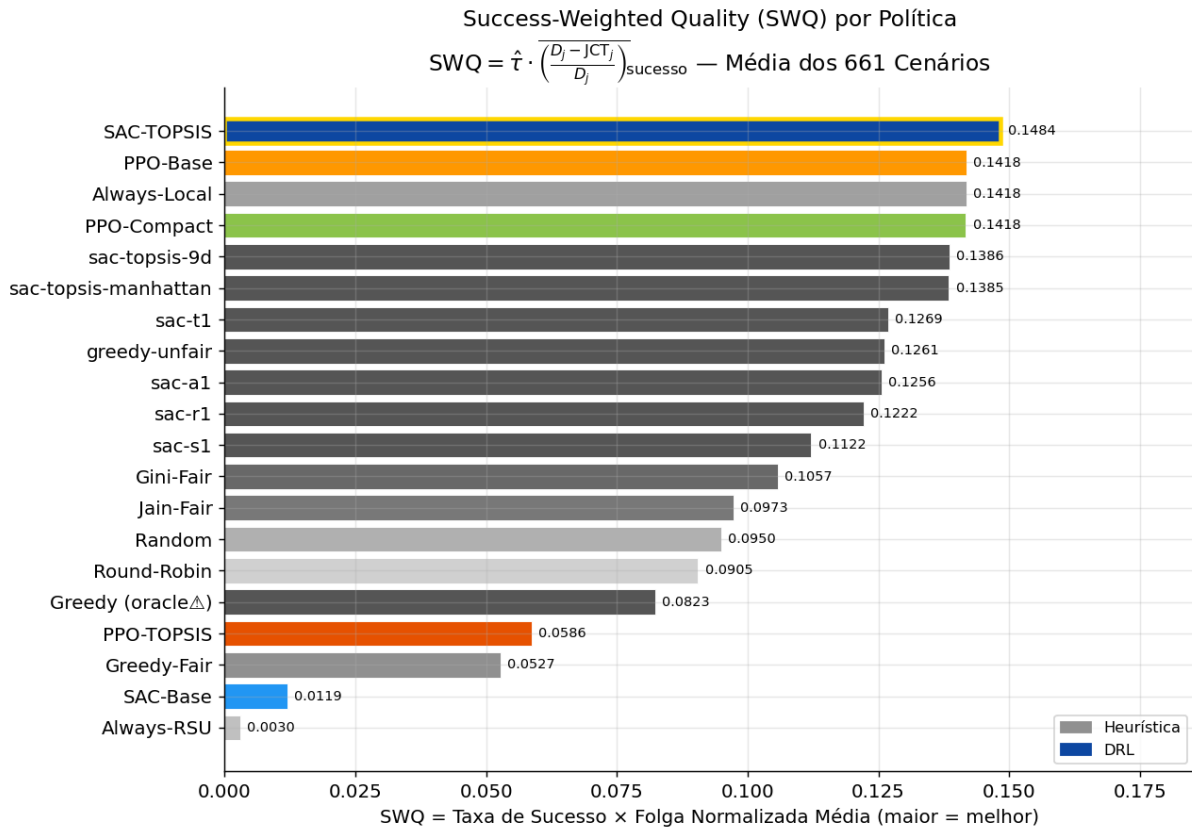


Figura 18 – SWQ médio por política sobre 661 cenários de teste ($\alpha = 0,35$, $\beta = 0,35$, $\gamma = 0,30$). Barras em cinza representam heurísticas; barras escuras representam políticas DRL. A borda dourada indica a melhor política. Maior é melhor.

4.7 Por que o TOPSIS agrega valor ao SAC mas não ao PPO?

Os resultados experimentais revelam uma assimetria marcante: o TOPSIS eleva a taxa de *deadline* do SAC de 4,6% (SAC-Base) para 45,3% (SAC-TOPSIS), um ganho de +40,7 pontos percentuais, enquanto o PPO com TOPSIS regride. Para isolar a causa desta assimetria, foram realizados dois experimentos controlados. Primeiro, o PPO-TOPSIS original com estado 12D e ação ternária — que permitia ao agente escolher entre $rank_1$, $rank_2$ e $rank_3$ — obteve 18,4%, abaixo do PPO-Base (42,4%). Segundo, o PPO-TOPSIS foi refatorado para utilizar *exatamente* o mesmo estado 6D, ação binária $\{0 = \text{LOCAL}, 1 = rank_1\}$ e função de recompensa do SAC-TOPSIS, diferindo apenas no algoritmo de otimização (PPO vs. SAC). O resultado deste experimento controlado foi igualmente 18,4%, com a política colapsando para 86% de *offload*

via $rank_1$ — o oposto do PPO-Compact (100% LOCAL), porém igualmente disfuncional. O PPO-TOPSIS-Compact, que usa o estado 6D mas foi treinado em cenários distintos, apenas iguala o PPO-Base. Em síntese: **o TOPSIS é uma alavanca decisiva para o SAC e um fator neutro ou prejudicial para o PPO, independentemente do design do estado.** Esta seção analisa as causas estruturais desta assimetria.

4.7.1 O problema do aprendizado redundante no PPO-TOPSIS

O PPO-TOPSIS foi implementado com um estado de 12 dimensões — três alternativas (LOCAL, VEC, V2V) ordenadas pelo score TOPSIS, cada uma descrita por quatro atributos normalizados (JCT estimado, energia, distância, *link lifetime*) — e um espaço de ação ternário $a_t \in \{0 = rank_1, 1 = rank_2, 2 = rank_3\}$. Esse design cria um problema de aprendizado redundante: o TOPSIS já determina deterministicamente que o candidato na posição 0 é o melhor segundo critérios multicritério rigorosos. O agente PPO, portanto, é treinado por reforço para redescobrir, por tentativa e erro, o que o TOPSIS já computou de forma ótima. A informação de valor genuíno que o agente deveria aprender — se há benefício em descarregar para o melhor candidato dado o estado da fila e a urgência do *deadline* — está enterrada sob o sinal dominante de que $rank_1$ é sempre superior a $rank_2$ e $rank_3$ em custo multicritério.

O SAC-TOPSIS resolve este problema por projeto: delega a pergunta “*para onde descarregar?*” inteiramente ao TOPSIS e reserva ao agente apenas a pergunta “*descarregar ou não?*” — uma decisão genuinamente nova, que o TOPSIS não responde, pois envolve a comparação entre o melhor candidato de *offload* e a execução local ponderada pela urgência temporal, pelo congestionamento da fila e pela vantagem objetiva Δ_{obj} codificada no estado 6D.

4.7.2 Colapso de política induzido pelo sinal TOPSIS

O segundo mecanismo de falha do PPO-TOPSIS é o colapso de política. Com o estado 12D, a posição $rank_1$ exibe consistentemente JCT menor, energia menor e distância menor que $rank_2$ e $rank_3$. Isso gera um gradiente de política fortemente unidirecional desde as primeiras iterações: $P(a=0) \rightarrow 1$. O PPO utiliza coeficiente de entropia $\alpha_{ent} = 0,01$, dez vezes menor que o do SAC ($\alpha_{ent} = 0,10$ com ajuste automático via gradiente dual). Com entropia tão fraca, uma vez que a distribuição de política colapsa para $P(rank_1) \approx 1$, as trajetórias coletadas nas iterações seguintes são quase exclusivamente de escolhas $rank_1$, reforçando o colapso. O mecanismo de *clipping* do PPO ($\epsilon = 0,2$) não reverte esse estado: ele apenas impede mudanças

grandes por passo, mas não impede a deriva acumulada durante os 400 episódios de treinamento.

O SAC, ao contrário, opera sobre um espaço de ação binário em que nenhuma das duas opções (LOCAL ou *offload* para $rank_1$) domina o outro de forma incondicional: há cenários em que a fila do VEC está saturada, ou o V2V está distante, ou o *deadline* é tão apertado que mesmo o melhor candidato de *offload* não chega a tempo. Nesses casos, a resposta correta é LOCAL, e o sinal de recompensa *deadline-aware* — com lacuna mínima de 1,5 entre o melhor valor de falha e o pior valor de sucesso — preserva um gradiente exploratório relevante. A alta temperatura entrópica do SAC garante que ambas as ações recebam probabilidade não nula ao longo do treinamento, evitando o colapso.

4.7.3 Assimetria de eficiência amostral com cenário único

O protocolo de treinamento emprega um único cenário (`cenario_v2x_10tps_w01`) durante os 400 episódios, com generalização *zero-shot* para os 661 cenários de teste. Essa escolha é compatível com o SAC, mas estruturalmente adversa ao PPO.

O SAC mantém um *replay buffer* de 50.000 transições e realiza atualizações com *minibatch* de 1024 amostras por passo. Com aproximadamente 100 tarefas por episódio, as 40.000 transições coletadas ao longo dos 400 episódios são amostradas em média ≈ 40 vezes cada durante o treinamento — uma reciclagem intensiva que extrai máxima informação de um conjunto de dados limitado. Além disso, a mistura de experiências de diferentes pontos do treinamento (política mais aleatória nos primeiros episódios, mais refinada nos últimos) cria uma diversidade implícita que previne o sobreajuste ao único cenário de treino.

O PPO, sendo *on-policy*, descarta cada lote de trajetórias após as $k = 8$ épocas de atualização. Com um único cenário, cada episódio gera essencialmente o mesmo padrão de chegada de tarefas, o mesmo volume de tarefas e as mesmas condições de canal. A política converge rapidamente para a estratégia que maximiza a recompensa nesse cenário específico — que, com a salência do sinal TOPSIS, é “*sempre escolha rank₁*” — e não tem como escapar desse ótimo local por falta de diversidade nos dados on-policy.

A evidência experimental confirma esta hipótese: o PPO-TOPSIS-Compact, que usa o estado 6D do SAC-TOPSIS (idêntico em conteúdo semântico) mas com algoritmo PPO, atinge 42,4% — o mesmo que o PPO-Base, que usa o estado bruto 11D. O estado TOPSIS mais informativo não agrega valor ao PPO porque o gargalo não é a qualidade do estado, mas a ineficiência amostral do algoritmo on-policy com cenário único.

4.7.4 Síntese: complementaridade estrutural SAC–TOPSIS

A superioridade do SAC-TOPSIS não decorre de nenhum componente isolado, mas da complementaridade estrutural entre os dois módulos em três dimensões:

1. **Divisão de responsabilidades:** O TOPSIS resolve deterministicamente a pergunta “*qual candidato de offload é melhor?*”; o SAC aprende a resposta à pergunta complementar “*vale a pena descarregar para esse candidato?*”. As duas perguntas são independentes e igualmente necessárias; nenhum dos dois módulos, isoladamente, as responde ambas.
2. **Off-policy + replay buffer:** O SAC recicla cada transição ≈ 40 vezes no treinamento com um único cenário, alcançando uma eficiência amostral inacessível ao PPO on-policy. Essa reciclagem é segura porque o SAC aprende uma função de valor $Q(s, a)$ que é off-policy por construção — ao contrário do PPO, cujo gradiente de política requer dados coletados pela política corrente.
3. **Regularização entrópica intensa:** O SAC mantém $\alpha_{\text{ent}} = 0,10$ com ajuste automático, dez vezes superior ao coeficiente fixo do PPO. Essa entropia elevada é precisamente o que impede o colapso de política frente ao sinal claro fornecido por Δ_{obj} no estado 6D — mantendo a exploração ativa mesmo quando o TOPSIS sugere fortemente que *offload* é melhor.

Em contraste, o PPO-TOPSIS sofre das três falhas simétricas: aprende uma pergunta que o TOPSIS já responde (redundância), colapsa para uma estratégia degenerada com entropia insuficiente para resistir ao sinal TOPSIS (colapso), e sobreajusta ao único cenário de treino sem o mecanismo de reciclagem do *replay buffer* (ineficiência amostral). O experimento controlado confirma inequivocamente este diagnóstico: ao trocar *apenas* o algoritmo — mantendo estado 6D, ação binária e recompensa idênticos — o SAC-TOPSIS atinge 45,3% enquanto o PPO-TOPSIS obtém 18,4%, um déficit de 26,9 pontos percentuais atribuível exclusivamente à natureza on-policy do PPO e à sua baixa temperatura entrópica. O TOPSIS, portanto, não é neutro quando mal integrado: *amplifica* as fraquezas do algoritmo hospedeiro.

4.8 Avaliação Comparativa de Políticas

As Tabelas 10 e 11 resumem os resultados de 25 políticas avaliadas em 658 cenários de teste comuns, abrangendo algoritmos de aprendizado por reforço (com e sem TOPSIS) e heurísticas de referência. As métricas reportadas são: taxa de *deadline* (%), SWQ ($= \hat{\tau} \cdot \overline{(D_j - \text{JCT}_j)} / D_{j_{\text{sucesso}}}$), JCT médio em segundos, energia média por tarefa em joules e o objetivo

$J(\pi) = 0,60 \cdot \hat{\tau} - 0,40 \cdot \frac{1}{2}(\text{JCT}/\text{JCT}^{\max} + E/E^{\text{ref}})$ (Equação 4.1), onde $\text{JCT}^{\max}$ e E^{ref} são os piores casos entre os modos viáveis por tarefa.

Tabela 10 – Ranking completo das 25 políticas avaliadas em 658 cenários comuns (ordenado por SWQ decrescente). * = usa TOPSIS.

#	Política	Cn	Taxa%	SWQ	JCT (s)	E (J)	J(π)	%Loc	%V2V	Categoria
1	sac-topsis *	658	45,21	0,1484	6,547	25,828	0,2454	74,1	25,9	RL+TOPSIS
2	ppo-topsis *	658	42,66	0,1426	5,895	21,333	0,2344	98,7	1,3	RL+TOPSIS
3	ppo-base	658	42,32	0,1418	6,045	24,414	0,2295	100,0	0,0	RL-Base
4	ppo-topsis-compact *	658	42,32	0,1418	6,045	24,414	0,2295	100,0	0,0	RL+TOPSIS
5	td3-base	658	42,32	0,1418	6,045	24,414	0,2295	100,0	0,0	RL-Base
6	a2c-topsis *	658	42,32	0,1418	6,045	24,414	0,2295	100,0	0,0	RL+TOPSIS
7	td3-topsis *	658	43,91	0,1388	3,703	22,995	0,2424	76,5	23,5	RL+TOPSIS
8	sac-topsis-9d *	658	42,82	0,1386	5,886	20,985	0,2348	94,7	5,3	RL+TOPSIS
9	sac-topsis-manhattan *	658	41,66	0,1385	11,831	23,656	0,2211	51,1	48,9	RL+TOPSIS
10	sac-t1 *	658	32,96	0,1270	11,487	26,126	0,1658	22,6	77,4	RL+TOPSIS
11	greedy-unfair	658	41,29	0,1262	5,613	7,396	0,2370	67,3	32,7	Heurística
12	sac-a1	658	39,11	0,1256	13,722	21,118	0,2064	82,9	17,1	RL-Base
13	sac-r1	658	38,26	0,1223	13,353	31,425	0,1914	58,8	41,2	RL-Base
14	sac-s1	658	28,69	0,1122	39,298	20,305	0,1210	38,2	61,8	RL-Base
15	gini	658	35,24	0,1057	3,456	39,395	0,1790	61,7	38,3	Heurística
16	dqn-topsis *	658	25,90	0,0997	21,722	23,979	0,1161	20,0	80,0	RL+TOPSIS
17	jain	658	32,54	0,0973	3,628	39,280	0,1627	58,6	41,4	Heurística
18	random	658	30,02	0,0950	17,853	27,330	0,1427	40,7	59,3	Heurística
19	roundrobin	658	28,50	0,0905	17,900	31,080	0,1315	34,9	65,1	Heurística
20	greedy	658	29,03	0,0823	29,265	6,168	0,1429	42,5	57,5	Heurística
21	a2c-base	658	21,74	0,0772	13,653	48,862	0,0785	20,3	79,7	RL-Base
22	dqn-base	658	22,10	0,0686	13,599	44,445	0,0839	21,5	78,5	RL-Base
23	greedy-fair	658	20,61	0,0528	40,754	5,634	0,0823	27,9	72,1	Heurística
24	sac-base	658	4,52	0,0119	57,075	24,230	-0,042	5,1	94,9	RL-Base
25	vec	658	1,86	0,0030	144,191	4,843	-0,123	1,2	98,8	Heurística

A Tabela 11 isola o efeito do TOPSIS para cada família de algoritmo, comparando a versão base com a versão aumentada pelo ranqueamento multicritério.

Tabela 11 – Ganho do TOPSIS por família de algoritmo (médias sobre 658 cenários comuns). Δ Taxa e Δ SWQ representam diferença absoluta; $\Delta J(\pi)$ é a diferença na função objetivo.

Algoritmo	Versão	Taxa%		SWQ		J(π)		Destaque
		Base	TOPSIS	Base	TOPSIS	Base	TOPSIS	
SAC	base	4,52	—	0,0119	—	-0,042	—	colapso total (94,9% V2V) +40,7 pp / +1147% SWQ
	topsis	—	45,21	—	0,1484	—	0,2454	
PPO	base	42,32	—	0,1418	—	0,2295	—	100% local +0,3 pp / +0,6% SWQ
	topsis	—	42,66	—	0,1426	—	0,2344	
TD3	base	42,32	—	0,1418	—	0,2295	—	100% local +1,6 pp taxa; JCT -44,5%
	topsis	—	43,91	—	0,1388	—	0,2424	
DQN	base	22,10	—	0,0686	—	0,0839	—	78,5% V2V sem critério +3,8 pp / +45% SWQ
	topsis	—	25,90	—	0,0997	—	0,1161	
Ator-Crítico com Vantagem (<i>Advantage Actor-Critic</i>) (A2C)	base	21,74	—	0,0772	—	0,0785	—	79,7% V2V, energia alta +20,6 pp / +84% SWQ
	topsis	—	42,32	—	0,1418	—	0,2295	

5 CRONOGRAMA DE PESQUISA E PLANO DE PUBLICAÇÃO

Este capítulo detalha o roteiro para a conclusão da pesquisa de doutorado e a estratégia para a disseminação dos resultados através de publicações de alto impacto, considerando o prazo final em março de 2027.

5.1 Cronograma de Pesquisa

As atividades de pesquisa restantes estão estruturadas para garantir a conclusão do modelo *Fuzzy-DRL* e a validação experimental. A Tabela 12 apresenta o cronograma para o período de 2026 a 2027, culminando na defesa da tese.

Tabela 12 – Atividades de pesquisa e cronograma de conclusão (2026–2027).

Atividade / Ano	2026		2027
	S1	S2	Q1
1. Exame de Qualificação	X		
2. Integração com Simulador (NS-3/SUMO)	X	X	
3. Refinamento de Algoritmos	X	X	
4. Avaliação de Desempenho e Coleta de Dados		X	
5. Análise dos Resultados		X	X
6. Escrita da Tese Final		X	X
7. Defesa Final			X

Fonte: Elaborado pelo autor.

5.2 Plano de Publicação

A disseminação das descobertas focará em periódicos e conferências especializadas em redes veiculares, computação de borda e sistemas inteligentes. Os locais planejados priorizados para submissão incluem:

- **Periódicos de Alto Impacto:**

- IEEE Transactions on Intelligent Transportation Systems (T-ITS);
- IEEE Transactions on Vehicular Technology (TVT);
- IEEE Internet of Things Journal.

- **Conferências Relevantes:**

- IEEE Vehicular Technology Conference (VTC);
- IEEE Global Communications Conference (GLOBECOM);
- IEEE International Conference on Communications (ICC).

6 CONCLUSÕES

Este capítulo apresenta as conclusões desta etapa de qualificação. O modelo proposto, combinando o ranqueamento multicritério TOPSIS para normalização de estado e Aprendizado por Reforço Profundo (SAC) para tomada de decisão otimizada, aborda os desafios críticos de escalabilidade e invariância de escala em redes VEC.

As principais contribuições alcançadas até agora incluem:

- A formulação de um estado compacto 6D invariante à escala via TOPSIS;
- O projeto de uma função de recompensa *deadline-aware* que equilibra latência e eficiência energética;
- O desenvolvimento de um simulador de eventos discretos em C++20 para avaliação de desempenho.

6.1 Considerações Finais

Como um documento de qualificação, este trabalho demonstra a viabilidade da abordagem SAC-TOPSIS e fornece uma base teórica e metodológica sólida para a tese final. Os próximos passos envolvem o refinamento do agente e uma comparação extensiva de desempenho com *baselines* do estado da arte. Espera-se que os resultados finais contribuam significativamente para o gerenciamento eficiente de recursos computacionais na próxima geração de sistemas de transporte inteligentes.

REFERÊNCIAS

- 3GPP. *Study on evaluation methodology of new V2X use cases for LTE and NR*. [S.l.], 2019.
- 3GPP. *Study on channel model for frequencies from 0.5 to 100 GHz (3GPP TR 38.901 version 16.1.0 Release 16)*. [S.l.], 2020. Release 16.
- ABBAS, Z.; XU, S.; ZHANG, X. An efficient partial task offloading and resource allocation scheme for vehicular edge computing in a dynamic environment. **IEEE Transactions on Intelligent Transportation Systems**, IEEE, 2024.
- ABD-ELNABY, M. *et al.* Comparative analysis of IEEE 802.11bd and 5G NR-V2X for intelligent transportation systems. **Vehicular Communications**, v. 45, p. 100678, 2024.
- AHMED, M.; RAZA, S.; MIRZA, M. A.; AZIZ, A.; KHAN, M. A.; KHAN, W. U.; LI, J.; HAN, Z. A survey on vehicular task offloading: Classification, issues, and challenges. **Journal of King Saud University - Computer and Information Sciences**, v. 34, n. 7, p. 4135–4162, 2022. ISSN 1319-1578. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1319157822001653>.
- ANSARI, K. The 5.9 GHz spectrum for safety-related V2X: The battle between DSRC and C-V2X. **IEEE Access**, v. 8, p. 143161–143171, 2020.
- ASPLUND, H.; ASTELY, D.; BUTOVITSCH, P. von; CHAPMAN, T.; FAXÉR, S.; FRENNE, M.; GHASEMZADEH, F.; HAGSTRÖM, M.; HOGAN, B.; JÖNGREN, G.; KARLSSON, J.; KRONESTEDT, F.; LARSSON, E. **Massive MIMO in Practice: From 5G to 6G**. 2nd. ed. [S.l.]: Academic Press (Elsevier), 2025.
- BAKER, T.; AGHBARI, Z. A.; KHEDR, A.; AHMED, N.; GIRIJA, S. Editors: Energy-aware dynamic task offloading using deep reinforcement and transfer learning for sdn-enabled edge cloud. **Internet of Things**, Elsevier, v. 25, p. 101118, 2024.
- BAO, H.; WANG, Y.; LI, P.; NIE, L.; YU, G.; HUO, Y.; WU, L. Joint task offloading, resource allocation, and service caching in mobile edge computing via soft actor-critic learning. **IEEE Transactions on Vehicular Technology**, IEEE, 2025.
- BOUKERCHE, A.; SOTO, V. Computation offloading and retrieval for vehicular edge computing: Algorithms, models and classification. **ACM Computing Surveys**, v. 53, n. 4, p. 1–35, 2020.
- BROCKMAN, G.; CHEUNG, V.; PETTERSSON, L.; SCHNEIDER, J.; SCHULMAN, J.; TANG, J.; ZAREMBA, W. Openai gym. **arXiv preprint arXiv:1606.01540**, 2016.
- CHEN, C.-T. Extensions of the TOPSIS for group decision-making under fuzzy environment. **Fuzzy Sets and Systems**, v. 114, n. 1, p. 1–9, 2000.
- CHEN, S.; AL. *et.* Vehicle-to-everything (v2x) services: Mode 4 communication in lte-v2x. **IEEE Access**, v. 5, p. 22461–22471, 2017.
- CHEN, S.; LI, W.; SUN, J.; PACE, P.; HE, L.; FORTINO, G. An efficient collaborative task offloading approach based on multi-objective algorithm in mec-assisted vehicular networks. **IEEE Transactions on Vehicular Technology**, 2025.

CHENG, Y.; CHAI, Z.; ANWAR, A. Characterizing co-located datacenter workloads: An alibaba case study. In: **Proceedings of the 9th Asia-Pacific Workshop on Systems**. Jeju Island, Republic of Korea: ACM, 2018. (APSys '18), p. 1–8. Disponível em: <https://doi.org/10.1145/3265723.3265731>.

COX, C. **An Introduction to 5G: The New Radio, 5G Network, 5G Advanced and Beyond**. 2nd. ed. [S.l.]: John Wiley & Sons Ltd, 2025.

CRUZ, L. F. d. **Ciência Sem Limites - 04/08/2014 - Motivação matemática e Lógica Fuzzy**. [S.l.]: TV Unesp, 2014. Entrevista concedida em vídeo.

FAN, Y.; CAI, X. A deep reinforcement approach for computation offloading in mec dynamic networks. **EURASIP Journal on Advances in Signal Processing**, Springer, v. 2024, n. 48, 2024.

FARIMANI, M. K. *et al.* Deadline-aware task offloading in vehicular networks using deep reinforcement learning. **Expert Systems With Applications**, Elsevier, v. 249, p. 123622, 2024.

Federal Communications Commission (FCC). **Use of the 5.850-5.925 GHz Band: Second Report and Order**. [S.l.], 2024. Adopted November 20, 2024. Establishing sunset for DSRC by December 2025.

FENG, J.; LIU, Z.; WU, C.; JI, Y. Ave: Autonomous vehicular edge computing framework with aco-based scheduling. **IEEE Transactions on Vehicular Technology**, IEEE, v. 66, n. 12, p. 10660–10675, 2017.

FOWDUR, T. P.; INDOONUNDON, M.; MILOVANOVIC, D. A.; BOJKOVIC, Z. S. **5G NR Modelling in MATLAB**. [S.l.]: CRC Press, 2025.

GARAI, M.; REKANE, K.; SOUIHI, S.; MELLOUK, A. A comprehensive survey exploring the multifaceted interplay between mobile edge computing and vehicular networks. **Future Internet**, v. 15, n. 12, p. 391, 2023.

GARCIA, M. H. C.; AL. *et.* A tutorial on 5g nr v2x communications. **IEEE Communications Surveys & Tutorials**, v. 23, n. 3, p. 1972–2025, 2021.

HAARNOJA, T. *et al.* Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: **ICML**. [S.l.: s.n.], 2018.

HASSELT, H. van. Double q-learning. **Advances in Neural Information Processing Systems**, v. 23, 2010.

HE, C.; WANG, W.; JIANG, W.; HE, Z.; WANG, J.; XIE, X. Security for the internet of vehicles with integration of sensing, communication, computing, and intelligence: A comprehensive survey. **Sensors**, v. 25, n. 16, 2025. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/25/16/5119>.

HOU, X.; LI, Y.; CHEN, M.; WU, D.; JIN, D.; CHEN, S. Vehicular fog computing: A viewpoint of vehicles as the infrastructures. **IEEE Transactions on Vehicular Technology**, IEEE, v. 65, n. 6, p. 3860–3873, 2016.

HU, M. **The Art of Reinforcement Learning: Fundamentals, Mathematics, and Implementations with Python**. [S.l.]: Apress, 2024.

HUANG, X.; YE, Q.; LI, J.; WU, Y. Deep reinforcement learning for offloading and resource allocation in vehicular edge computing. **IEEE Transactions on Vehicular Technology**, v. 69, n. 8, p. 8334–8345, 2020.

HWANG, C.-L.; YOON, K. **Multiple Attribute Decision Making: Methods and Applications**. Berlin, Heidelberg: Springer-Verlag, 1981.

ITU-R. **IMT Vision - Framework and overall objectives of the future development of IMT for 2020 and beyond**. Geneva, Switzerland, 2015.

ITU-R. **Minimum requirements related to technical performance for IMT-2020 radio interface(s)**. Geneva, Switzerland, 2017.

ITU-R. **Detailed specifications of the terrestrial radio interfaces of International Mobile Telecommunications-2020 (IMT-2020)**. Geneva, Switzerland, 2021.

JANSEN, M.; AL-DULAIMY, A.; PAPADOPOULOS, A. V.; TRIVEDI, A.; IOSUP, A. **The SPEC-RG Reference Architecture for the Compute Continuum**. [S.l.], 2022.

KANDALI, K.; NOUH, S.; BENNIS, L.; BENNIS, H. A comprehensive survey on vanet–iot integration toward the internet of vehicles: Architectures, communications, and system challenges. **Data and Metadata**, v. 4, p. 521, 2025.

KESHARI, N.; SINGH, D.; MAURYA, A. K. A survey on vehicular fog computing: Current state-of-the-art and future directions. **Vehicular Communications**, v. 38, p. 100512, 2022. Disponível em: <https://doi.org/10.1016/j.vehcom.2022.100512>.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.

KUMAR, A.; PAL, A. A survey on computation offloading and current trends. **IEEE Access**, v. 13, p. 185318–185356, 2025.

KUMAR, A. S.; ZHAO, L.; FERNANDO, X. Task offloading and resource allocation in vehicular networks: A lyapunov-based deep reinforcement learning approach. **IEEE Transactions on Vehicular Technology**, IEEE, v. 72, 2023.

KUMARI, S. *et al.* Task offloading in vehicular edge computing network using deep q-network. **International Journal of Intelligent Engineering and Systems**, v. 18, n. 9, p. 659–672, 2025.

LI, X.; PU, Y.; YAN, C.; DUAN, W.; ZHANG, X. Computation offloading and association in mec-assisted v2x network for delay minimization and collision avoidance via deep q-network. **Discover Computing**, Springer, v. 28, n. 302, 2025.

LI, Z.; ZHU, Q. Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing. **Information**, MDPI, v. 11, n. 2, p. 83, 2020.

LIU, L.; CHEN, C.; PEI, Q.; MAHARJAN, S.; ZHANG, Y. Vehicular edge computing and networking: A survey. **Mobile Networks and Applications**, v. 26, n. 3, p. 1145–1168, jun. 2021. ISSN 1572-8153. Disponível em: <https://doi.org/10.1007/s11036-020-01624-1>.

LIU, Z.; SU, J.; WEI, J.; CHEN, W.; CHAN, K. Y.; YUAN, Y.; GUAN, X. Joint robust power control and task scheduling for vehicular offloading in cloud-assisted mec networks. **IEEE Transactions on Network Science and Engineering**, 2024.

LONG, Y.; WANG, Z.; LAN, S.; ZHANG, R.; XU, K. Energy-latency tradeoff for task offloading and resource allocation in vehicular edge computing. **Computer Networks**, Elsevier, v. 258, p. 111026, 2025.

LOPEZ, P. A.; BEHRISCH, M.; BIEKER-WALZ, L.; ERDMANN, J.; FLÖTTERÖD, Y.-P.; HILBRICH, R.; LÜCKEN, L.; RUMMEL, J.; WAGNER, P.; WIESSNER, E. Microscopic traffic simulation using sumo. In: **The 21st IEEE International Conference on Intelligent Transportation Systems**. [S.l.]: IEEE, 2018. p. 2575–2582.

LUO, Q.; LUAN, T. H.; SHI, W.; FAN, P. Heterogeneous task oriented data scheduling in vehicular edge computing via deep reinforcement learning. **IEEE Transactions on Vehicular Technology**, v. 73, n. 12, p. 19582–19596, 2024.

MA, R.; ZHOU, J.; WANG, R.; CHEN, H.-H.; LIU, G. Computation offloading and resource allocation in vehicular mec: A parameterized deep reinforcement learning approach. **IEEE Transactions on Vehicular Technology**, IEEE, 2025.

MACH, P.; BECVAR, Z. Mobile edge computing: A survey on architecture and computation offloading. **IEEE Communications Surveys & Tutorials**, v. 19, n. 3, p. 1628–1656, 2017.

MAO, Y.; YOU, C.; ZHANG, J.; HUANG, K.; LETAIEF, K. B. A survey on mobile edge computing: The communication perspective. **IEEE Communications Surveys & Tutorials**, v. 19, n. 4, p. 2322–2358, 2017.

MEGHAMALA, Y.; PAUL, P. J.; KUMAR, M. A. A multi-agent deep reinforcement learning framework for mec resource allocation in 5g networks. **Electrical and Electronics Research (IJEER)**, v. 13, n. 4, p. 909–919, 2025.

MIRIYALA, S.; CHIRRA, V. R. Deep learning approaches for computation offloading in edge computing: A critical review. **Telecommunication Systems**, Springer, v. 89, p. 42, 2026.

MNIH, V. *et al.* Human-level control through deep reinforcement learning. **Nature**, v. 518, n. 7540, p. 529–533, 2015.

MOERLAND, T. M.; BROEKENS, J.; PLAAT, A.; JONKER, C. M. Model-based reinforcement learning: A survey. **Foundations and Trends® in Machine Learning**, Now Publishers, v. 16, n. 1, p. 1–118, 2023.

MOGHADDASI, K.; RAJABI, S.; GHAREHCHOPOGH, F. S. Multi-objective secure task offloading strategy for blockchain-enabled iov-mec systems: A double deep q-network approach. **IEEE Access**, 2024.

MOGHADDASI, K.; RAJABI, S.; GHAREHCHOPOGH, F. S.; HOSSEINZADEH, M. An energy-efficient data offloading strategy for 5g-enabled vehicular edge computing networks using double deep q-network. **Wireless Personal Communications**, v. 133, n. 3, p. 2019–2064, 12 2023. ISSN 1572-834X. Disponível em: <https://doi.org/10.1007/s11277-024-10862-5>.

MOHAMMAD, A. A. S.; AL-HAWARY, S. I. S.; HINDIEH, A.; VASUDEVAN, A.; AL-SHORMAN, H. M.; AL-ADWAN, A. S.; ALSHURIDEH, M. T.; ALI, I. Intelligent data-driven task offloading framework for internet of vehicles using edge computing and reinforcement learning. **Data and Metadata**, v. 4, p. 521, 2025.

MORAIS, D. H. **5G/5G-Advanced, Wi-Fi 6/7, and Bluetooth 5/6: A Primer on Smartphone Wireless Technologies**. 2nd. ed. [S.l.]: Springer Nature, 2025.

NAIK, G.; CHOUDHURY, B.; PARK, J.-M. J. IEEE 802.11bd & 5g nr v2x: Evolution of radio access technologies for v2x communications. **IEEE Access**, v. 7, p. 70113–70128, 2019.

NAIK, G. *et al.* Next generation vehicular communications: From DSRC to V2X. **IEEE Access**, v. 7, p. 142934–142947, 2019.

NG, A. Y.; HARADA, D.; RUSSELL, S. Policy invariance under reward transformations: Theory and application to reward shaping. In: MORGAN KAUFMANN. **Proceedings of the 16th International Conference on Machine Learning (ICML)**. [S.l.], 1999. p. 278–287.

NIETO, G.; IGLESIA, I. de la; LOPEZ-NOVOA, U.; PERFECTO, C. Deep reinforcement learning techniques for dynamic task offloading in the 5g edge-cloud continuum. **Journal of Cloud Computing**, v. 13, n. 1, p. 94, 2024. ISSN 2192-113X. Disponível em: <https://doi.org/10.1186/s13677-024-00658-0>.

OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C. L.; MISHKIN, P.; ZHANG, C.; AGARWAL, S.; SLAMA, K.; RAY, A.; SCHULMAN, J.; HILTON, J.; KELTON, F.; MILLER, L.; SIMENS, M.; ASKELL, A.; WELINDER, P.; CHRISTIANO, P.; LEIKE, J.; LOWE, R. **Training language models to follow instructions with human feedback**. 2022.

PARRA, O. J. S. *et al.* The evolution of VANET: A review of emerging trends in artificial intelligence and software-defined networks. **IEEE Access**, IEEE, v. 13, p. 49195–49212, 2025.

PTV Group. **PTV Vissim - User Manual**. Karlsruhe, Germany: PTV Planung Transport Verkehr AG, 2020.

QIN, X. *et al.* A survey and taxonomy on task offloading for edge-cloud computing. **IEEE Access**, IEEE, v. 8, 2020.

RAGHAVAN, V.; LI, J.; SAMPATH, A.; KOYMEN, O. H.; LUO, T. (Ed.). **Millimeter Wave Communications in 5G and Towards 6G**. [S.l.]: CRC Press, 2025.

RANI, P.; SHARMA, R. Intelligent transportation system for internet of vehicles based vehicular networks for smart cities. **Computers and Electrical Engineering**, v. 105, p. 108543, 2023. ISSN 0045-7906. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0045790622007583>.

REN, J.; YU, G.; CAI, Y.; HE, Y. Latency optimization for resource allocation in mobile-edge computation offloading. **IEEE Transactions on Wireless Communications**, IEEE, v. 17, n. 8, p. 5506–5519, 2018.

REN, J.; YU, G.; HE, Y.; LI, G. Y. Collaborative cloud and edge computing for latency minimization. **IEEE Transactions on Vehicular Technology**, IEEE, v. 68, n. 5, p. 5031–5044, 2019.

RODRÍGUEZ-PIÑEIRO, J. *et al.* 6g-enabled vehicle-to-everything communications: Current research trends and open challenges. **IEEE Open Journal of Vehicular Technology**, v. 6, p. 1–18, August 2025.

SAE International. **Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles**. Warrendale, PA, USA, 2021. Disponível em: https://www.sae.org/standards/content/j3016_202104/.

SCHUETZ, F. **simcpp20: Discrete-event simulation in C++20 using coroutines**. 2025. Disponível em: <https://github.com/fschuetz04/simcpp20>.

SHEIKH, J. A.; BHAT, Z. A.; MASOODI, I. S. (Ed.). **Radio Frequency, Communications and Signal Processing for 5G and Beyond 5G Networks**. [S.l.]: CRC Press, 2026.

SHI, W.; CHEN, L.; ZHU, X. Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. **Sensors**, v. 23, n. 17, 2023. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/23/17/7595>.

SIDWINI, S. L.; ROUT, S.; PATNAIK, H. K.; DASH, B. B.; DE, U. C.; PATRA, S. S. Task offloading in edge computing: Energy optimization through sac based reinforcement learning. In: . [S.l.: s.n.], 2024. Preprint / Anais de Conferência.

SOFLA, M. S.; KASHANI, M. H.; MAHDIPOUR, E.; MIRZAEI, R. F. Towards effective offloading mechanisms in fog computing. **Multimedia Tools and Applications**, v. 81, p. 1997–2042, 2022.

SOUZA, A. B. D.; REGO, P. A.; CARNEIRO, T.; RODRIGUES, J. D. C.; FILHO, P. P. R.; SOUZA, J. N. D.; CHAMOLA, V.; ALBUQUERQUE, V. H. C. D.; SIKDAR, B. Computation offloading for vehicular environments: A survey. **IEEE Access**, IEEE, v. 8, p. 198214–198243, 2020.

SOUZA, A. B. de; REGO, P. A. L.; CARNEIRO, T.; ROCHA, P. H. G.; SOUZA, J. N. de. A context-oriented framework for computation offloading in vehicular edge computing using wave and 5g networks. **Vehicular Communications**, Elsevier, v. 32, p. 100389, 2021.

STALLINGS, W. **5G Wireless: A Comprehensive Introduction**. [S.l.]: Pearson Education, 2021.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning: An Introduction**. 2nd. ed. [S.l.]: The MIT Press, 2018.

SÁNCHEZ, J. *et al.* The evolution of vehicular communications: From V2V to V2X and beyond. **IEEE Communications Surveys & Tutorials**, v. 27, n. 1, p. 100–145, 2025.

TANG, H.; DU, M.; WU, H.; JIAO, P. Link topology-adaptive offloading method on vehicular edge computing. **IEEE Transactions on Vehicular Technology**, IEEE, 2024.

TIAN, S.; ZHU, X.; FENG, B.; ZHENG, Z.; LIU, H.; LI, Z. Partial offloading strategy based on deep reinforcement learning in the internet of vehicles. **IEEE Transactions on Mobile Computing**, IEEE, 2025.

UDDIN, A.; ZHANG, N.; SAKR, A. H. Intelligent offloading in vehicular edge computing: A comprehensive review of deep reinforcement learning approaches and architectures. **arXiv preprint arXiv:2502.06963v2**, 2025.

VIEIRA, A. S. de S.; SOUZA, A. B. de; JÚNIOR, J. C. Foff: an energy-efficient task offloading in vec-enabled vehicular networks using fuzzy topsis. **Cluster Computing**, Springer, v. 28, n. 13, p. 822, 2025.

WANG, D. *et al.* Iot task offloading in mec: A comprehensive review. **Heliyon**, v. 10, n. e29916, 2024.

- WANG, F.; XU, J.; WANG, X.; CUI, S. Joint offloading and computing optimization in wireless powered mobile-edge computing systems. In: **IEEE International Conference on Communications (ICC)**. [S.l.: s.n.], 2017. p. 1–6.
- WANG, M.; YI, H.; JIANG, F.; LIN, L.; GAO, M. Review on offloading of vehicle edge computing. **Journal of Artificial Intelligence and Technology**, v. 2, n. 4, p. 132–143, 2022. Disponível em: <https://doi.org/10.37965/jait.2022.0120>.
- WANG, T.; OUYANG, X.; SUN, D.; CHEN, Y.; LI, H. Offloading strategy based on graph neural reinforcement learning in mobile edge computing. **Electronics**, MDPI, v. 13, n. 12, p. 2387, 2024.
- WANG, Z.; SCHAUL, T.; HESSEL, M.; HASSELT, H. v.; LANCTOT, M.; FREITAS, N. d. Dueling network architectures for deep reinforcement learning. In: **ICML**. [S.l.: s.n.], 2016.
- WATKINS, C. J. C. H.; DAYAN, P. Q-learning. **Machine learning**, v. 8, n. 3-4, p. 279–292, 1992.
- WEI, R.; QIN, T.; HUANG, J.; YANG, Y.; REN, J.; YANG, L. Resource allocation scheduling scheme for task migration and offloading in 6g cybertwin internet of vehicles based on drl. **IET Communications**, John Wiley & Sons, v. 18, p. 1244–1265, 2024.
- WHAIDUZZAMAN, M.; SOOKHAK, M.; GANI, A.; BUYYA, R. A survey on vehicular cloud computing. **Journal of Network and Computer Applications**, v. 40, p. 325–344, 2014. Disponível em: <https://doi.org/10.1016/j.jnca.2013.08.004>.
- WILLIAMS, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. **Machine learning**, v. 8, n. 3-4, p. 229–256, 1992.
- WU, H.; GU, A.; LIANG, Y. Federated reinforcement learning-empowered task offloading for large models in vehicular edge computing. **IEEE Transactions on Vehicular Technology**, IEEE, v. 74, n. 2, p. 1979–1991, 2025.
- WU, J. *et al.* Efficient gan-based federated optimization for vehicular task offloading with mobile edge computing in 6g network. **IEEE Internet of Things Journal**, Feb 2025.
- XIE, Y. *et al.* Efficient task offloading in double roadside ris-assisted vehicular edge computing networks using deep reinforcement learning. **IEEE Transactions on Intelligent Transportation Systems**, Aug 2025.
- YOU, Q.; TANG, B. Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. **Journal of Cloud Computing**, Springer, v. 10, n. 1, p. 41, 2021.
- ZADEH, L. A. Fuzzy sets. **Information and Control**, Elsevier, v. 8, n. 3, p. 338–353, 1965.
- ZHANG, K.; MAO, Y.; LENG, S.; HE, L.; ZHANG, Y.; PENG, X.; VINEL, A. Energy-efficient offloading for mobile edge computing in 5g networks. **IEEE Wireless Communications**, v. 25, n. 2, p. 20–27, 2018.
- ZHANG, S.; YI, N.; MA, Y. A survey of computation offloading with task types. **IEEE Transactions on Intelligent Transportation Systems**, IEEE, v. 25, n. 8, p. 8313–8333, 2024.

ZHANG, Y.; DU, P.; WANG, J.; BA, T.; DING, R.; XIN, N. Resource scheduling for delay minimization in multi-server cellular edge computing systems. **IEEE Access**, IEEE, v. 7, p. 86265–86273, 2019.

ZHANG, Y. *et al.* A review of the current task offloading algorithms, strategies and approach in edge computing systems. **Computer Modeling in Engineering & Sciences**, v. 134, n. 1, p. 35–88, 2023.

ZHAO, L.; DONG, X.; HAWBANI, A.; BI, Y.; HE, Q.; LIU, Z. Optimizing task offloading in vec: A pdqkm scheme combining deep reinforcement learning and kuhn-munkres matching. **IEEE Transactions on Intelligent Transportation Systems**, IEEE, 2025.

ZHAO, S. **Mathematical Foundations of Reinforcement Learning**. [S.l.]: Tsinghua University Press and Springer, 2024.

ZHENG, K.; ZHENG, Q.; CHATZIMISIOS, P.; XIANG, W.; ZHOU, Y. Heterogeneous vehicular networking: A survey on architecture, challenges, and solutions. **IEEE Communications Surveys & Tutorials**, v. 17, n. 4, p. 2377–2396, 2015.

ZHOU, Z.; CHEN, X.; LI, E.; ZENG, L.; LUO, K.; ZHANG, J. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. **Proceedings of the IEEE**, v. 107, n. 8, p. 1738–1762, 2019.

APÊNDICE A – CONVERSÃO DE ESCALA DE TEMPO DO *TRACE* ALIBABA

O *trace* de carga de trabalho do Alibaba (CHENG; CHAI; ANWAR, 2018) registra os instantes de chegada das tarefas em segundos absolutos a partir do início da coleta, cujos valores são da ordem de 10^5 s (equivalente a dezenas de horas corridas). Para utilizar esse *trace* em uma simulação veicular com janela temporal de 600 s, foram realizadas três etapas de conversão.

Etapla 1 — Seleção de janela. Uma janela contígua de 600 s é extraída do *trace* a partir de um instante $t_{\text{início}}$ escolhido de modo que a taxa média de chegada de tarefas na janela corresponda à carga desejada (e.g., ≈ 30 tarefas/s). A duração de 600 s alinha-se ao horizonte temporal da mobilidade gerada pelo Simulation of Urban MObility (SUMO).

Etapla 2 — Normalização do instante de chegada. O instante de chegada de cada tarefa j no simulador é obtido subtraindo o início da janela:

$$t_{\text{veicular},j} = t_{\text{Alibaba},j} - t_{\text{início}}, \quad (\text{A.1})$$

mapeando os instantes para o intervalo $[0, 599]$ s.

Etapla 3 — Derivação dos requisitos computacionais. A quantidade de ciclos de CPU necessários para executar a tarefa j é calculada como:

$$C_j = N_{\text{inst},j} \times \text{plan_cpu}_j \times \alpha \cdot \Delta t_j \times F_{\text{ref}}, \quad (\text{A.2})$$

onde $N_{\text{inst},j}$ é o número de instâncias paralelas, plan_cpu_j é a alocação de CPU normalizada pelo *trace*, $\Delta t_j = t_{\text{fim},j} - t_{\text{início},j}$ é a duração original no *datacenter* (em segundos), $\alpha = 0,01$ é o fator de compressão temporal e $F_{\text{ref}} = 25$ MHz é a frequência de referência de normalização. O fator α comprime 100 vezes o tempo de execução original, de modo que tarefas que levavam segundos no *cluster* Alibaba passam a demandar dezenas de milissegundos no VEC, compatíveis com as restrições de latência do cenário veicular.

Exemplo numérico

A Tabela 13 ilustra a conversão para uma tarefa extraída do cenário `cenario_v2x_30tps` (janela com $t_{\text{início}} = 205.200$ s).

O valor de $C_j = 62,5 \times 10^6$ ciclos representa a demanda computacional que o VEC (operando a $F_{\text{VEC}} = 1,2$ GHz) processaria em aproximadamente 52 ms, compatível com o prazo gerado para essa tarefa.

Tabela 13 – Exemplo de conversão de escala de tempo para uma tarefa do *trace* Alibaba.

Grandeza	Valor original	Valor convertido
Instante de chegada (t_{Alibaba})	205.201 s	$t_{\text{veicular}} = 205.201 - 205.200 = 1,0$ s
Duração no <i>datacenter</i> (Δt)	5 s	$\text{proc_time} = 5 \times 0,01 = 0,05$ s = 50 ms
Instâncias / alocação de CPU ($N \times \text{plan_cpu}$)	1×50	—
Ciclos computacionais (C_j)	—	$1 \times 50 \times 0,05 \times 25 \times 10^6 = 62,5 \times 10^6$ ci